

CHAPTER

Parallel Programming

13.1	Introduction to Parallel Processing	583
13.2	Parallel Processing and Programming Languages	587
13.3	Threads	595
13.4	Semaphores	604
13.5	Monitors	608
13.6	Message Passing	615
13.7	Parallelism in Non-Imperative Languages	622

The idea of **parallel processing**, or executing many computations in parallel, has been around at least since the 1960s, when the first **multiprogramming** became available. In these early systems, many processes shared a single processor, thus appearing to execute simultaneously. Pseudoparallelism of this sort represented a considerable advance in computer technology, and it is still the standard way most larger machines operate today. However, it has long been clear that **true parallelism** represents an even greater advance in computer technology; in fact, it is one way of solving the problem of the von Neumann bottleneck (see Chapter 1). In true parallelism, many processors are connected to run in concert, either as a single system incorporating all the processors (a **multiprocessor system**) or as a group of stand-alone processors connected together by high-speed links (a **distributed system**).

However, this seemingly simple idea has not been simple to put into practice, and truly parallel systems, in which a single process can share different processors, are even at the time of this writing relatively uncommon, despite extensive study of the issues involved.¹ Thus, a comprehensive study of parallel processing is beyond the scope of this text.

The situation has become even more complex with the advent of high-speed networks, including the Internet, in that physically distant computers are now capable of working together, using the same or similar programs to handle large problems. Thus, organized networks of independent computers can also be viewed as a kind of distributed system for parallel processing, and, indeed, may have already become more important than traditional styles of parallel computing.² Obviously, many of the issues involved in networked parallel processing are also beyond the scope of this text. (See the Notes and References for networking texts that provide more information.)

Nevertheless, programming languages have affected and been affected by the parallelism in a number of ways. First, programming languages have been used to express algorithms to solve the problems presented by parallel processing systems. Second, programming languages have been used to write operating systems that have implemented these solutions on various architectures. Also, programming languages have been used to harness the capabilities of multiple processors to solve application problems efficiently. Finally, programming languages have been used to implement and express communication across networks.

A survey of the principles and practice of programming language design would therefore not be complete without a study of the basic ways that programmers have used programming languages to express parallelism. In this study, we must also distinguish between the parallelism as expressed in a

¹Larger computers often have multiple processors, but it is still rare for a single process to have access to more than one processor at a time.

²Indeed, the Internet can be viewed as a kind of gigantic parallel computer, and several initiatives have successfully used the Internet in that way; see the Notes and References.

programming language and the parallelism that actually exists in the underlying hardware. Programs that are written with parallel programming constructs do not necessarily result in actual parallel processing, but could simply be implemented by pseudoparallelism, even in a system with multiple processors. Thus, parallel programming is sometimes referred to as **concurrent programming** to emphasize the fact that parallel constructs express only the **potential** for parallelism, not that parallel processing actually occurs (which is decided by the architecture of the particular system, the operating system, and the pragmatics of the translator interface). However, we will suppress this distinction and will refer to concurrent programming as parallel programming, without implying that parallel processing must occur.

In this chapter, we briefly survey the basic concepts of parallelism, without which an understanding of language issues is impossible. We then survey basic language issues and introduce the standard approaches to parallelism taken by programming language designers. These include threads, semaphores and their structured alternative, the monitor; and message passing. Java and Ada are used for most of the examples. Finally, a brief look is taken at some ways of expressing parallelism in functional and logical programming languages.

13.1 Introduction to Parallel Processing

The fundamental notion of parallel processing is that of the **process**: It is the basic unit of code executed by a processor. Processes have been variously defined in the literature, but a simple definition is the following:

A process is a program in execution.

This is not quite accurate, because processes can consist of parts of programs as well as whole programs, more than one process can correspond to the same program, and processes do not need to be currently executing to retain their status as processes. A better definition might be the following:

A process is an instance of a program or program part that has been scheduled for independent execution.

Processes used to be called **jobs**, and in the early days of computing, jobs were executed in purely sequential, or **batch** fashion. Thus, there was only one process in existence at a time, and there was no need to distinguish between processes and programs.

With the advent of pseudoparallelism, several processes could exist simultaneously in one of three states. A process could be **executing** (that is, in possession of the processor), it could be **blocked** (that is, waiting for some activity such as input-output to finish, or some event such as a keypress to occur), or it could be **waiting** for the processor to execute it.³ In such a system the operating system needs to apply some algorithm to schedule processes for execution, and to manage a data structure, usually a queue, to maintain waiting and blocked processes. It also needs a method to cause processes to relinquish the processor, or timeout. The principal method for accomplishing this is the **hardware interrupt**.

With the existence of multiple processes, a distinction can also be made between heavyweight processes and lightweight processes. A **heavyweight process** corresponds to the earlier notion of a

³This is a simplified description. Actual operating systems have a more complex state structure.

program in execution. It exists as a full-fledged independent entity, together with all of the memory and other resources that are ordinarily allocated by the operating system to an entire program. A **lightweight process**, on the other hand, shares its resources with the program it came from: It does not have an independent existence except insofar as it executes a particular sequence of instructions from its parent program independently of other execution paths, while sharing all memory and other resources. A lightweight process is also called a **thread**. Lightweight processes can be particularly efficient, since there is less overhead in their creation and management by the operating system.

In a true parallel processing system, where several processors are available, the notion of process and process state is retained much as in the pseudoparallel system. The complication is that each processor may individually be assigned to a process, and a clear distinction must be made between process and processor. Each processor may or may not be assigned its own queues for maintaining blocked and waiting processes.

The organization of the different processors is a critical issue to the operation of processes in a parallel processing system. Two primary requirements for the organization of the processors are

1. There must be a way for processors to synchronize their activities.
2. There must be a way for processors to communicate data among themselves.

For example, in a typical situation, one processor will be handling the input and sorting of data, while a second processor performs computations on the data. The second processor must not begin processing data before it is sorted by the first processor. This is a synchronization problem. Also, the first processor needs to communicate the actual sorted data to the second processor. This is a communication problem.

In some machines one processor is designated as a controller, which manages the operation of the other processors. In some cases this central control extends even to the selection of instructions, and all the processors must execute the same instructions on their respective registers or data sets. Such systems are called **single-instruction, multiple-data (SIMD) systems** and are by their nature multiprocessing rather than distributed systems. Such systems are also often synchronous, in that all the processors operate at the same speed and the controlling processor determines precisely when each instruction is executed by each processor. This implicitly solves the synchronization problem.

In other architectures all the processors act independently. Such systems are called **multiple-instruction, multiple-data**, or **MIMD systems** and may be either multiprocessor or distributed processor systems. In an MIMD system the processors may operate at different speeds, and therefore such systems are asynchronous. Thus, the synchronization of the processors in an MIMD system becomes a critical problem.

Hybrid systems are also possible, with each processor retaining some, but not complete, independence from the other processors. The difference between an SIMD and an MIMD system is illustrated in Figure 13.1.

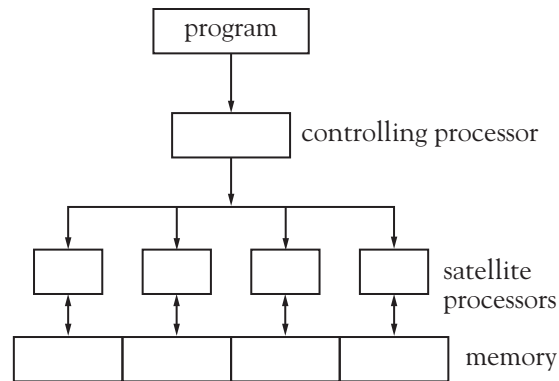


Figure 13.1a Schematic of an SIMD processor

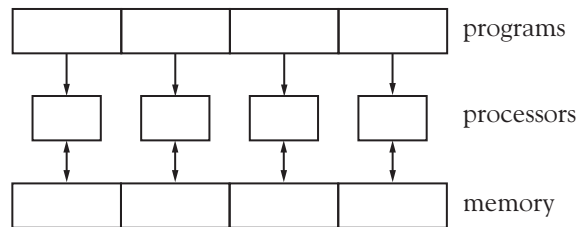


Figure 13.1b Schematic of an MIMD processor

Just as instructions are shared in an SIMD system, in a MIMD system, memory may be shared. A system in which one central memory is shared by all the processors is called a **shared-memory system** and is also by nature a multiprocessor rather than a distributed system, while a system in which each processor has its own independent memory is called a **distributed-memory system** (and may be either an actual distributed system or a multiprocessor system).

In a shared-memory system the processors communicate through the changes each makes to the shared memory. If the processors operate asynchronously, they may also use the shared memory to synchronize their activities. For this to work properly, each processor must have exclusive access to those parts of memory that it is changing. Without mutual exclusion, different processes may be modifying the same memory locations in an interleaved and unpredictable way, a situation that is sometimes called a **race condition**. Solving the mutual exclusion problem typically means blocking processes when another process is already accessing shared data using some kind of locking mechanism. This can cause a new problem to arise, namely, **deadlock**, in which processes end up waiting forever for each other to unblock. Detecting or preventing deadlock is typically one of the most difficult problems in parallel processing.

Distributed-memory systems do not have to worry about mutual exclusion, since each processor has its own memory inaccessible to the other processors. On the other hand, distributed processors have a **communication problem**, in that each processor must be able to send messages to and receive messages from all the other processors asynchronously. Communication between processors depends on the configuration of links between the processors. Sometimes processors are connected in sequence, and processors may need to forward information to other processors farther along the link. If the

number of processors is small, each processor may be fully linked to every other processor. Note that communicating processes may also block while waiting for a needed message, and this, too, can cause deadlock. Indeed, solving deadlock problems in a distributed system can be even more difficult than for a shared memory system, because each process may have little information about the status of other processes. For example, if another process is running on another computer connected to a network, and that computer crashes, it may be difficult to learn anything about what happened.

Again, it is possible for a system to be a hybrid between a shared memory and a distributed-memory system, with each processor maintaining some private memory in addition to the shared memory and the processors having some communication links separate from the shared memory. Figure 13.2 illustrates the difference between a shared-memory system and a fully linked distributed-memory system.

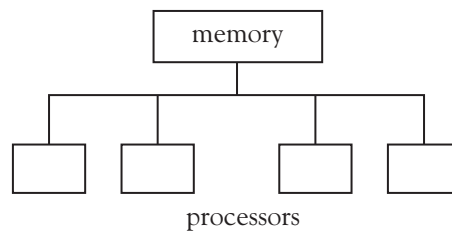


Figure 13.2a Schematic of a shared-memory system

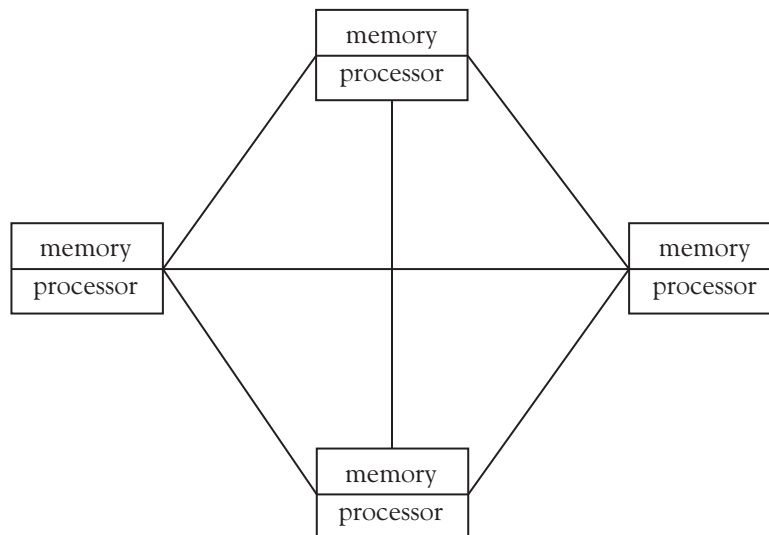


Figure 13.2b Schematic of a fully linked distributed-memory system

Regardless of the organization of the underlying machine, it is the task of the operating system to integrate the operation of the processors and to shield the user from needing to know too much about the particular configuration. The operating system can also change the view the user has of the hardware, depending on the utilities it makes available. For example, the operating system could assign distinct

parts of memory in a shared-memory system for exclusive use by each processor and use other parts of memory to simulate communications channels, thus making a shared-memory system appear to be a distributed-memory system. The operating system can even shield the user entirely from the fact that there is more than one processor and schedule users or processes on different processors automatically, according to an internal algorithm that attempts to allocate resources in an efficient way. However, automatic allocation of multiple processors is almost always less than optimal, and there is usually a need for operating systems to provide facilities for users to manage processes and processors manually. In general, an operating system will need to provide:

1. A means of creating and destroying processes.
2. A means of managing the number of processors used by processes (for example, a method of assigning processes to processors or a method for reserving a number of processors for use by a program).
3. On a shared-memory system, a mechanism for ensuring mutual exclusion of processes to shared memory. Mutual exclusion is used for both process synchronization and communication.
4. On a distributed-memory system, a mechanism for creating and maintaining communication channels between processors. These channels are used both for interprocess communication and for synchronization. A special case of this mechanism is necessary for a network of loosely connected independent computers, such as computers cooperating over the Internet.

In the next section, we will see how similar facilities must be provided by programming languages to make parallel processing available to programmers.

13.2 Parallel Processing and Programming Languages

Programming languages are like operating systems in that they need to provide programmers with mechanisms for process creation, synchronization, and communication. However, a programming language has stricter requirements than an operating system. Its facilities must be machine-independent and must adhere to language design principles such as readability, writability, and maintainability. Nevertheless, most programming languages have adopted a particular model of parallel organization in providing parallel facilities. Thus, some languages use the shared-memory model and provide facilities for mutual exclusion, typically by providing a built-in thread mechanism or thread library, while others assume the distributed model and provide communication facilities. A few languages have included both models, the designers arguing that sometimes one model and sometimes the other will be preferable in particular situations.

A language designer can also adopt the view that parallel mechanisms should not be included in a language definition at all. In that case, the language designer can still provide parallel facilities in other ways. Since this is the easiest approach to parallelism (from the point of view of the language designer), we will study this first. Also in this section, we will consider some approaches to process creation and destruction, since these are (more or less) common to both the shared-memory and the distributed models of parallelism. Model-specific facilities will be left to later sections in this chapter.

In particular, Section 13.3 will discuss **threads**, especially the Java thread implementation. Sections 13.4 and 13.5 will discuss **semaphores** and **monitors**, two approaches to the shared-memory model. Section 13.6 will study **message passing**, a mechanism that follows the distributed model, with examples from Ada. That section will also briefly mention a few of the issues surrounding the network model (which is a kind of distributed model).

We will also need a number of standard problems in parallel processing to demonstrate the use of particular mechanisms. We will use the following two problems throughout the remainder of this chapter (other problems are discussed in the exercises):

1. *The bounded buffer problem.* This problem assumes that two or more processes are cooperating in a computational or input-output situation. One (or more) process produces values that are consumed by another process (or processes). An intermediate buffer, or buffer process, stores produced values until they are consumed. A solution to this problem must ensure that no value is produced until there is room to store it and that a value is consumed only after it has been produced. This involves both communication and synchronization. When the emphasis is not on the buffer, this problem is called the *producer-consumer problem*.
2. *Parallel matrix multiplication.* This problem is different from the previous one in that it is an example of an algorithmic application in which the use of parallelism can cause significant speedups. Matrices are essentially two-dimensional arrays, as in the following declaration of integer matrices (we consider only the case where the size of both dimensions is the same, given by the positive integer constant N):

```
typedef int Matrix [N][N];
Matrix a,b,c;
```

The standard way of multiplying two matrices a and b to form a third matrix c is given by the following nested loops:

```
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
        c[i][j] = 0;
        for (k = 0; k < N; k++)
            c[i][j] = c[i][j] + a[i][k] * b[k][j];
```

This computation, if performed sequentially, takes N^3 steps. If, however, we assign a process to compute each $c[i][j]$, and if each process executes on a separate processor, then the computation can be performed in the equivalent of N steps. Algorithms such as this are studied extensively in courses on parallel algorithms. In fact, this is the simplest form of such an algorithm, since there are no write conflicts caused by the computation of the $c[i][j]$ by separate processes. Thus, there is no need to enforce mutual exclusion in accessing the matrices as shared memory. There is, however,

a synchronization problem in that the product *c* cannot be used until all the processes that compute it have finished. A programming language, if it is to provide useful parallelism, must provide facilities for implementing such algorithms with a minimum of overhead.

13.2.1 Parallel Programming Without Explicit Language Facilities

As we have noted, one possible approach to parallelism is simply not to express it explicitly at all in the language. This is easiest in functional, logic, and object-oriented languages, which have a certain amount of inherent parallelism implicit in the language constructs. (We have mentioned this before, but we will review it again in Section 13.7.)

In theory it is possible for language translators, using optimization techniques, to automatically use operating system utilities to assign different processors to different parts of a program. However, as with operating systems, the automatic assignment of processors is likely to be suboptimal, and manual facilities are needed to make full use of parallel processors. In addition, a programming language may be used for purposes that require explicitly indicating the parallelism, such as the writing of operating systems themselves or the implementation of intricate parallel algorithms.

A second alternative to defining parallel constructs in a programming language is for the translator to offer the programmer **compiler options** to allow the explicit indicating of areas where parallelism is called for. This is usually better than automatic parallelization. One of the places where this is most effective is in the use of nested loops, where each repetition of the inner loop is relatively independent of the others. Such a situation is the matrix multiplication problem just discussed.

Figure 13.3 shows an example of a parallel loop compiler option in FORTRAN code for the matrix multiplication problem. The example is for a FORTRAN compiler on a Sequent parallel computer. The compiler option is:

```
C$doacross share(a, b, c), local(j, k)
```

that causes a preprocessor to insert code that parallelizes the outer loop. The `share` and `local` declarations indicate that *a*, *b*, and *c* are to be accessed by all processes, but that *j* and *k* are local variables to each process. The call to `m_set_procs` sets the number of processes (and processors) that are to be used, returning an error code if not enough processors are available. (Note that in the example the number of processes is 10, far below the number needed to compute optimally the matrix product; see Exercise 13.4.) The call to `m_kill_procs` synchronizes the processes, so that all processes wait for the entire loop to finish and that only one process continues to execute after the loop.

```
integer a(100, 100), b(100, 100), c(100, 100)
integer i, j, k, numprocs, err
numprocs = 10
C code to read in a and b goes here
err = m_set_procs(numprocs)
C$doacross share(a, b, c), local(j, k)
  do 10 i = 1,100
    do 10 j = 1,100
```

Figure 13.3 FORTRAN compiler options for parallelism (*continues*)

(continued)

```

        c(i, j) = 0
        do 10 k = 1,100
            c(i, j) = c(i, j) + a(i, k) * b(k, j)
10 continue
        call m_kill_procs
C code to write out c goes here
    end

```

Figure 13.3 FORTRAN compiler options for parallelism

A third way of making parallel facilities available without explicit mechanisms in the language design is to provide a library of functions to perform parallel processing. This is a way of passing the facilities provided by an operating system directly to the programmer. This way, different libraries can be provided, depending on what facilities an operating system or parallel machine offers. Of course, if a standard parallel library is required by a language, then this is the same as including parallel facilities in the language definition.

Figure 13.4 is an example in C where library functions are used to provide parallel processing for the matrix multiplication problem. (C itself has no parallel mechanisms.) We note that the example of the use of a translator option to indicate parallelism (Figure 13.3) also used some of the same library procedures. (This example is also for a Sequent computer, for comparison purposes.)

```

#include <parallel/parallel.h>
#define SIZE 100
#define NUMPROCS 10

shared int a[SIZE][SIZE], b[SIZE][SIZE], c[SIZE][SIZE];

void multiply(){
    int i, j, k;
    for (i = m_get_myid(); i < SIZE; i += NUMPROCS)
        for (j = 0; j < SIZE; j++)
            for (k = 0 ; k < SIZE; k++)
                c[i][j] += a[i][k] * b[k][j];
}

main(){
    int err;
    /* code to read in the matrices a and b goes here */
    m_set_procs(NUMPROCS);
    m_fork(multiply);
    m_kill_procs();
    /* code to write out the matrix c goes here */
    return 0;
}

```

Figure 13.4 Use of a library to provide parallel processing

In Figure 13.4 the four procedures `m_set_procs`, `m_fork`, `m_kill_procs`, and `m_get_myid` are imported from a library (the `parallel/parallel` library). `m_set_procs` and `m_kill_procs` are as in the previous example. `m_fork` creates the 10 processes, which are all instances of the procedure `multiply` (the name `fork` comes from the UNIX operating system, discussed shortly). In procedure `multiply`, `m_get_myid` gets the number of the process instance (from 0 to 9). The remainder of the code then divides the work among the processes, so that process 0 calculates `c[0][i]`, `c[10][i]`, $\dots$, `c[90][i]` for all `i`, process 2 calculates `c[2][i]`, `c[12][i]`, $\dots$, `c[92][i]`, and so on.

A final alternative to introducing parallelism into a language is to simply rely on operating system features directly to run programs in parallel. Essentially, this requires that a parallel program be split up into separate, independently executable pieces and then set up to communicate via operating system mechanisms (thus, allowing only program-level parallelism as described shortly). A typical example of this is to string programs together in a UNIX operating system through the use of **pipes**, which is a method of streaming text input and output from one program to another without storing to intermediate files (which could be inefficient or impossible if these files are large). A simple example is the following, which will list all filenames in the current directory containing the string “java.”⁴

```
ls | grep "java"
```

This command runs the two UNIX utility programs `ls` and `grep` in parallel (`ls` is the directory listing program; `grep` is the “global regular expression print” program that finds string patterns in text). The output of `ls` is piped to `grep` using the pipe symbol (the vertical bar), and becomes the input to `grep`. As a more complex example, the pipe

```
cat *.java | tr -sc A-Za-z '\012' | sort | uniq -c | sort -rn
```

will list all of the words in Java programs in the current directory, with a count of their usage, and sorted in order of decreasing number of uses (we leave as an exercise for the reader the explanation of each of the five programs used in this pipeline).

13.2.2 Process Creation and Destruction

A programming language that contains explicit mechanisms for parallel processing must have a construct for creating new processes. You have seen this informally already in Figure 13.4, where calls to the library procedures `m_set_procs` and `m_fork` together created a fixed number of processes.

There are two basic ways that new processes can be created. One is to split the current process into two or more processes that continue to execute copies of the same program. In this case, one of the processes is usually distinguished as the **parent**, while the others become the **children**. The processes can execute different code by a test of process identifiers or some other condition, but the basic program is the same for all processes. This method of process creation resembles the SIMD organization of Section 13.1 and is, therefore, called **SPMD programming** (for single program multiple data). Note, however, that SPMD programs may execute different segments of their common code and so do not necessarily operate synchronously. Thus, there is a need for process synchronization.

⁴This can actually be condensed to the single command “`ls *java*`” in UNIX.

In the second method of process creation, a segment of code (commonly a procedure) is explicitly associated with each new process. Thus, different processes have different code, and we can call this method **MPMD programming**. A typical case of this is the so-called **fork-join** model, where a process creates several child processes, each with its own code (a fork), and then waits for the children to complete their execution (a join). Unfortunately, the name is confusing, because the UNIX system called `fork()` (studied shortly) is really an SPMD process creator, not a fork-join creator. We will, therefore refer to MPMD process creation rather than a fork-join creation. Note that Figure 13.4 is an example of MPMD programming (with `m_kill_procs` taking the place of the join).

An alternative view of process creation is to focus on the size of the code that can become a separate process. In some designs, individual statements can become processes and be executed in parallel. A second possibility is for procedures to be assigned to processes. This was the case in Figure 13.4, where the procedure `multiply` was assigned to processes via the call `m_fork(multiply)`. A third possibility is for processes to represent whole programs only. Sometimes the different size of the code assignable to separate processes is referred to as the **granularity** of processes. The three choices of constructs for parallel execution that we have just listed could be described as follows:

1. Statement-level parallelism: fine-grained
2. Procedure-level parallelism: medium-grained
3. Program-level parallelism: large-grained

Granularity can be an issue in program efficiency: Depending on the kind of machine, many small-grained processes can incur significant overhead in their creation and management, thus executing more slowly than fewer larger processes. On the other hand, large-grained processes may have difficulty in exploiting all opportunities for parallelism within a program. An intermediate case that can be very efficient is that of a thread, which typically represents fine-grained or medium-grained parallelism without the overhead of full-blown process creation.

Regardless of the method of process creation, it is possible to distinguish between process creator (the parent process) and process created (the child process), and for every process creation mechanism, the following two questions must be answered:

1. Does the parent process suspend execution while its child processes are executing, or does it continue to execute alongside them?
2. What memory, if any, does a parent share with its children or the children share among themselves?

In Figure 13.4, the assumption was that the parent process suspends execution while the child processes compute. It was also necessary to indicate explicitly that the global variables `a`, `b`, and `c` are to be shared by all processes, using the keyword `shared`.

In addition to process creation, a parallel programming language needs a method for process termination. In the simplest case, a process will simply execute its code to completion and then cease to exist. In more complex situations, however, a process may need to continue executing until a certain condition is met and then terminate. It may also be necessary to select a particular process to continue execution.

We will briefly study process creation and destruction mechanisms for each kind of granularity.

13.2.3 Statement-Level Parallelism

A typical construct for indicating that a number of statements can be executed in parallel is the **parbegin-parend** block:⁵

```
parbegin
  S1;
  S2;
  ...
  Sn;
parend;
```

In this statement the statements $S_1, \dots, S_n$ are executed in parallel. It is assumed that the main process is suspended during their execution, and that all the processes of the S_i share all variables not locally declared within an S_i .

An extension of this mechanism is the parallel loop construct of Fortran95, or **forall** construct, which indicates the parallel execution of each iteration of a loop, as in

```
forall(i = 1:100, j = 1:100)
  c(i, j) = 0
  do 10 k = 1,100
    c(i, j) = c(i, j) + a(i, k) * b(k, j)
10  continue
end forall
```

This is similar to the \$doacross compiler option of Figure 13.3.

13.2.4 Procedure-Level Parallelism

In this form of process creation/destruction, a procedure is associated with a process, and the process executes the code of the procedure. Schematically, such a mechanism has the form

```
x = newprocess(p);
...
...
killprocess(x);
```

where p is a declared procedure and x is process designator—either a numeric process number or a variable of type process. Figure 13.4 uses library procedures that create processes essentially this way. An alternative to this is to use declarations to associate procedures to processes:

```
process x(p);
```

⁵This example is taken from the programming language **Occam**, which is based on CSP—see the Notes and References.

Then the scope of `x` can be used as the region where `x` is active: `x` begins execution when the scope of its declaration is entered, and `x` is terminated on exit from its scope (if it has not already executed to completion). This is the method used by Ada in the declaration of tasks and task types, which is discussed more fully in Section 13.5. (The **task** is Ada's term for a process.)

13.2.5 Program-Level Parallelism

In this method of process creation only whole programs can become processes. Typically, this occurs in MPMD style, where a program creates a new process by creating a complete copy of itself. The typical example of this method is the `fork` call of the UNIX operating system. A call to `fork` causes a second child process to be created that is an exact copy of the calling process, including all variables and environment data at the moment of the fork. Processes can tell which is the child and which is the parent by the returned value of the call to `fork`: A zero value indicates that the process is the child, while a nonzero value indicates the process is the parent (the value itself is the process number of the child just created). By testing this value the parent and child processes can be made to execute different code:

```
if (fork() == 0)
    /* ... child executes this part ... */
else
    /* ... parent executes this part ... */
```

After a call to `fork`, a process can be terminated by a call to `exit`. Process synchronization can be achieved by calls to `wait`, which causes a parent to suspend its execution until a child terminates.

Figure 13.5 gives sample C code for parallel matrix multiplication using `fork`, `exit`, and `wait`. In this code, a `fork` is performed for each of the `NUMPROCS` child processes, and a `wait` is performed for each of the child processes as well. For simplicity, this code assumes that global variables are shared by all processes. (Warning! This is not true of processes in a standard UNIX implementation. See Exercises 13.15 and 13.16 for a discussion.)

```
#define SIZE 100
#define NUMPROCS 10

int a[SIZE][SIZE], b[SIZE][SIZE], c[SIZE][SIZE];

main(){
    int myid;
    /* code to input a,b goes here */
    for (myid = 0; myid < NUMPROCS; ++myid)
        if (fork() == 0){
            multiply(myid) ;
```

Figure 13.5 Sample C code for the fork construct (*continues*)

(continued)

```

        exit(0) ;
    }
    for (myid = 0; myid < NUMPROCS; ++myid)
        wait(0) ;
    /* code to output c goes here */
    return 0;
}

void multiply (int myid){
    int i, j, k;
    for (i = myid; i < SIZE; i+= NUMPROCS)
        for (j = 0; j < SIZE; ++j){
            c[i][j] = 0;
            for (k = 0; k < SIZE; ++k)
                c[i][j] += a[i][k] * b[k][j];
        }
}

```

Figure 13.5 Sample C code for the fork construct

13.3 Threads

As we have noted, threads can be an efficient mechanism for fine- or medium-grained parallelism in the shared memory model. Since Java has a widely used thread implementation, we shall study the thread implementation in Java in some detail here, as a good illustration of the principles we have discussed so far. In particular, we will describe process creation, destruction, synchronization, and mutual exclusion in Java, and illustrate these facilities with a variation of the bounded buffer problem.

13.3.1 Threads in Java

Threads are built into the Java language, in that the `Thread` class is part of the `java.lang` package, and the reserved word `synchronize` is used to establish mutual exclusion for threads.⁶ A Java thread is created by instantiating a `Thread` object and by defining a `run` method that will be executed when the thread starts. This can be done in two ways, either by extending `Thread` through inheritance and overriding the (empty) `Thread.run` method,

⁶Actually, Java offers two thread packages, called **green threads** and **native threads**. Green threads (the default) do not use mechanisms provided by the underlying operating system, but all parallelism is managed by the Java Virtual Machine itself. Native threads, on the other hand, use special features of the underlying operating system, and may, therefore, operate more efficiently by taking advantage of system features, including hardware parallelism. Green threads are necessary if threads are to operate securely across an Internet connection.

```

class MyThread extends Thread{

    public void run()
    { ... }

}

...

Thread t = new MyThread();

...

```

or by defining a class that implements the `Runnable` interface (which only requires the definition of a `run` method), and then passing an object of this class to the `Thread` constructor:

```

class MyRunner implements Runnable{

    public void run()
    { ... }

}

...

MyRunner m = new MyRunner();
Thread t = new Thread(m);

...

```

This latter mechanism is more versatile, so we use it in our examples.

Defining a thread does not begin executing it. Instead, a thread begins running when the `start` method is called:

```
t.start(); // now t will execute the run method
```

The `start` method in turn will call `run`. Although one could call the `run` method directly, this will in general not work, since the system must perform some internal bookkeeping before calling `run`. Note also that every Java program already is executing inside a thread whose `run` method is `main`. Thus, when a new thread's execution is begun by calling `start`, the main program will still continue to execute to completion in its own thread (in other words, the `start` method of a thread immediately returns, while the thread itself continues to execute). However, the entire *program* will not finish execution until all of its threads complete the execution of their `run` methods.⁷

⁷There is also an alternative kind of thread in Java, called a **daemon**, which is killed off when the main program exits, regardless of whether it has completed execution. See the Notes and References.

How are threads destroyed? As we have just noted, the simplest mechanism is to let each thread execute its `run` method to completion, at which time it will cease to exist without any programmer intervention. Threads can also wait for other threads to finish before continuing by calling the `join` method on the thread object:

```
Thread t = new Thread(m);
t.start(); // t begins to execute
// do some other work
t.join(); // wait for t to finish
// continue with work that depends on t being finished
```

Thus, Java threads exhibit fork-join parallelism as previously defined (with the fork operation given by the `start` method).

An alternative to waiting for a thread to finish executing is to interrupt it using the `interrupt` method:

```
Thread t = new Thread(m);
t.start(); // t begins to execute
// do some other work
t.interrupt(); // tell t that we are waiting for it
t.join(); // continue to wait
// continue
```

The `interrupt` method, as this code indicates, does not actually stop the interrupted thread from executing, but simply sets an internal flag in the thread object that can be used by the object to test whether some other thread has called its `interrupt` method. This allows a thread to continue to execute some cleanup code before actually exiting. The semantics of the `interrupt` method are somewhat complex, so we defer a fuller discussion temporarily.

Note that, when `t.join()` is called, the current thread (i.e., the thread that makes this call) becomes blocked, and will only unblock once `t` exits its `run` method. If `t` is also in a blocked state waiting for an event, then deadlock can result. If there is a real potential for deadlock, then we could try to avoid it by waiting only a specified amount of time, and then timing out:

```
t.join(1000); // wait 1 second for t, then give up
```

In general, any Java method such as `join` that blocks the current thread has a timeout version such as the above, because the Java runtime system makes no attempt to discover or prevent deadlock. It is entirely up to the programmer to ensure that deadlock cannot occur.

So far we have not discussed how to enforce mutual exclusion for shared data in Java. Even with only the mechanisms described so far, however, Java threads can be useful, and indeed parallel matrix multiplication can already be written (see Exercise 13.17), since there is no need for mutual exclusion.

In general, Java threads will share some memory or other resources, typically by passing the objects to be shared to constructors of the `Runnable` objects that are used to create the threads.⁸ For example, suppose several distinct threads are adding and removing objects using a shared queue:

⁸It is also possible to create inner `Runnable` classes and share data via nonlocal access.

```

class Queue{

    ...
    public Object dequeue(){
        if (empty()) throw EmptyQueueException ;
        ...
    }

    public void enqueue(Object obj) { ... }
    ...
}

class Remover implements Runnable{

    public Remover(Queue q){ ... }
    public void run(){ ... q.dequeue() ... }
    ...
}

class Inserter implements Runnable{

    public Inserter(Queue q){ ... }
    public void run(){ ... q.enqueue(...) ... }
    ...
}

Queue q = new Queue(...);
...
Remover r = new Remover(q);
Inserter i = new Inserter(q);
Thread t1 = new Thread(r);
Thread t2 = new Thread(i);
t1.start();
t2.start();
...

```

Now the queue `q` is shared between threads `t1` and `t2`, and some mechanism for ensuring mutual exclusion within `q` is necessary, so that it is not possible for one thread to be executing `q.dequeue()` while the other is executing `q.enqueue()`. In Java, this is done by using the `synchronized` keyword⁹ in the definition of class `Queue`:

⁹In this text, we only discuss method synchronization. In fact, in Java any block of code can be synchronized on any object for which a local reference exists (synchronized methods implicitly synchronize on the current object `this`). Method synchronization is a special case of block synchronization; a method such as `enqueue` above can be synchronized by the following equivalent code: `public void enqueue(Object obj) { synchronized(this) {...} }`.

```

class Queue{
    ...
    synchronized public Object dequeue(){
        if (empty()) throw EmptyQueueException ;
        ...
    }

    synchronized public void enqueue(Object obj){ ... }
    ...
}

```

In Java, every object has a single **lock** (see Section 13.4) that is available to threads. When a thread attempts to execute a synchronized method of an object, it must first acquire the lock on the object. If the lock is in possession of another thread, it waits until the lock is available, acquires it, and then proceeds to execute the synchronized method. After exiting the method, it releases the lock before continuing its own execution.¹⁰ Thus, no two threads can be simultaneously executing any synchronized methods of the same object. (Unsynchronized methods are not affected.) This solves the preceding mutual exclusion problem.

There is one more thread mechanism in Java that we discuss here before considering an extended example: how to cause threads to wait for certain explicit conditions and then resume execution. For example, suppose that, in the previous queue example, we know that if `dequeue` is ever called, then an object will eventually exist for the calling thread to remove from the queue. Then, it is undesirable to generate an exception on an empty queue—the thread should simply wait for the object to appear in the queue.

If a condition is testable in the code for a synchronized method, a thread executing that method can be manually stalled by calling the `wait()` method of the object (`wait` is a method of class `Object`, so every object has this method). Once a thread is waiting for a certain condition, it also needs to be manually reawakened by another thread (typically, the thread that establishes the desired condition). This is done using either the `notify()` or the `notifyAll()` methods of the object. Note that each object in Java has a wait-list containing all threads that have called `wait` on this object, in some unspecified order. The `notify()` method will wake up only one (arbitrary) thread in this list, while `notifyAll()` will wake up all threads in the list. Typically, `notifyAll()` is used in preference to `notify()`, since it is impossible to predict which thread will be awakened by `notify()`, and a thread in the wait-list may be waiting for a different condition than the one that has just been established.

Now let us reconsider the above queue example. Instead of generating an exception on an empty queue, the `dequeue` method should call `wait`, and the `enqueue` operation should call `notifyAll`, since it makes the queue nonempty:

¹⁰This is the simplest case; in fact, a thread can acquire multiple locks, or even the same lock multiple times; each synchronized exit releases only one “hold” on one lock.

```

(1) class Queue
(2) { ...
(3)     synchronized public Object dequeue()
(4)         { try // wait can generate InterruptedException
(5)             { while (empty()) wait();
(6)                 ...
(7)             }
(8)             catch (InterruptedException e) // reset interrupt
(9)                 { Thread.currentThread().interrupt(); }
(10)        }
(11)    synchronized public void enqueue(Object obj)
(12)        // tell waiting threads to try again
(13)    { // add obj into the queue
(14)        ...
(15)        notifyAll();
(16)    }
(17) }

```

Two things deserve special mention here. First, a while loop is used on line 5 in `dequeue`, because `empty()` should be retested after a wake-up: Another thread may have already emptied the queue before this thread acquires the lock. Second, a call to `wait` can generate an `InterruptedException`, which must be handled or rethrown. This exception will be generated if another thread calls `interrupt` on a thread that is in the wait-queue, causing it to wake up and resume execution as though the exception occurred in the usual way.

We have noted previously that a call to `interrupt` does not actually stop a thread from executing but sets a flag that can be tested by the thread code, so that it may perform cleanup before exiting. However, if a thread is waiting for a lock prior to executing a synchronized method, or is in a wait-list, then it is not executing and so cannot test for an interrupt. This problem is solved by having a call to `interrupt` generate an exception if the thread is not currently executing and simultaneously wake the stalled thread. Also, the interrupt flag is cleared when the exception is generated. Thus, a thread typically will want to test for both the interrupt flag and `InterruptedException`, if it is to handle `InterruptedException` itself. Alternatively (and this is often the simplest), the synchronized method itself can handle the exception, in which case it is reasonable to reset the interrupt flag before exiting, as in line 9, so that the client thread can still discover that it has been interrupted. Then, the client thread code could be as follows:

```

class Remover implements Runnable{
    public Remover(Queue q) { ... }
    public void run(){
        while (!Thread.currentThread().isInterrupted()){
            ... q.dequeue() ...
        }
        // perform some cleanup up
    } // and now exit
    ...
}

```

Note how the interrupt flag is set and tested by calls to the `interrupt` and `isInterrupted` methods of `Thread.currentThread()`. Indeed, `currentThread` is a static method of the `Thread` class (i.e., a class method) and must be called on the `Thread` class object, in which case it returns the current `Thread` object. This is necessary, since the run code inside a `Runnable` class has no way of identifying the actual thread that has been interrupted, but it will always be the *currently executing* thread.

13.3.2 A Bounded Buffer Example in Java

To finish our discussion of Java threads, we discuss in some detail a solution to the bounded buffer problem mentioned in Section 13.2. We will cast this problem as an input-output problem, with the producer reading characters from standard input and inserting them into the buffer, and the consumer removing characters from the buffer and writing them to standard output. To add interest to this example, we set the following additional requirements:

- The producer thread will continue to read until an end of file is encountered, whence it will exit.
- The consumer thread will continue to write until the producer has ended and no more characters remain in the buffer. Thus, the consumer should echo all the characters that the producer enters into the buffer.

The complete code for this Java program is given in Figure 13.6. We make the following explanatory remarks about this code.

```
(1) import java.io.*;

(2) class BoundedBuffer{

(3)     public static void main(String[] args){
(4)         Buffer buffer = new Buffer(5); // buffer has size 5
(5)         Producer prod = new Producer(buffer);
(6)         Consumer cons = new Consumer(buffer);
(7)         Thread read = new Thread(prod);
(8)         Thread write = new Thread(cons);
(9)         read.start();
(10)        write.start();
(11)        try {
(12)            read.join();
(13)            write.interrupt();
(14)        }
(15)        catch (InterruptedException e) {}
(16)    }
(17) }

(18) class Buffer{
(19)     private final char[] buf;
```

Figure 13.6 Java code for a bounded buffer problem (*continues*)

(continued)

```

(20)     private int start = -1;
(21)     private int end = -1;
(22)     private int size = 0;

(23)     public Buffer(int length){
(24)         buf = new char[length];
(25)     }
(26)     public boolean more()
(27)     { return size > 0; }

(28)     public synchronized void put(char ch)
(29)     { try {
(30)         while (size == buf.length) wait();
(31)         end = (end+1) % buf.length;
(32)         buf[end] = ch;
(33)         size++;
(34)         notifyAll();
(35)     }
(36)     catch (InterruptedException e)
(37)     { Thread.currentThread().interrupt(); }
(38)     }

(39)     public synchronized char get()
(40)     { try {
(41)         while (size == 0) wait();
(42)         start = (start+1) % buf.length;
(43)         char ch = buf[start];
(44)         size--;
(45)         notifyAll();
(46)         return ch;
(47)     }
(48)     catch (InterruptedException e)
(49)     { Thread.currentThread().interrupt(); }
(50)     return 0;
(51)     }
(52) }

(53) class Consumer implements Runnable{
(54)     private final Buffer buffer;

(55)     public Consumer(Buffer b)
(56)     { buffer = b;
(57)     }

```

Figure 13.6 Java code for a bounded buffer problem (*continues*)

(continued)

```

(58)     public void run() {
(59)         while (!Thread.currentThread().isInterrupted())
(60)         {   char c = buffer.get();
(61)             System.out.print(c);
(62)         }
(63)         while(buffer.more()) // clean-up
(64)         {   char c = buffer.get();
(65)             System.out.print(c);
(66)         }
(67)     }
(68) }

(69) class Producer implements Runnable{
(70)     private final Buffer buffer;
(71)     private final InputStreamReader in
(72)         = new InputStreamReader(System.in);

(73)     public Producer(Buffer b) { buffer = b; }

(74)     public void run() {
(75)         try {
(76)             while (!Thread.currentThread().isInterrupted()) {
(77)                 int c = in.read();
(78)                 if (c == -1) break; // -1 is end of file
(79)                 buffer.put((char)c);
(80)             }
(81)         }
(82)         catch (IOException e) {}
(83)     }
(84) }

```

Figure 13.6 Java code for a bounded buffer problem

The main program creates a buffer of five characters, a reader (the producer) for the buffer and a writer (the consumer) that prints characters. Two threads are created, one for the producer and one for the consumer (so that three threads in all exist for this program, including the main thread). The main thread then waits for the reader to finish (line 12), and then interrupts the writer (line 13). (Since `join` can, like `wait`, generate `InterruptedException`, we must also handle this exception.)

The bounded buffer class itself uses an array to hold the characters, along with two indices that traverse the array in a circular fashion, keeping track of the last positions where insertions and removals took place (this is a standard implementation technique discussed in many data structures books). Both the `put` (insert) and `get` (remove) operations are synchronized to ensure mutual exclusion. The `put` operation delays a thread if the buffer is full (line 30), while the `get` operation delays a thread if the

buffer is empty (line 41). As a result, both operations must handle the `InterruptedException`. Both also call `notifyAll` after performing their respective operations (lines 34 and 45).

The `Producer` class (lines 69–84) is relatively simple. Its `run` method checks for an interrupt—even though `interrupt` will never be called on its thread in this program, it is reasonable to write general code such as this. Its only other behavior is to check for an end of file (line 78, where `EOF = -1`), whence it exits.

The `Consumer` class is almost symmetrical to the `Producer` class. The major difference is that, when a `Consumer` is interrupted, we want to make sure that the buffer has been emptied before the `Consumer` exits. Thus, on reaching line 63, we know that the producer has finished, so we simply test for more characters in the buffer and write them before exiting.

13.4 Semaphores

A **semaphore** is a mechanism to provide mutual exclusion and synchronization in a shared-memory model. It was first developed by E. W. Dijkstra in the mid-1960s and included in the languages Algol68 and PL/I. A semaphore is a shared integer variable that may be accessed only via three operations: **InitSem**, **Signal**, and **Delay**. The **Delay** operation tests the semaphore for a positive value, decrementing it if it is positive and suspending the calling process if it is zero or negative. The **Signal** operation tests whether processes are waiting, causing one of them to continue if so and incrementing the semaphore if not. (These operations were originally called *P* and *V* by Dijkstra, but we will use the more descriptive names as given; note that **Signal** is analogous to `notify` in Java, and **Delay** is analogous to `wait`.)

Given a semaphore *S*, the **Signal** and **Delay** operations can be defined in terms of the following pseudocode:

Delay(*S*): if $S > 0$ then $S := S - 1$ else suspend the calling process
Signal(*S*): if processes are waiting then wake up a process else $S := S + 1$

Unlike ordinary code, however, the system must ensure that each of these operations executes **atomically**, that is, by only one process at a time. (If two processes were to try to increment *S* at the same time, the actual final value of *S* would be unpredictable; see Exercise 13.52.)

Given a semaphore *S*, we can ensure mutual exclusion by defining a **critical region**, that is, a region of code that can be executed by only one process at a time. If *S* is initialized to 1, then the following code defines such a critical region:

```
Delay(S);  
{ critical region }  
Signal(S);
```

A typical critical region is code where shared data is read and/or updated. Sometimes semaphores are referred to as **locks**, since they lock out processes from critical regions.

Semaphores can also be used to synchronize processes. If, for example, process *p* must wait for process *q* to finish, then we can initialize a semaphore *S* to 0, call *Delay*(*S*) in *p* to suspend its execution, and call *Signal*(*S*) at the end of *q* to resume the execution of *p*.

An important question to be addressed when defining semaphores is the method used to choose a suspended process for continued execution when a call to *Signal* is made. Possibilities include making a random choice, using a first in, first out strategy, or using some sort of priority system. This choice has a major effect on the behavior of concurrent programs using semaphores.

As defined, a semaphore can be thought of as an abstract data type, with some added requirements. Indeed, semaphores can be defined using a Java class definition as given in Figure 13.7.¹¹

```
class Semaphore{

    private int count;

    public Semaphore(int initialCount){
        count = initialCount;
    }

    public synchronized void delay() throws InterruptedException{
        while (count <= 0) wait();
        count--;
    }

    public synchronized void signal(){
        count++;
        notify();
    }

}
```

Figure 13.7 Simplified Java code for a semaphore

This code uses a call to `notify` rather than `notifyAll`, because each semaphore's wait-list is waiting on a single condition—that count should be greater than 0—so that only one waiting thread needs to be awakened at a time.

13.4.1 A Bounded Buffer Using Semaphores

We can write a version of the bounded buffer class that uses semaphores to enforce synchronization and mutual exclusion, as shown in Figure 13.8. (The rest of the program of Figure 13.6 can remain exactly the same.) We note a number of things about this solution.

The solution uses three semaphores. The first, `mutex`, provides mutual exclusion to code that changes the private state of a `Buffer`. The semaphores `nonEmpty` and `nonFull` maintain the status of the number of stored items available. Note that all Java synchronization mechanisms—synchronized methods, calls to `wait` and `notifyAll`—are now gone from the `Buffer` code itself, since they are indirectly supplied by the semaphores.

¹¹This simplified code ignores several Java issues, particularly those surrounding interrupts (as indicated by the `throws InterruptedException` declaration for `delay`); see Exercise 13.28.

```
(1) class Buffer{
(2)     private final char[] buf;
(3)     private int start = -1;
(4)     private int end = -1;
(5)     private int size = 0;
(6)     private Semaphore nonFull, nonEmpty, mutex;

(7)     public Buffer(int length) {
(8)         buf = new char[length];
(9)         nonFull = new Semaphore(length);
(10)        nonEmpty = new Semaphore(0);
(11)        mutex = new Semaphore(1);
(12)    }

(13)    public boolean more()
(14)    { return size > 0; }

(15)    public void put(char ch){
(16)        try {
(17)            nonFull.delay();
(18)            mutex.delay();
(19)            end = (end+1) % buf.length;
(20)            buf[end] = ch;
(21)            size++;
(22)            mutex.signal();
(23)            nonEmpty.signal();
(24)        }
(25)        catch (InterruptedException e)
(26)        { Thread.currentThread().interrupt(); }
(27)    }
(28)    public char get(){
(29)        try{
(30)            nonEmpty.delay();
(31)            mutex.delay();
(32)            start = (start+1) % buf.length;
(33)            char ch = buf[start];
(34)            size--;
(35)            mutex.signal();
(36)            nonFull.signal();
(37)            return ch;
(38)        }
(39)        catch (InterruptedException e)
(40)        { Thread.currentThread().interrupt(); }
(41)        return 0;
(42)    }
(43) }
```

Figure 13.8 The bounded buffer problem using semaphores

13.4.2 Difficulties with Semaphores

The basic difficulty with semaphores is that, even though the semaphores themselves are protected, there is no protection from their incorrect use or misuse by programmers. For example, if a programmer incorrectly writes:

```
Signal(S);
...
Delay(S);
```

then the surrounded code is not a critical region and can be entered at will by any process. On the other hand, if a programmer writes:

```
Delay(S);
...
Delay(S);
```

then it is likely that the process will block at the second *Delay*, never to resume execution. It is also possible for the use of semaphores to cause deadlock. A typical example is represented by the following code in two processes, with two semaphores S_1 and S_2 :

```
Process 1: Delay(S1);
           Delay(S2);
           ...
           Signal(S2);
           Signal(S1);

Process 2: Delay(S2);
           Delay(S1);
           ...
           Signal(S1);
           Signal(S2);
```

If Process 1 executes *Delay*(S_1) at the same time that Process 2 executes *Delay*(S_2), then each will block waiting for the other to issue a *Signal*. Deadlock has occurred.

To remove some of the insecurities in the use of semaphores, the monitor was invented; see Section 13.5.

13.4.3 Implementation of Semaphores

Generally, semaphores are implemented with some form of hardware support. Even on single-processor systems this is not an entirely trivial proposition, since an operating system may possibly interrupt a process between any two machine instructions. One common method for implementing semaphores on a single-processor system is the **TestAndSet** machine instruction, which is a single machine instruction that tests a memory location and simultaneously increments or decrements the location if the test succeeds.

Assuming that such a *TestAndSet* operation returns the value of its location parameter and decrements its location parameter if it is > 0 , we can implement *Signal* and *Delay* with the following code schemas:

```
Delay(S): while TestAndSet(S) <= 0 do {nothing};
Signal(S) : S := S + 1;
```

This implementation causes a blocked process to **busy-wait** or **spin** in a while-loop until S becomes positive again through a call to *Signal* by another process. (Semaphores implemented this way are sometimes called **spin-locks**.) It also leaves unresolved the order in which waiting processes are reactivated: It may be random or in some order imposed by the operating system. In the worst case, a waiting process may be preempted by many incoming calls to *Delay* from new processes and never get to execute despite a sufficient number of calls to *Signal*. Such a situation is called **starvation**. Starvation is prevented by the use of a scheduling system that is **fair**—that is, guarantees that every process will execute within a finite period of time. In general, avoiding starvation is a much more difficult problem to solve than avoidance of deadlock (which itself is not trivial). For example, neither Java nor Ada automatically provide for fair scheduling, although it is possible to achieve it with some effort. (See the Notes and References.)

Modern shared-memory systems often provide facilities for semaphores that do not require busy-waiting. Semaphores are special memory locations that can be accessed by only one processor at a time, and a queue is provided for each semaphore to store the processes that are waiting for it.

13.5 Monitors

A monitor is a language construct that attempts to encapsulate the mutual exclusion and synchronization mechanisms of semaphores. The idea is that a more structured construct will reduce programming errors and improve the readability and correctness of code. Originally designed by Per Brinch-Hansen and C. A. R. Hoare in the early 1970s, it has been used in the languages Concurrent Pascal and Mesa. It is also the basis for the concurrency mechanisms of both Java and Ada95, as you will see shortly.

A **monitor** is an abstract data type mechanism with the added property of mutual exclusion. It encapsulates shared data and operations on these data. At most one process at a time can be “inside” the monitor—using any of the monitor’s operations. To keep track of processes waiting to use its operations, a monitor has an associated wait queue, which is organized in some fair fashion (so that processes do not wait in the queue forever as other processes arrive)—for example, as a regular first in, first out queue.

A monitor, therefore, can be viewed as a language entity with the schematic structure shown in Figure 13.9.

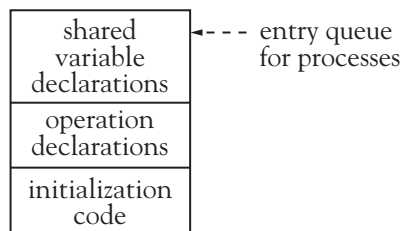


Figure 13.9 The structure of a monitor

This organization of a monitor provides for mutual exclusion in accessing shared data, but it is not adequate by itself to synchronize processes that must wait for certain conditions before continuing to execute. For example, in the bounded buffer problem, a consumer process must wait if no items are in the buffer, and a producer must wait if the buffer is full.

For this reason, a monitor must also provide **condition variables**, which are shared variables within the monitor resembling semaphores. Each has an associated queue made up of processes waiting for the condition, and each has associated **suspend** and **continue** operations, which have the effect of enqueueing and dequeuing processes from the associated queue (as well as suspending and continuing execution). Unfortunately, sometimes these operations are also called signal and delay, but their operation is different from the **Signal** and **Delay** operations of semaphores. If a condition queue is empty, a call to **continue** will have no effect, and a call to **suspend** will **always** suspend the current process.

What happens when a **continue** call is issued to a waiting process by a process in the monitor? In this situation, there are now potentially two processes active in the monitor, a situation that is forbidden. Two possibilities exist: (1) the suspended process that has just been awakened by the **continue** call must wait further until the calling process has left the monitor, or (2) the process that issued the **continue** call must suspend until the awakened process has left the monitor.

It is possible to imitate the behavior of a monitor using semaphores. (Indeed, in the last section you saw how to implement a semaphore in Java using what amounts to a monitor, and then used the semaphore to implement a bounded buffer monitor). Thus, monitors and semaphores are equivalent in terms of the kinds of parallelism they can express. However, monitors provide a more structured mechanism for concurrency than semaphores, and they ensure mutual exclusion. Monitors cannot guarantee the absence of deadlock, however. (See Exercise 13.26.)

Both Java and Ada have monitor-like mechanisms. In the next subsection, we describe how Java synchronized objects fit into the preceding description of monitors. After that, we discuss how the Java Lock and Condition interfaces, introduced in Java 1.5, provide a closer approximation to a true monitor than synchronized objects. In the final subsection of this section on Monitors, we briefly describe Ada's concurrency and monitor mechanisms, and provide an Ada bounded buffer example. Further Ada concurrency mechanisms will also be studied in Section 13.6 (message passing).

13.5.1 Java Synchronized Objects as Monitors

Java objects, all of whose methods are synchronized, are essentially monitors; for the purposes of this discussion, we will call such Java objects **synchronized objects**. Java provides an entry queue for each synchronized object. A thread that is inside the synchronized object (that is, that executes a synchronized method of the object) has the lock on the synchronized object. One immediate problem with these queues in Java, however, is that they do not operate in a fair fashion. No rules control which thread is chosen on a wait queue when the executing thread leaves a method (hence the tendency in Java to call `notifyAll()` instead of `notify()`). Also, in Java an object may have both synchronized and unsynchronized methods. What's more, any of the unsynchronized methods may be executed without acquiring the lock or going through the entry queue.

Additionally, Java's synchronized objects do not have separate condition variables. There is only one wait queue per synchronized object for any and all conditions (which is of course separate from the entry queue). A thread is placed in a synchronized object's wait queue by a call to `wait` or `sleep`;

threads are removed from an object's wait queue by a call to `notify` or `notifyAll`. Thus, `wait` and `sleep` are **suspend** operations as described previously, while `notify` and `notifyAll` are **continue** operations. Note that a thread may only be placed in a synchronized object's wait queue if it has the lock on the synchronized object, and it gives up the lock at that time. When it comes off the wait queue, it must again acquire the object's lock, so that in general it must go back on the entry queue from the wait queue. Thus, awakened threads in Java must wait for the awakening thread to exit the synchronized code before continuing (and are given no priority over threads that may have just arrived at the entry queue).

13.5.2 The Java Lock and Condition Interfaces

Java 1.5 introduced the `java.concurrent.locks` package, which includes a set of interfaces and classes that support a more authentic monitor mechanism than synchronized objects. Using this new model, the programmer represents a monitor as an explicit lock object associated with a shared data resource. Methods that access the resource under mutual exclusion are now not specified as synchronized, but instead acquire the lock by running the `unlock` method and release it by running the `lock` method on the lock itself. After acquiring the lock, a method either finishes or waits on an explicit condition object (using the method `await`). Multiple condition objects can be associated with a single lock, and each condition object has its own queue of threads waiting on it. When a method finishes, it can signal other threads waiting on a condition object by using the method `signal` or `signalAll` with that object.

The next code segment shows modifications to the data for the bounded buffer implementation of Figure 13.6, which now includes an explicit lock and conditions:

```
import java.concurrent.locks.*;

class Buffer{

    private final char[] buf;
    private int start = -1;
    private int end = -1;
    private int size = 0;

    private Lock lock;                // The lock for this resource
    private Condition okToGet;        // Condition with queue of
                                    // waiting readers
    private Condition okToPut;        // Condition with queue of
                                    // waiting writers

    public Buffer(int length){
        buf = new char[length];
        lock = new ReentrantLock();    // Instantiate the lock and its
                                    // two conditions
        okToGet = lock.newCondition();
    }
}
```

(continues)

(continued)

```

        okToPut = lock.newCondition();

    }

    public void put(char ch){                // Not synchronized on the Buffer
        ...                                object,
                                           // but unlocks and locks its lock
                                           instead
    }

    public char get(){
        ...
    }
}

```

Note that the methods `put` and `get` are no longer specified as `synchronized`. Instead, their first step is to acquire the buffer's lock, using the call `lock.unlock()`. The code to release the lock, `lock.unlock()`, should be placed in a `finally` clause associated with the `try-catch` statement. The method `put` then waits on the `okToPut` condition and signals the `okToGet` condition, whereas the method `get` waits on the `okToGetCondition` and signals the `okToPut` condition. The completion of the modified implementation is left as an exercise for the reader.

13.5.3 Ada95 Concurrency and Monitors

Concurrency in Ada is provided by independent processes called **tasks**, which are similar to Java threads. A task is declared using specification and body declarations similar to the package mechanism (see Chapter 11):

```

task T; -- task specification
-- see next section for more elaborate versions of this

task body T is
-- declarations
begin
-- code executed when task runs
end;

```

Unlike Java, an Ada task begins to execute as soon as the scope of its declaration is entered. It executes the code of its body in sequential fashion. When the end of the scope of the task declaration is reached, the program waits for the task to terminate before continuing execution. A task terminates by reaching the end of its code or by executing a `terminate` statement (discussed in Section 13.6). It is

also possible to declare task types and variables to get more than one task of a particular type or to get dynamic control over task execution. For example,

```
task type T is
...
end;

task body T is
...
begin
...
end T;

p,q: T;
```

declares a task type T and two tasks p and q of type T.

In addition to tasks, which are part of Ada83, Ada95 also has monitors, called **protected objects**, that correspond to the synchronized objects of Java. There is a similar separation of a protected type or object declaration into specification and body. For example, a protected queue type can be declared as follows:

```
protected type Queue is -- specification
    procedure enqueue (item: in ...);
    entry dequeue (item: out ...);
    function empty return boolean;
private:
    -- private data here
    size: natural;
    ...
end Queue;

protected body Queue is
    -- implementation of services here
end Queue;
```

As we see in this example, operations within a protected object are of three different kinds: functions, procedures, and **entries**. All three are synchronized in the Java sense, but with the following differences: Functions are not allowed to change the local state of a protected object but can be executed by any number of callers, provided that no procedures or entries are currently executing. Procedures and entries, however, can only be executed by a single caller at a time, and no functions can be executing simultaneously with a procedure or entry (this is a standard multi-reader, single-writer protocol).

The difference between a procedure and an entry in a protected object is that a procedure can always be executed (subject to the mutual exclusion rules), while an entry can only be executed under a certain condition, called the **entry barrier**. For example, the dequeue operation is an entry, because it can only be executed if the queue is nonempty, while an enqueue operation is a procedure, because it can always execute (assuming the queue size can be arbitrarily expanded). If a task calls an entry whose

barrier is closed (i.e., false), then the task is suspended and placed in a wait queue *for that entry* until the barrier becomes open (i.e., true), when a task in that entry's wait queue is reactivated. (Closed entries are reevaluated each time another task exits a procedure or entry that may have changed the value of the entry barrier.) Thus, Ada-protected object entries correspond to monitor condition variables (and associated code to execute when the condition is true).

Ada entries are similar to Java synchronized methods that call `wait` or `sleep`. The difference is that in Java there is only one wait queue per object, while in Ada there is a wait queue for each entry. Also, the Ada runtime system automatically recomputes the entry barrier at appropriate times and wakes a waiting task, so there are no `notify` or `notifyAll` calls in Ada.

As an example of an entry declaration, we can flesh out the previous Queue body a bit as follows:

```
protected body Queue is
  procedure enqueue (item: in ...) is
  begin
    ...
  end enqueue;

  entry dequeue (item: out ...) when size > 0 is
  begin
    ...
  end dequeue;

  function empty return boolean is
  begin
    return size > 0;
  end empty;

end Queue;
```

As a complete example of the use of protected types and tasks in Ada, Figure 13.10 gives a solution to the bounded buffer problem.

```
(1) with Text_IO; use Text_IO;

(2) procedure BoundedBuffer is

(3) type StoreType is array (positive range <>) of character;

(4) protected type Buffer (MaxBufferSize: positive) is
(5)   entry insert(ch: in character);
(6)   entry delete(ch: out character);
(7)   function more return boolean;
(8) private
(9)   store: StoreType(1..MaxBufferSize);
```

Figure 13.10 A bounded buffer solution using Ada tasks and protected types (*continues*)

(continued)

```

(10)  bufferStart: integer := 1;
(11)  bufferEnd: integer := 0;
(12)  bufferSize: integer := 0;
(13) end Buffer;
(14) protected body Buffer is

(15)  entry insert(ch: in character)
(16)    when bufferSize < MaxBufferSize is
(17)  begin
(18)    bufferEnd := bufferEnd mod MaxBufferSize + 1;
(19)    store(bufferEnd) := ch;
(20)    bufferSize := bufferSize + 1;
(21)  end insert;

(22)  entry delete(ch: out CHARACTER)
(23)    when bufferSize > 0 is
(24)  begin
(25)    ch := store(bufferStart);
(26)    bufferStart := bufferStart mod MaxBufferSize + 1;
(27)    bufferSize := bufferSize - 1;
(28)  end delete;

(29)  function more return boolean is
(30)  begin
(31)    return bufferSize > 0;
(32)  end more;

(33) end Buffer;

(34) buf: Buffer(5); -- buffer of size 5

(35) task producer;
(36) task body producer is
(37) ch: character;
(38) begin
(39)  loop
(40)    if (end_of_file) then exit;
(41)  end if;
(42)    if (end_of_line) then
(43)      skip_line;
(44) -- use carriage return in buf to indicate new line:
(45)      buf.insert(character'(Standard.Ascii.CR));
(46)    else
(47)      get(ch);
(48)      buf.insert(ch);

```

Figure 13.10 A bounded buffer solution using Ada tasks and protected types (*continues*)

(continued)

```

(49)     end if;
(50)   end loop;
(51) end producer;

(52) task consumer;
(53) task body consumer is
(54) ch: character;
(55) begin
(56)   while (not producer'terminated or buf.more) loop
(57)     buf.delete(ch);
(58)     -- carriage return indicates new line:
(59)     if ch = character'(Standard.Ascii.CR) then
(60)       new_line;
(61)     else put(ch);
(62)     end if;
(63)   end loop;
(64) end Consumer;

(65) begin
(66) null; -- no code needed, tasks execute automatically
(67) end BoundedBuffer;

```

Figure 13.10 A bounded buffer solution using Ada tasks and protected types

13.6 Message Passing

Message passing is a mechanism for process synchronization and communication using the distributed model of a parallel processor. It was introduced around 1970 by Brinch-Hansen and others.

In its most basic form, a message-passing mechanism in a language consists of two operations, *send* and *receive*, which may be defined in C syntax as follows:

```

void send(Process to, Message m);
void receive(Process from, Message m);

```

In this form, both the sending process and the receiving process must be named. This implies that every sender must know its receiver, and vice versa. In particular, the sending and receiving processes must have names within the scope of each other. A less restrictive form of *send* and *receive* removes the requirement of naming sender and receiver:

```

void send(Message m);
void receive(Message m);

```

In this case, a sent message will go to any process willing to receive it, and a message will be received from any sender. More commonly, a message-passing mechanism will require *send* to name a receiver, but allow *receive* to receive from any process. This is asymmetrical, but it mimics the situation in a

procedure call, where only the caller must know the name of the called procedure, while the called procedure has in general no knowledge of its caller.

Other questions that must be answered about the *send* and *receive* operations revolve around the synchronization of processes that wish to communicate via *send* and *receive*:

1. Must a sender wait for a receiver to be ready before a message can be sent, or can a sender continue to execute even if there is no available receiver? If so, are messages stored in a buffer for later receipt?
2. Must a receiver wait until a message is available to be sent, or can a receiver receive a null message and continue to execute?

In the case where both sender and receiver must wait until the other is ready, the message-passing mechanism is sometimes called **rendezvous**. When messages are buffered, additional questions arise. For example, is there a size limit on the number of messages that can be buffered? And what process manages the buffer? If a separate process manages the buffer, then sometimes the buffer (or its managing process) is named in the *send* and *receive* calls, instead of the sending and receiving processes. In this case, we have a **mailbox** mechanism, where processes “drop off” and “retrieve” messages from named (or numbered) mailboxes. Sometimes mailboxes are assigned **owners** (for example, a process that creates a mailbox can become its owner). In this case, the mailbox may be managed by its owner instead of a separate process.

Essential to any message-passing mechanism are **control facilities** to permit processes to test for the existence of messages, to accept messages only on certain conditions, and to select from among several possible messages. Often these control structures are influenced by or are based on Dijkstra’s **guarded if** and **guarded do** commands (see Chapter 9).

In the following, we will discuss Ada’s version of message passing, which is a form of rendezvous. Other forms of message passing include Hoare’s **Communicating Sequential Processes (CSP)** language framework, and its implementation in the language **occam**; mechanisms for distributing computation over a large network, such as **Remote Procedure Call (RPC)** or **Remote Method Invocation (RMI)**, which is supported by the Java networking library; and various forms of interfacing standards to facilitate the uniform sharing of code execution among different computers and operating systems, such as **MPI** (Message Passing Interface), **CORBA** (Common Object Request Broker Architecture) and **COM** (Common Object Model). See the Notes and References for information on these mechanisms, which are beyond the scope of this introductory chapter.

13.6.1 Task Rendezvous in Ada

So far you have seen Ada tasks as similar to Java threads, each executing a body of code, and communicating via protected (synchronized) code inside monitors. However, Ada tasks can also pass messages to each other via a rendezvous mechanism. Indeed, Ada83 did not have monitors, so rendezvous was the only way to share data among tasks in a synchronized way. (We will return to this issue shortly.)

Rendezvous points are defined by **task entries**, which are similar in appearance to protected type entries but are used (confusingly) in a completely different way. Entries are defined in a task’s specification just as they are for protected types:

```

task userInput is
  entry buttonClick (button: out ButtonID);
  entry keyPress (ch: out character);
end userInput;

```

A task exports entry names to the outside world just as protected types do. Thus, another task can rendezvous with `userInput` by “calling” `userInput.buttonClick(b)` or `userInput.keyPress(c)`. However, entries do not have code bodies. Instead, entries must appear inside an `accept` statement, which provides the code to execute when the entry is called. `Accept` statements can only appear inside the body of a task, and their position inside the body determines when they will be executed. The caller of an entry will wait for the task to reach a corresponding `accept` statement, and, similarly, if no call has occurred, a task that reaches an `accept` statement will wait for a corresponding call. Thus, a rendezvous occurs at the point in the called task’s body where the corresponding `accept` statement appears, and the message that is passed at the rendezvous is represented by the entry parameters. Each entry has an associated queue to maintain processes that are waiting for an `accept` statement to be executed. This queue is managed in a first in, first out fashion.

To continue with the previous example, the `userInput` body might, therefore, look as follows:

```

task body userInput is
begin
  ...
  -- respond to a button click request:
  accept buttonClick (button: out ButtonID) do
    -- get a button click while the caller waits
  end buttonClick; -- caller can now continue
  -- continue with further processing
  ...
  -- now respond to a keypress request:
  accept keyPress (ch: out character) do
    -- get a keypress while the caller waits
  end keyPress; -- caller can now continue
  -- continue with further processing
end userInput;

```

An `accept` statement has the following form (in EBNF notation):

```

accept entry-name [ formal-parameter-list ]
[do statement-sequence end [entry-name] ] ;

```

When an `accept` statement is executed, the caller remains suspended while the code in the `accept` body is executed (the statements between the `do` and the `end`). An `accept` statement does not need to name the caller of the entry. Thus, Ada’s message-passing mechanism is asymmetric like procedure calls. The entry/accept mechanism can be used entirely for synchronization, in which case a body of code for the entry is unnecessary.

Typically, a task that operates as a server for other client tasks will want to maintain a set of possible entries that it will accept and wait for one of these entries to be called, whereupon it will process the entry and loop back to wait for the next entry. Which entries can be selected can depend on a number of conditions (such as whether a queue is empty or a buffer is full). Maintaining a set of entries to accept is done through the `select` statement. The EBNF for a `select` statement is:

```
select
[when condition =>] select-alternative
{ or [when condition =>] select-alternative }
[else statement-sequence]
end select ;
```

where a *select-alternative* is an `accept` statement followed by a sequence of statements, or a `delay` statement (not described here) followed by a sequence of statements, or a `terminate` statement.

The semantics of the `select` statement are as follows. All the conditions in the `select` statement are evaluated, and those that evaluate to true have their corresponding select alternatives tagged as **open**. An open `accept` statement is selected for execution if another task has executed an entry call for its entry. If several accepts are available, one is chosen arbitrarily. If no open accepts are available and there is an `else` part, the statement sequence of the `else` part is executed. If there is no `else` part, the task waits for an entry call for one of the open accepts. If there are no open accepts, then the `else` part is executed if it exists. If there is no `else` part, an exception condition is raised (see Chapter 9).

Looping back to a set of `select` alternatives is not part of the basic `select` command but can easily be achieved by surrounding a `select` statement with a loop.

Continuing with the previous example, the `userInput` body might be structured as follows to allow it to behave as a server for `keyPress` and `buttonClick` requests:

```
(1) task body userInput is
(2) begin
(3)   loop
(4)     select
(5)       when buttonClicked =>
(6)         -- respond to a button click request:
(7)           accept buttonClick (button: out ButtonID) do
(8)             -- get and return a button click
(9)             -- while the caller waits
(10)          end buttonClick; -- caller can now continue
(11)         -- do some buttonClick cleanup
(12)       or when keypressed =>
(13)         -- now respond to a keypress request:
(14)           accept keyPress (ch: out character) do
(15)             -- get a keypress while the caller waits
(16)             end keyPress; -- caller can now continue
(17)         -- do some keyPress cleanup
```

(continues)

(continued)

```
(18)      or terminate;
(19)      end select;
(20)  end loop;
(21) end userInput;
```

Note the `terminate` alternative in the `select` statement (line 18). Termination of tasks in Ada can occur in one of two ways. Either the task executes to completion and has not created any dependent tasks (i.e., child processes) that are still executing, or the task is waiting with an open `terminate` alternative in a `select` statement, and its **master** (the block of its parent task in which it was created) has executed to completion. In that case, all the other tasks created by the same master must also have terminated or are waiting at a `terminate` alternative, in which case they all `terminate` simultaneously. This avoids the necessity of writing explicit synchronizing statements (such as `join` or `wait`) in a parent task that must wait for the completion of its children.

Of course, a task with the preceding behavior might, depending on circumstances, be better written as a protected type in Ada. For example, we can write a version of the bounded buffer (Figure 13.10) that uses a task and rendezvous to provide mutual exclusion and synchronization, even though a protected type is likely to be more appropriate. The code is in Figure 13.11. Note that, because tasks, unlike protected types, cannot have functions that return values, but must return values throughout parameters in entries, the buffer code in Figure 13.11 requires a small modification in the consumer code of Figure 13.10. We leave the details to the reader (Exercise 13.39).

We offer two additional examples of tasks in Ada. The first (Figure 13.12) is an implementation of a semaphore type as a task type. Note in that example that the code block in an `accept` of a parameterless entry such as `signal` or `wait` can be empty, in which case the `do . . . end` can be omitted. The second example (Figure 13.13) is an Ada package using tasks for parallel matrix multiplication.

```
task buf is
  entry insert(ch: in character);
  entry delete(ch: out character);
  entry more (notEmpty: out boolean);
end;
task body buf is
  MaxBufferSize: constant integer := 5;
  store: array (1..MaxBufferSize) of character;
  bufferStart: integer := 1;
  bufferEnd: integer := 0;
  bufferSize: integer := 0;
begin
  loop
    select
      when bufferSize < MaxBufferSize =>
        accept insert(ch: in character) do
          bufferEnd := bufferEnd mod MaxBufferSize + 1;
          store(bufferEnd) := ch;
```

Figure 13.11 A bounded buffer as an Ada task (continues)

(continued)

```

        end insert;
        bufferSize := bufferSize + 1;
    or when bufferSize > 0 =>
        accept delete(ch: out character) do
            ch := store(bufferStart);
        end delete;
        bufferStart := bufferStart mod MaxBufferSize + 1;
        bufferSize := bufferSize - 1;
    or
        accept more(notEmpty: out boolean) do
            notEmpty := bufferSize > 0;
        end more;
    or terminate;
end select;
end loop;
end buf;

```

Figure 13.11 A bounded buffer as an Ada task

```

task type Semaphore is
    entry initSem (n: in integer);
    entry wait; -- use wait instead of delay, which is reserved in Ada
    entry wait;
end;
task body Semaphore is
    count : integer;
begin
    accept initSem (n: in integer) do
        count := n;
    end initSem;
    loop
        select
            when count > 0 =>
                accept wait;
                count := count - 1 ;
            or
                accept signal;
                count := count + 1;
            or
                terminate;
        end select;
    end loop;
end Semaphore;

```

Figure 13.12 A semaphore task type in Ada


```

generic Size: INTEGER;
package IntMatrices is
  type IntMatrix is array (1..Size,1..Size) OF INTEGER;
  function ParMult(a,b: in IntMatrix) return IntMatrix;
end;

package body IntMatrices is
  function ParMult(a,b: in IntMatrix) return IntMatrix is
    c: IntMatrix;

    task type Mult is
      entry DoRow (i: in INTEGER);
    end;
    task body Mult is
      iloc: INTEGER;
    begin

      accept DoRow (i: in INTEGER) do
        iloc := i;
      end;
      for j in 1..Size loop
        c(iloc,j) := 0;
        for k in 1.. Size loop
          c(iloc,j) := c(iloc,j) + a(iloc,k) * b(k,j);
        end loop;
      end loop;
    end Mult;

  begin -- ParMult
    declare m: array (1..Size) of Mult;
    begin
      for i in 1..Size loop
        m(i).DoRow(i);
      end loop;
    end;
    return c;
  end ParMult;
end IntMatrices;

```

Figure 13.13 Parallel matrix multiplication in an Ada package

13.7 Parallelism in Non-Imperative Languages

Parallel processing using functional or logic programming languages is still in an experimental and research phase, despite a significant amount of work since the mid-1980s. Nevertheless, a number of good implementations of research proposals exist. In this section we discuss several of these, including MultiLisp, QLisp, Parlog, and FGHC, after a brief general introduction to the area. Finally, we explore parallel programming with Erlang, a non-imperative language that supports a message-passing style. The reader is encouraged to consult the references at the end of the chapter for more information.

In earlier chapters, we have mentioned that non-imperative languages such as Lisp and Prolog offer more opportunities for automatic parallelization by a translator than do imperative languages. The opportunities for parallel execution in such languages fall into two basic classes: and-parallelism and or-parallelism. We'll consider and-parallelism first.

13.7.1 And-Parallelism

In and-parallelism, a number of values can be computed in parallel by child processes, while the parent process waits for the children to finish and return their values. This type of parallelism can be exploited in a functional language in the computation of arguments in a function call. For example, in Lisp, if a process executes a function call

```
(f a b c d e)
```

it can create six parallel processes to compute the values f , a , $\dots$, e . It then suspends its own execution until all values are computed, and then calls the (function) value of f with the returned values of a through e as arguments. Similarly, in a let-binding such as:

```
(let ((a e1) (b e2) (c e3)) (...))
```

the values of $e1$, $e2$, and $e3$ can be computed in parallel. In Prolog, a similar and-parallel opportunity exists in executing the clause

```
q :- p1, p2, ..., pn.
```

The $p1$ through p_n can be executed in parallel, and q succeeds if all the p_i succeed. Implicit in this description is that the computations done in parallel do not interfere. In a purely functional language, the evaluation of arguments and let-bindings causes no side effects, so noninterference is guaranteed. However, most functional languages are not pure, and side effects or state changes require that and-parallelism be synchronized. In Prolog, the instantiation of variables is a typical and necessary side effect that can affect the behavior of and-parallelism. For example, in the clause

```
process(N,Data) :-
    M is N - 1,
    Data = [X|Data1],
    process(M,Data1).
```

the three goals on the right-hand side cannot in general be executed in parallel, since each of the first two contribute instantiations to the last. Thus, synchronization is also necessary here.

13.7.2 Or-Parallelism

In or-parallelism, execution of several alternatives can occur in parallel, with the first alternative to finish (or succeed) causing all other alternative processes to be ignored (and to terminate). In Lisp, an example of such parallelism can occur in the evaluation of a `cond` expression:

```
(cond (p1 e1) (p2 e2) ... (pn en))
```

In this situation, it may be possible for the e_i that correspond to true p_i conditions to be evaluated in parallel, with the first value to be computed becoming the value of the `cond` expression. (It may also be possible to compute the p_i themselves in parallel.) In this case, it may also be necessary to synchronize the computations. However, there is another problem as well: or-parallel computation makes the `cond` into a nondeterministic construct (similar to Dijkstra's guarded if), which may change the overall behavior of the program if the order of evaluation is significant. For example, an `else-part` in a `cond` should not be evaluated in parallel with the other cases.

In Prolog, or-parallelism is also possible in that a system may try to satisfy alternative clauses for a goal simultaneously. For example, if there are two or more clauses for the same predicate,

```
p(X) :- q(X).
p(X) :- r(X).
```

a system may try to satisfy q and r in parallel, with p succeeding with X instantiated according to the one that finishes first (or perhaps even saving other instantiations in a queue). In fact, this is consistent with the semantics of pure logic programming, since alternative goals are satisfied nondeterministically. However, in common Prolog implementations, the correct execution of a program may depend on the order in which alternatives are tried. In this case, a strict ordering may need to be applied. For example, in a common program for factorial in Prolog,

```
fact (X, Y) :- X = 0, !, Y = 1.
fact (X, Y) :- Z is X - 1, fact (Z, Y1), Y is X * Y1.
```

the first clause needs to be tried before the second (or reaching the cut in the first should terminate the second).

The synchronization and order problems we have mentioned for both and-parallelism and or-parallelism are difficult to solve automatically by a translator. For this reason, language designers have experimented with the inclusion of a number of manual parallel constructs in non-imperative languages. In some cases, these are traditional constructs like semaphores or mailboxes, with modified semantics to provide better integration into the language. In other cases, more language-specific means are used to indicate explicit parallelism.

There is, however, another argument for the inclusion of explicit parallelism in non-imperative languages. In many situations, one may wish to suppress the creation of small (i.e., fine-grained) processes when the computational overhead to create the process is greater than the advantage of computing the result in parallel. This may be the case, for example, in highly recursive processes, where as one approaches the base case, one may not want to create new processes but switch to ordinary computation. Or, one may wish to suppress parallel computation altogether when the values to be computed have a small computational overhead.

Next, we examine a few of the explicit parallel mechanisms employed in some of the parallel Lisps and Prologs mentioned earlier in this chapter.

13.7.3 Parallelism in Lisp

The more natural form of parallelism for Lisp seems to be and-parallelism, and this is the kind most often implemented. In the language Multilisp, which like Scheme (see Chapter 3) has static scoping and first-class function values, parallel evaluation of function calls is indicated by the syntax

```
(pcall f a b c...)
```

which is equivalent to the evaluation of `( f a b c ... )` but with parallel evaluation of its subexpressions. Even more parallelism can be achieved in Multilisp by the use of a “future:” If `f` has already been computed in the function call `( f a b c )`, then the execution of `f` can proceed even before the values of `a`, `b`, and `c` have been computed, at least up to the point in `f` where those values are used. For example, in an `f` defined by:

```
(define (f a b) (cond ((= a 0) ...)
                      ((> b 0) ...) ...))
```

the value of `b` is never used if `a` evaluates to 0, so the computation can proceed regardless of how long it takes to compute `b`.

A **future** is a construct that returns a pointer to the value of a not-yet-finished parallel computation. A future resembles (but is not identical to) delayed evaluation, as represented in the `de lay` primitive of Scheme, studied in Section 10.5. In Multilisp a call

```
(pcall f (future a) (future b) ...)
```

allows the execution of `f` to proceed before the values of `a` and `b` have been computed. When the execution of `f` reaches a point where the value of `a` is needed, it suspends execution until that value is available. Futures have been included in other parallel Lisps besides Multilisp—for example, Qlisp.

Qlisp includes a parallel let-construct called `ql et` to indicate and-parallelism in let-bindings:

```
(ql et p (bindings) exp)
```

This construct will evaluate the bindings in parallel under the control of the predicate `p`. If `p` evaluates to `nil`, ordinary evaluation occurs. If `p` evaluates to a non-`nil` value, then the bindings will be computed in parallel. If `p` evaluates to the special keyword `:eager`, then futures will be constructed for the values to be bound.

13.7.4 Parallelism in Prolog

The more natural form of parallelism for Prolog appears to be or-parallelism, although both forms have been implemented. In fact, or-parallelism fits so well with the semantics of logic programming (as noted above) that it is often performed automatically by a parallel Prolog system.

And-parallelism is more difficult in Prolog because of the interactions of instantiations mentioned earlier, and because there is no natural return point for backtracking. One version of and-parallelism uses guarded Horn clauses to eliminate backtracking. A guarded Horn clause is one of the form:

$$h : - g_1, \dots, g_n \mid p_1, \dots, p_m.$$

The $g_1, \dots, g_n$ are **guards**, which are executed first and which are prohibited from establishing instantiations not already provided. If the guards succeed, the system **commits** to this clause for h (no backtracking to other clauses occurs), and the $p_1, \dots, p_m$ are executed in parallel. Such a system is FGHC (for flat guarded Horn clauses). A sample FGHC program is the following, which generates lists of integers:

```
generate(N,X) :- N = 0 | X = [].
generate(N,X) :- N > 0 | X = [N|T], M is N-1,
                    generate(M,T).
```

The problem with FGHC is that it places severe constraints on variable instantiation and, hence, on unification. This results in a significant reduction in the expressiveness of the language. An alternative is to provide **variable annotations** that specify the kind of instantiations allowed and when they may take place. A language that does this is **Parlog**. In Chapters 3, 4, and 10 we discussed the difference between input variables and output variables of a procedure: Input variables are like value parameters; that is, they have incoming values but no outgoing values. Output variables, on the other hand, have only outgoing values. In Prolog this means that input variables may be instantiated when initiating a goal, but may not be instantiated during the process of satisfying the goal, and similarly for output variables. Parlog distinguishes input variables from output variables in a so-called **mode declaration** by writing input variables with a “?” and output variables with a “^.” If during the process of satisfying a goal, an uninstantiated input variable must be unified, the process suspends until such time as the variable becomes instantiated. A further tool for controlling instantiations is “directional” unification: The goal $X = Y$ has a variant $X \leq Y$, which only allows X to be instantiated, not Y .

As an example of and-parallelism in Parlog, consider the following quicksort program:

```
mode qsort (P?,S^).
qsort([],[]).
qsort([H|T] S) :- partition(H,T,L,R),
                    qsort(L,L1),
                    qsort(R,R1),
                    append(L1,[ H|R],S).

mode partition(P?,Q?,R^,S^).
partition(P,[A|X],[A|Y],Z) :- A < P:
                                partition(P,X,Y,Z).
partition(P,[A|X],Y,[A|Z]) :- A >= P:
                                partition(P,X,Y,Z).
partition(P,[],[],[]).
```

Parlog selects an alternative from among clauses by the usual unification process and also by using guards. It then commits to one alternative. In the foregoing `qsort`, the `partition` predicate has guards to prevent incorrect choices. After selecting a clause, Parlog creates a process for each goal on the right-hand side. Thus, Parlog will execute the four goals on the right-hand side of `qsort` in parallel. Since `L` and `R` are input variables to the two right-hand calls to `qsort`, their associated processes will suspend as soon as these are needed for unification, until the first `partition` process produces them. The `append` process (whose definition is not shown) will also suspend until its first two arguments become available.

13.7.5 Parallelism with Message Passing in Erlang

Erlang is a functional language developed by Joe Armstrong at Ericsson, the telecommunications company, in 1986. Armstrong was assigned to develop software for the kinds of distributed, fault-tolerant, real-time applications needed in the telecommunications industry. Like Lisp, Erlang uses strict evaluation and dynamic typing. The language has a syntax similar to that of Haskell (Chapter 3), with powerful pattern-matching capability. However, Erlang extends this capability with variable binding via single assignment, and distinguishes between atoms and variables, as Prolog distinguishes between constant terms and logical variables. Finally, Erlang supports concurrency and parallelism based on message passing. The absence of side effects and the presence of message passing combine in Erlang to produce a very simple model of parallel programming. The examples of Erlang code that follow are adapted from Armstrong [2007], which provides an excellent introduction to the language.

Like the other functional languages we have studied, an Erlang implementation includes an interactive interpreter and a compiler. The following session with the Erlang interpreter shows the use of pattern matching with variables (capitalized names), numbers, constant terms (lowercase names), and tuples (items enclosed in {}):

```
1> Rectangle = {rectangle, 8, 4}
{rectangle, 8, 4}
2> Circle = {circle, 3.5}
{circle, 3.5}
3> {rectangle, Width, Height} = Rectangle.
{rectangle, 8, 4}
4> Width.
8
5> {circle, Radius} = Circle.
{circle, 3.5}
6> Radius.
3.5
```

In this code, rectangles and circles are represented as tuples containing numeric information and a constant term that serves as a type tag. Single assignment with the = operator allows a variable to be bound to a value just once but then be referenced multiple times thereafter. The assignments at prompts 3 and 5 illustrate pattern matching. The variables nested in the tuples on the left side of the = operator are matched and bound to the corresponding values in the structures referenced by the variables to the right of this operator.

Pattern matching and variable binding also form the basis of function applications in Erlang. The next code segment shows a `geometry` module containing the definition of an `area` function for the representations of rectangles and circles described earlier. This code is saved in the file `geometry.erl`.

```
-module(geometry).
-export([area/1]).

area({rectangle, Width, Height}) -> Width * Height;
area({circle, Radius}) -> 3.14159 * Radius * Radius.
```

Note that the `area` function consists of two clauses, one for rectangles and the other for circles. When the programmer applies the `area` function to a value, the interpreter matches this value to the header of the appropriate clause of the `area` function as defined in the `geometry` module. Note that the form of the argument to the function must match the header of one of the two clauses specified in the module, or an error is raised. When an argument matches a function clause, the variables in the header of this clause are bound to the corresponding values contained in the argument. The clause's code following the `->` symbol is then executed in the context of these bindings. Here is an example session that compiles and tests the `geometry` module:

```
1> c(geometry).
{ok, geometry}
2> geometry:area({rectangle, 8, 4}).
32
3> geometry:area({circle, 3.5}).
38.4844775
```

Before we examine how Erlang can execute this code in parallel with other code, we need to see how functions can be passed as arguments to other functions. Mapping, which we have seen before in other functional languages, illustrates the use of function arguments. Erlang's standard `lists` module includes a `map` function for mapping other functions onto lists of arguments. A function argument to `lists:map` (or to any other function) must be a special type of Erlang function called a `fun`. The following session shows the use of `fun` in three applications of `lists:map`:

```
1> lists:map(fun(N) -> 2 * N end, [2, 4, 6]).
[4, 8, 12]
2> Double = fun(N) -> 2 * N end.
#Fun<erl_eval.6.57006448>
3> lists:map(Double, [2, 4, 6]).
[4, 8, 12]
4> lists:map(fun math:sqrt/0, [4, 9, 16]).
[2, 3, 4]
```

At prompt 1, `fun` is used, like `lambda` in Lisp, to build an anonymous function to pass as an argument to `lists:map`. At prompt 2, the variable `Double` is bound to a `fun`. This variable is then passed as an argument to `lists:map` at prompt 3. At prompt 4, the standard `math` function `sqrt` is wrapped in a `fun` before it is passed as an argument to `lists:map` at prompt 4.

Armed with these basic ideas, we are ready to see how Erlang supports parallel programming. An Erlang process is a very lightweight software object that runs independently of any other process and communicates with other processes by sending and receiving messages. The essential operations on processes are **spawn**, which creates a new process; **send**, for sending messages to other processes; and **receive**, for receiving messages. Table 13.1 describes the syntax and semantics of these operations in Erlang.

Table 13.1 The essential process operations in Erlang	
Operation	What It Does
Pid = spawn(Fun)	Creates a new process to execute the function Fun in parallel with the caller and returns the process identifier for this process. Pid can then be used to send messages to the new process.
Pid ! message	Sends message (a pattern) to the process identified by Pid. The sender does not wait, but continues its execution.
receive Pattern1 [when Guard1] -> Expressions1 Pattern2 [when Guard2] -> Expressions2 . . . end	Receives a message (a pattern). If the message matches one of the given patterns, the corresponding code is executed. Otherwise, the message is saved for later processing.

Let’s return to the example of the area function, defined earlier in a geometry module. In a parallel computing context, we can view this function as a **server** that executes in its own process. Another process, called a **client**, makes a request of this server for the area of a given geometric shape. The server handles this request by computing the area and returning it to the client.

The client process, which in our new example is just the interactive interpreter’s process, begins by spawning a server process. When the server process is spawned, it runs a function that executes an infinite loop. This loop receives a message from a client, computes an area, and sends the result back to that client. In the new implementation, the area function is updated to handle the transaction between client and server. This function now expects the server’s process identifier as well as the shape pattern as arguments. The next session demonstrates an interaction between the user/client and the area server.

```
1> c(area_server).  
{ok, area_server}  
2> Pid = spawn(fun area_server:loop/0).  
<0.35.0>  
3> area_server:area(Pid, {rectangle, 8, 4}).  
32  
4> area_server:area(Pid, {circle, 3.5}).  
38.4844775
```

At prompt 1, the module of function definitions, called `area_server`, is compiled. At prompt 2, a server process is spawned. It is passed the `loop` function of the `area_server` module, suitably wrapped in a `fun`. The `loop` function, ready to receive and process messages, begins concurrent execution immediately in the new process, and the variable `Pid` is bound to the server’s process identifier. At prompts 3 and 4, the new area function is called, with the server’s process identifier and some shape patterns as arguments. Behind the scenes, the server process receives messages from this function, which returns the results of the server’s computations.

Here is the code for the module of server functions, followed by an explanation.


```

-module(area_server).
-export([area/2, loop/0].

area(Pid, shape) ->
  Pid ! {self(), shape},
  receive
    {Pid, Result} ->
      Result
  end.

loop() ->
  receive
    {From, {rectangle, Width, Height}} ->
      From ! {self(), Width * Height},
      loop();
    {From, {circle, Radius}} ->
      From ! {self(), 3.14159 * Radius * Radius},
      loop();
    {From, Other} ->
      From ! {self(), error, Other},
      loop()
  end.

```

As mentioned earlier, the `area` function now expects the server's process identifier and the shape's pattern as arguments. This function creates a new pattern containing the shape and the client's own process identifier (obtained by a call to `self()`). The server will use the client's process identifier to send its result back to the client. This pattern is then sent as a message to the server (`Pid`). The `area` function then receives the result from the server. Note that this result is extracted from the pattern `{Pid, Result}`. This notation guarantees that the client receives, by a successful match, only a message sent by this particular server, rather than another message that might be sent by another process at this point.

The `loop` function handles requests from the client. This function is passed as an argument to the `spawn` function to start up the server's process (essentially telling the process to execute this function). Note that the `loop` function contains a single `receive` form, which in turn contains three guards for matching patterns sent as messages from the client. The variable `From` in each pattern is bound to the client's process identifier and is used to send the result back to that client. The first guard handles the case of rectangles, the second guard deals with circles, and the last guard recovers from the case of an unrecognized shape pattern. In each case, the message sent back to the client (`From`) includes the server's process identifier (`self()`), so that the client will receive only a message sent by this server. Finally, note that in each case, after the result is sent back to the client, the `loop` function executes a recursive call as its last logical step. This has the effect of continuing the infinite loop and also of freeing the server to receive other messages. Because the tail-recursive `loop` function compiles to a genuine loop, it can run forever without stack overflow.

Interactions between processes often are not as tightly coupled as in the `area` server example. On the one hand, a process may hand off work to other processes and go its merry way without waiting for a response. On the other hand, a process may wait longer than is desirable to receive a message, because

the sender has crashed or an error has occurred in the sender's algorithm. For this case, the receiver process can include an `after` clause to time out the `receive` operation. Its form is:

```
receive
  Pattern1 [ when Guard1 ] ->
    Expressions1
  Pattern2 [ when Guard2 ] ->
    Expressions2
  . . .
  after Time ->
    Expressions
end
```

The `Time` variable stands for the maximum number of milliseconds that the `receive` operation must wait for a message, after which control proceeds to the `Expressions`, which are executed. This feature can be used to define a `sleep` function, which essentially suspends the calling process for a given amount of time:

```
sleep(Time) ->
  receive
  after Time ->
    true
  end
```

The Erlang programmer also must be aware of the manner in which messages are scheduled for a process. Each process is associated with a **mailbox**, a queue-like data structure. When another process sends a message, it goes at the end of the receiver process's mailbox. The process's `receive` operation attempts to match the message at the front of its mailbox to one of its patterns. If a match occurs, the message is removed from the mailbox, discarded, and the expressions in the clause are executed. If no match occurs, the message is removed from the mailbox and put at the rear of the process's **save queue**. The next message in the mailbox is then tried, and so forth, until the mailbox becomes empty. If that happens, the process itself is suspended, until a new message arrives in the mailbox, at which point, the process itself is rescheduled for execution to examine this message. If a message is matched, all of the messages in the saved queue are transferred to the mailbox in their original order. The same transfer of messages from saved queue to mailbox is carried out if a timeout due to an `after` clause occurs.

Our final example defines a function `flush_mailbox`, which empties the mailbox of a process.

```
flush_mailbox() ->
  receive
    _Any ->
      flush_mailbox()
  after 0 ->
    true
  end
```

In this code, the variable `_Any` matches any message/pattern in the mailbox, so the recursive calls of `flush_mailbox` continue until there are no more messages in the box. At this point, the `after` clause immediately times out the `receive` form, so that the process does not suspend forever.

Erlang also includes an array of resources for developing **distributed applications** that run on a network of independent computers. Independent processes are embedded in software objects called **nodes**, which allow messages to be sent and received across a network of machines. The reader is encouraged to consult the Armstrong text [2007] for further information.

Exercises

- 13.1 How many links does a fully linked distributed system with n processors need? How do you think this affects the design of fully linked systems with many processors?
- 13.2 Some parallel processors limit the number of processes that a user can create in a program to one less than the number of processors available. Why do you think this restriction is made? Is such a restriction more or less likely on an SIMD or MIMD system? Might there be a reason to create more processes than processors? Explain.
- 13.3 It is possible to compute the sum of n integers in fewer than n steps. Describe how you could use k processors to do this, where $k < n$. Would there be any advantage to having $k > n$?
- 13.4 The text mentioned that matrix multiplication can be done in n steps using n^2 processors. Can it be done in fewer steps using more processors?
- 13.5 Describe the difference between SPMD and MPMD programming. Would you characterize the program of Figure 13.3 as SPMD or MPMD? Why?
- 13.6 In the chapter discussion, why did we use the term “MPMD” instead of “fork-join”?
- 13.7 Explain why both large-grained and small-grained parallelism can be less efficient than medium grained. Give programming examples to support your argument.
- 13.8 The C code of Figure 13.4 assumes that `NUMPROCS` is less than `SIZE`. What happens if `NUMPROCS > SIZE`? Rewrite the code of Figure 13.4 to take advantage of the extra processors if `NUMPROCS > SIZE`.
- 13.9 Explain what happens in each step of the following UNIX pipeline:


```
cat *.java | tr -sc A-Za-z '\012' | sort | uniq -c | sort -rn
```
- 13.10 Command-line windows in versions of Microsoft Windows also allow certain commands to be piped, such as `dir | more`. Determine if these pipes are true pipes in the UNIX sense (that is, each program in the pipe is a separate process, with no intermediate files saved to disk).
- 13.11 A typical process synchronization problem is that of resource allocation. For example, if a system has three printers, then at most three processes can be scheduled to print simultaneously. Write a program to allocate three printers to processes in
 - (a) Java
 - (b) Ada95
 - (c) Ada83

- 13.12** A standard problem (like the bounded buffer problem) that is used to test a concurrent language mechanism is the **readers-writers** problem. In this problem, stored data are continuously accessed (read) and updated (written) by a number of processes. A solution to this problem must allow access to the data by any number of readers, but by only one writer at a time. Write a solution to this problem using **(a)** Java; **(b)** Ada tasks.
- 13.13** The Java specification promises that getting the value of a single variable of any built-in type *except* `long` or `double`, or updating such a variable, is an atomic operation, but that such operations on `long` or `double` variables may not be atomic.
- (a)** Explain the reason for this difference.
 - (b)** In what way did this fact affect the code for the bounded buffer problem (Figure 13.6)?
 - (c)** Does this fact have any effect on the solution to the readers-writers problem? Explain.
- 13.14** Another standard concurrency problem is the **dining philosophers problem** of Dijkstra. In this problem, five philosophers spend their time eating and thinking. When a philosopher is ready to eat, she sits at one of five places at a table heaped with spaghetti. Unfortunately, there are only five forks, one between every two plates, and a philosopher needs two forks to eat. A philosopher will attempt to pick up the two forks next to her, and if she succeeds, she will eat for a while, leave the table, and go back to thinking. For this problem,
- (a)** Describe how deadlock and starvation can occur.
 - (b)** Why are there five philosophers and not four or three?
 - (c)** Write a deadlock-free solution to this problem in either Java or Ada.
 - (d)** Is it possible to guarantee no starvation in your solution to part (c)? Explain.
- 13.15** The C code for matrix multiplication in Figure 13.5 doesn't work in a typical UNIX implementation, since a fork does not cause memory to be shared. This problem can be solved using files, since forked processes continue to share files. Rewrite the code to compute the matrix `c` correctly using files. Comment on the efficiency of your solution.
- 13.16** Can Figure 13.5 be corrected by using pointer variables to share the addresses of the arrays among several processes? Explain why or why not.
- 13.17** **(a)** Write a solution to the matrix multiplication problem in Java using one thread for each row and column of the answer.
- (b)** Write a solution to the matrix multiplication problem in Java that uses a fixed number of threads less than the number of elements in the answer matrix, where each thread looks for the next element in the answer matrix that has not yet been worked on.
- 13.18** Repeat the previous exercise in Ada.
- 13.19** Given two processes p and q , and a semaphore S initialized to 1, suppose p and q make the following calls:

```

p: delay(S);
    ...
    delay(S);
q: delay(S);
    ...
    signal(S);

```

Describe what will happen for all possible orders of these operations on S .

- 13.20** Describe how the busy-wait implementation of a semaphore can cause starvation.
- 13.21** In addition to deadlock and starvation, concurrent processes can experience **livelock**, where no process is blocked, all processes get to execute, but each continually repeats the same code without possibility of success. Write a simple program in **(a)** Java or **(b)** Ada that is an example of livelock.
- 13.22** Here is an alternative to the code for the semaphore operations:

```

Delay(S) : S := S - 1;
           if S < 0 then suspend the calling process;
Signal(S): S := S + 1;
           if S >= 0 then wake up a waiting process;

```

Compare this to the code in Section 13.4. Is there any difference in behavior?

- 13.23** Does it make any sense to initialize a semaphore to a negative value? Why or why not?
- 13.24** Sometimes a language will provide only a **binary semaphore** mechanism, in which the stored value is Boolean instead of integer. Then, the operations wait and signal become

```

Delay(S) : if S = true then S := false
           else suspend the calling process
Signal(S): if processes are waiting then wake up a process
           else S := true;

```

(To distinguish them from binary semaphores, the semaphores described in the text are sometimes called **counting semaphores**.) Show how a counting semaphore can be implemented using binary semaphores.

- 13.25** A lock is sometimes distinguished from a semaphore by being nonblocking: Only one process can acquire a lock at a time, but if a process fails to acquire a lock, it can continue execution. Suppose that a lock L has two operations, a Boolean function **lock-lock(L)** that returns true if the lock has been acquired and false otherwise, and **unlock-lock(L)**, which unlocks the lock (having no effect if the lock is already unlocked). Write an implementation for a lock in:
- (a)** Java
 - (b)** Ada
- 13.26** If one monitor entry calls another monitor entry, deadlock may result.
- (a)** Construct an example to show how this can happen.
 - (b)** How is this problem dealt with in Java?
 - (c)** In Ada?
- 13.27** Suppose that the code for the signal operation in the Java implementation of a semaphore (Figure 13.7) were changed to:

```

public synchronized void signal(){
    notify();
    count++;
}

```

Would this work? Explain.

- 13.28** The Java semaphore code in Figure 13.7 is overly simple, because of the possibility of interrupts. Here is an improved version for the `delay()` procedure:

```
// adapted from Lea [2000], p. 267

public void delay() throws InterruptedException{
    if (Thread.interrupted())
        throw new InterruptedException();
    synchronized(this){
        try{
            while (count <= 0) wait();
            count--;
        }catch(InterruptedException ie){
            notify(); throw ie;
        }
    }
}
```

Explain carefully the reason for each of the differences between this code and the code of Figure 13.7.

- 13.29** Explain the behavior of the Java code for the bounded buffer problem if the keyword `synchronized` is removed from the `put` operation (line 28 of Figure 13.6).
- 13.30** In Ada, you saw that the bounded buffer could be represented either by task or a monitor. Is one of these solutions preferred over the other? Why?
- 13.31** A shared-memory system can imitate the mailbox version of message passing by defining a mailbox utility using mutual exclusion. Write a package to implement mailboxes (i.e., queues of data with *send* and *receive* primitives) in:
- (a) Java
 - (b) Ada
- 13.32** In Ada, the caller of an entry must suspend execution during the execution of the corresponding `accept`-statement. Why is this necessary?
- 13.33** In an Ada `select` alternative, an `accept`-statement can be followed by more statements. For example, in the bounded buffer solution in Ada, we wrote:

```
accept insert (ch: in character) do
    bufferEnd := bufferEnd mod MaxBufferSize + 1;
    store (bufferEnd) := ch;
end insert;
bufferSize := bufferSize + 1;
```

Note that `bufferSize` is incremented outside the `accept` statement. Is there a reason for this? Could the statement `store(bufferEnd) := ch` be moved out of the `accept`? Why?

- 13.34** In Ada, a task type can be used in the declaration of other types. In particular, pointer (or access) types to task types can be declared, as in:

```
task type T is ... end;
type A is access T;
...
x: A;
```

The variable `x` now represents a pointer to a task. When does the associated task begin executing? When might it terminate?

- 13.35** In Ada, a task is not automatically terminated when the end of the scope of its declaration is reached in its parent task. Instead, the parent suspends until the task completes. Why do you think this choice was made in the design of Ada?
- 13.36** In the Ada implementation of a semaphore, the `Signal` entry always incremented the semaphore value. Is this correct? Why?
- 13.37** An alternative for the Ada implementation of semaphores is to test for the existence of a waiting task using the attribute `COUNT`, which is predefined for entries. The code inside the loop would then read as follows:

```
select
  when count > 0 =>
    accept Wait;
    count := count - 1;
  or
    accept Signal;
    if Wait'COUNT > 0 then
      accept Wait;
    else
      count := count + 1;
    end if;
  or
    terminate;
end select;
```

Is this implementation preferable to the one given? Why or why not?

- 13.38** Write an implementation of a semaphore as an Ada protected type, and compare this with the task implementation of Figure 13.10.
- 13.39** Rewrite the Ada bounded buffer code of Figure 13.10 to use the buffer task of Figure 13.11.
- 13.40** In Figure 13.13 (parallel matrix multiplication in Ada), we used a local variable `i loc` inside task `Mult` to store the current row index of matrix multiplication for use outside the accept-statement. We could have avoided the need for `i loc` by writing the whole body of `Mult` inside the accept statement, as follows:

```

task body Mult is
begin
    accept DoRow (i: in INTEGER ) do
        for j in 1..Size loop
            c(i, j) := 0;
            for k in 1..Size loop
                c(i, j) := c(i, j) + a(i, k)*b(k, j);
            end loop;
        end loop;
    end;
end Mult;

```

What is wrong with this solution?

- 13.41** This chapter ignored the question of formal semantics for concurrent programs. (Some studies are mentioned in the references.) Can you think of problems that may be encountered in applying axiomatic semantics to concurrent programs? What about denotational semantics?
- 13.42** The matrix multiplication solution in Ada of Figure 13.13 uses the size of the matrix to determine the number of tasks. Rewrite the solution to use a number of tasks specified as an input parameter to the `ParMult` function.
- 13.43** Or-parallelism in Prolog can be viewed as a parallel search of the tree of alternatives (see Chapter 4), where at each node a new process is created to search each child. Describe how these processes can coordinate their activities to allow backtracking.
- 13.44** Write a procedure to perform a parallel search of an unordered binary tree in:
- (a) Java
 - (b) Ada
- 13.45** Discuss the claim made in the text that or-parallelism is more natural for Prolog, whereas and-parallelism is more natural for Lisp.
- 13.46** Describe in detail the way or-parallel Prolog computes the three solutions to `delete([1, 2, 3], X, T)` in parallel. What time savings would you expect (in terms of numbers of steps)?
- 13.47** How much actual parallelism is performed in the and-parallel Prolog example of `generate` on page 625? Explain.
- 13.48** In and-parallel Prolog, if a guard encounters an uninstantiated variable, execution is suspended until the variable is instantiated (by another process). Why is this necessary? Are there any problems with this requirement?
- 13.49** Explain why backtracking is difficult in and-parallel Prolog.
- 13.50** In Figures 13.4 and 13.5, the main process creates new child processes to compute the matrix product and then suspends until all the child processes have finished. This is wasteful. Rewrite these programs so that the main process also performs some computations in parallel with its children.
- 13.51** Compare the notion of mailbox to that of a buffer.

- 13.52** Suppose that two processes both attempt to increment a shared variable S that is unprotected by a mutual exclusion mechanism. Suppose the assignment

$$S := S + 1$$

consists within each process of three machine instructions:

Load S into a reg
Increment the *reg*
Store the *reg* to S

What values might S have after both processes complete their increment operations? Show how each value can be obtained.

Notes and References

The field of parallel/concurrent programming is a huge one, involving operating system, architecture, and language issues. In this chapter, we have barely touched on specific language issues, using primarily Java and Ada as our examples. Typically, an entire course is devoted to parallel programming, often using one of the language-independent protocols supported by third-party libraries. Two examples of useful texts in this area are Andrews [2000] and Wilkinson and Allen [1997].

Surveys of parallel programming languages include the September 1989 issue of ACM Computing Surveys (Vol. 21, No. 3) and the July 1989 issue of IEEE Software. Some individual articles from these issues are also mentioned in the following. Another survey article is Andrews and Schneider [1983].

Programming using Java threads is studied in Lea [2000], Horstmann and Cornell [1999] (Vol. II), and Arnold et al. [2000]. For an alternative view of threads, see Butenhof [1997], which describes the Pthreads standard for UNIX. Programming using Ada tasks and Ada95 protected types is studied in Cohen [1996]; see also Wegner and Smolka [1983].

A survey of parallel architectures appears in Duncan [1990]. The diagrams in Section 13.1 are adapted from Karp [1987], where several methods of adding parallelism to FORTRAN are also studied. See also Karp and Babb [1988] for examples similar to those of Section 13.2.

Semaphores were introduced by Dijkstra [1968b]. See Stallings [2000] for a study of their use in operating systems. Monitors were introduced by Hoare [1974] and made part of the Concurrent Pascal language designed by Brinch-Hansen [1975].

The question of scheduling policies, fairness, and starvation were only mentioned briefly in this chapter, and the associated issue of process priorities was not discussed at all. For a discussion of the Java approach, see Lea [2000], especially Sections 3.4.1 and 3.7. For a discussion of the Ada95 approach, see Cohen [1996], Section 18.6.

Message passing, including a theoretical framework called Communicating Sequential Processes, or CSP, was introduced by Hoare [1978]. CSP is often used as a basis for formal semantics and verification of concurrent programs; see Schneider [2000]. CSP was also the inspiration for the programming language **Occam**, designed to run on a proprietary parallel system called the **transputer**; see Jones and Goldsmith [1988]. For a different approach to formal semantics of concurrent programs using a technique called **process algebra**, see Milner [1994].

A description of Multilisp is in Halstead [1985]. QLisp is described in Goldman and Gabriel [1989]. Surveys of parallel logic programming include Kergommeaux and Codognet [1994], Ciancarini [1992], Shapiro [1989], and Tick [1991]. Parlog is studied in Conlon [1989], Gregory [1987], and Ringwood [1988]. FGHC is described in Ueda [1987]. Concurrency and constraints in logic programming (Chapter 4) are closely related; see Ueda [2000] or Saraswat and Van Hentenryck [1995].

Information on various protocols for distributed parallel computing on networks can be found in Horstmann and Cornell [1999], Vol. II (Java's RMI), and Pacheco [1996] (MPI). Books on CORBA and COM are too numerous to mention, since these protocols are used for program interoperability as well as concurrency. A brief introduction can be found in Horstmann and Cornell [1999].

Several projects have used the Internet as a large parallel processing system; perhaps the best known is the Great Internet Mersenne Prime Search Project (see www.mersenne.org), which currently holds the record for the largest known prime number.

Erlang is described in Armstrong [2007] and [2010].

- Abelson et al. 1998. "Revised Report on the Algorithmic Language Scheme," *Higher-Order and Symbolic Computation* 11(1): 5–105, August 1998. (Also known as R²RS.)
- Abelson, H. and G. J. Sussman with Julie Sussman. 1996. *Structure and Interpretation of Computer Programs* (2nd ed.). Cambridge, MA: MIT Press.
- Aggoun, A. and N. Beldiceanu. 1991. "Overview of the CHIP Compiler System," in K. Furukawa (Ed.) *Logic Programming, Proceedings of the Eighth International Conference, Paris, France*, Cambridge: MIT Press, pp. 775–789.
- Aho, A. V., J. E. Hopcroft, and J. D. Ullman. 1983. *Data Structures and Algorithms*. Reading, MA: Addison-Wesley.
- Aho, A. V., B. W. Kernighan, and P. J. Weinberger. 1988. *The AWK Programming Language*. Reading, MA: Addison-Wesley.
- Aho, A. V., R. Sethi, and J. D. Ullman. 1986. *Compilers: Principles, Techniques, and Tools*. Reading, MA: Addison-Wesley.
- Alexandrescu, A. 2001. *Modern C++ Design: Generic Programming and Design Patterns Applied*. Reading, MA: Addison-Wesley.
- Andrews, G. R. 2000. *Foundations of Multithreaded, Parallel, and Distributed Programming*. Reading, MA: Addison-Wesley.
- Andrews, G. R. and F. B. Schneider. 1983. "Concepts and Notations for Concurrent Programming." *ACM Computing Surveys* 15(1): 3–43.
- ANSI-1815A. 1983. *Military Standard: Ada Programming Language*. Washington, DC: American National Standards Institute.
- ANSI-X3.226. 1994. *Information Technology—Programming Language—Common Lisp*. Washington, DC: American National Standards Institute.
- Antoniou, G. and M. Williams. 1997. *Nonmonotonic Reasoning*. Cambridge, MA: MIT Press.
- Antoy, S. and Hanus, M. 2010. "Functional Logic Programming," *Communications of the ACM*, (53)4, pp. 74–85.
- Appel, A. 1992. *Compiling with Continuations*. Cambridge, UK: Cambridge University Press.
- Appel, A. and D. MacQueen. 1994. "Separate Compilation for Standard ML," *SIGPLAN Conference on Programming Language Design and Implementation*.
- Apt, K. R., V. W. Marek, M. Truszczyński, and D. S. Warren (Eds.). 1999. *The Logic Programming Paradigm: A 25-Year Perspective*. New York: Springer-Verlag.
- Armstrong, J. [2010] "Erlang," *Communications of the ACM*, (53)9: 68–75.
- Armstrong, J. [2007] *Programming Erlang: Software for a Concurrent World*. Raleigh, NC: Pragmatic Bookshelf.
- Arnold, K., J. Gosling, and D. Holmes. 2000. *The Java Programming Language* (3rd ed.). Reading, MA: Addison-Wesley.
- Ashley, R. 1980. *Structured COBOL: A Self-Teaching Guide*. Hoboken, NJ: John Wiley & Sons.
- Baase, S. 1988. *Computer Algorithms: Introduction to Design and Analysis* (2nd ed.). Reading, MA: Addison-Wesley.
- Backus, J. W. 1981. "The History of FORTRAN I, II, and III." In Wexelblat [1981], pp. 25–45.
- Backus, J. W. 1978. "Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs." *Comm. ACM* 21(8): 613–641.
- Backus, J. W. et al. 1957. "The FORTRAN Automatic Coding System." *Proceedings of the Western Joint Computing Conference*, pp. 188–198. Reprinted in Rosen [1967], pp. 29–47.
- Barendregt, H. 1984. *The Lambda Calculus: Its Syntax and Semantics* (Revised Edition). Amsterdam: North-Holland.
- Barnes, J. [2006] *Programming in Ada 2005*. London: Pearson Education Limited.
- Barnes, J. G. P. 1980. "An overview of Ada." *Software Practice and Experience* 10, 851–887. Also reprinted in Horowitz [1987].
- Barron, D. W. 1977. *An Introduction to the Study of Programming Languages*. Cambridge, UK: Cambridge University Press.
- Bergin, T. J. and R. G. Gibson. 1996. *History of Programming Languages—II*. New York: ACM Press and Reading, MA: Addison-Wesley.
- Bird, R., T. E. Scruggs, and M. A. Mastropieri. 1998. *Introduction to Functional Programming*. Englewood Cliffs, NJ: Prentice-Hall.
- Birnes, W. J. (Ed.). 1989. *High-Level Languages and Software Applications*. New York: McGraw-Hill.
- Birtwistle, G. M., O.-J. Dahl, B. Myrhaug, and K. Nygaard. 1973. *Simula Begin*. Philadelphia: Auerbach.
- Bishop, J. 1986. *Data Abstraction in Programming Languages*. Reading, MA: Addison-Wesley.
- Björner, B. and O. N. Oest (Eds.). 1981. *Towards a Formal Description of Ada*, New York: Springer Verlag.
- Black, D. 2009. *The Well-Grounded Rubyist*. Manning Publications.
- Bobrow, D. G., L. G. DeMichiel, R. P. Gabriel, S. Keene, G. Kiczales, and D. A. Moon. 1988. "The Common Lisp Object System Specification." *ACM SIGPLAN Notices* 23(9) (special issue).
- Bobrow, D. G. and M. Stefik. 1983. *The LOOPS Manual*. Palo Alto, CA: Xerox Palo Alto Research Center.
- Böhm, C. and G. Jacopini. 1966. "Flow Diagrams, Turing Machines and Languages with Only Two Formation Rules." *Comm. ACM* 29(6), 471–483.
- Booch, G. 1994. *Object-Oriented Analysis and Design with Applications* (Addison-Wesley Object Technology Series) Reading, MA: Addison-Wesley.

- Booch, G., D. Bryan, and C. G. Petersen. 1993. *Software Engineering with Ada*. Reading, MA: Addison-Wesley.
- Bratko, I. 2000. *PROLOG Programming for Artificial Intelligence* (3rd ed.) Reading, MA: Addison-Wesley.
- Bridges, D. and F. Richman. 1987. *Varieties of Constructive Mathematics*, London Mathematical Society Lecture Note Series #97. Cambridge, UK: Cambridge University Press.
- Brinch-Hansen, P. 1999. "Java's Insecure Parallelism." *ACM SIGPLAN Notices* 34(4), pp. 38–45.
- Brinch-Hansen, P. 1996. "Monitors and Concurrent Pascal: A Personal History." In Bergin and Gibson [1996], pp. 121–172.
- Brinch-Hansen, P. 1981. "The Design of Edison." *Software Practice and Experience* 11, pp. 363–396.
- Brinch-Hansen, P. 1975. "The Programming Language Concurrent Pascal." *IEEE Transactions on Software Engineering* SE-1(2): 199–207.
- Brodie, L. 1981. *Starting FORTH: An Introduction to the FORTH Language*. Englewood Cliffs, NJ: Prentice-Hall.
- Brooks, F. 1996. "Language Design as Design." In Bergin and Gibson [1996], pp. 4–16.
- Bruce, K. [2002] *Foundations of Object-Oriented Languages: Types and Semantics*. Cambridge, MA: MIT Press.
- Budd, T. 1987. *A Little Smalltalk*. Reading, MA: Addison-Wesley.
- Budd, T. 1997. *An Introduction to Object-Oriented Programming*. Reading, MA: Addison-Wesley.
- Burks, A. W., H. H. Goldstine, and J. von Neumann. 1947. "Preliminary Discussion of the Logical Design of an Electronic Computing Instrument." In *John von Neumann: Collected Works*, Vol. V, pp. 34–79. New York: Macmillan, 1973.
- Burstall, R., D. MacQueen, and D. Sanella. 1980. *HOPE: An Experimental Applicative Language*. Report CSR-62-80, Computer Science Department, Edinburgh University, Scotland.
- Butenhof, D. R. 1997. *Programming with POSIX Threads*. Reading, MA: Addison-Wesley.
- Cardelli, L., J. Donahue, L. Glassman, M. Jordan, B. Kaslow, and G. Nelson. 1992. "Modula-3 Language Definition," *SIGPLAN Notices* 27(8): 15–42.
- Cardelli, L., J. Donahue, M. Jordan, B. Kaslow, and G. Nelson. 1989. "The Modula-3 Type System." *Sixteenth Annual ACM Symposium on Principles of Programming Languages*, pp. 202–212.
- Cardelli, L. and P. Wegner. 1985. "On Understanding Types, Data Abstraction, and Polymorphism." *ACM Computing Surveys* 17(4): 471–522.
- Carriero, N. and D. Gelernter. 1990. *How to Write Parallel Programs: A First Course*. Cambridge, MA: MIT Press.
- Carriero, N. and D. Gelernter. 1989a. "How to Write Parallel Programs: A Guide to the Perplexed." *ACM Computing Surveys* 21(3): 323–357.
- Carriero, N. and D. Gelernter. 1989b. "Linda in Context." *Comm. ACM* 32(4): 444–458.
- Chapman, S. J. 1997. *Fortran 90/95 for Scientists and Engineers*. New York: McGraw-Hill.
- Chomski, N. A. 1956. "Three Models for the Description of Language." *I. R. E. Transactions on Information Theory* IT-2(3): 113–124.
- Church, A. 1941. *The Calculi of Lambda Conversion*. Princeton, NJ: Princeton University Press.
- Ciancarini, P. 1992. "Parallel Programming with Logic Languages." *Computer Languages* 17(4): 213–239.
- Clark, R. L. 1973. "A Linguistic Contribution to GOTO-less Programming." *Datamation* 19(12), 62–63. Reprinted in *Comm. ACM* 27(4), April 1984.
- Clark, K. L. and S.A. Tärnlund (Eds.) 1982. *Logic Programming*. New York: Academic Press.
- Cleaveland, J. C. 1986. *An Introduction to Data Types*. Reading, MA: Addison-Wesley.
- Clocksin, W. F. and C. S. Mellish. 1994. *Programming in Prolog* (4th ed.). Berlin: Springer-Verlag.
- Cohen, J. 1988. "A View of the Origins and Development of Prolog." *Comm. ACM* 31(1), 26–37.
- Cohen, J. 1981. "Garbage Collection of Linked Data Structures." *ACM Computing Surveys* 13(3): 341–367.
- Cohen, N. 1996. *Ada as a Second Language* (2nd ed.). New York: McGraw-Hill.
- Colburn, D. R., C. H. Moore, and E. D. Rather. 1996. "The Evolution of FORTH." In Bergin and Gibson [1996], pp. 625–658.
- Colmerauer, A. and P. Roussel. 1996. "The Birth of Prolog." In Bergin and Gibson [1996], pp. 331–352.
- Colmerauer, A. 1982. "Prolog and Infinite Trees." In Clark and Tärnlund [1982].
- Conlon, T. 1989. *Programming in PARLOG*. Reading, MA: Addison-Wesley.
- Cooper, D. 1983. *Standard Pascal User Reference Manual*. New York: W. W. Norton.
- Cox, B. 1986. *Object-Oriented Programming: An Evolutionary Approach*. Reading, MA: Addison-Wesley.
- Cox, B. 1984. "Message/Object Programming: An Evolutionary Change in Programming Technology." *IEEE Software* 1(1): 50–69.
- Curry, H. B. and R. Feys. 1958. *Combinatory Logic*, Vol. 1. Amsterdam: North-Holland.
- Dahl, O.-J., E. W. Dijkstra, and C. A. R. Hoare. 1972. *Structured Programming*. New York: Academic Press.
- Dahl, O.-J. and K. Nygaard. 1966. "SIMULA—An Algol-based Simulation Language." *Comm. ACM* 9(9): 671–678.
- Dane, A. 1992. "Birth of an Old Machine." *Popular Mechanics*, March 1992, 99–100.
- Davis, R. E. 1982. "Runnable Specifications as a Design Tool." In Clark and Tärnlund [1982].
- Demers, A. J., J. E. Donahue, and G. Skinner. 1978. "Data Types as Values: Polymorphism, Type-checking, Encapsulation." *Conference Record of the Fifth Annual ACM Symposium on Principles of Programming Languages*, New York: ACM Press pp. 23–30.
- Dijkstra, E. W. and C. S. Scholten. 1990. *Predicate Calculus and Program Semantics (Texts and Monographs in Computer Science)*. New York: Springer-Verlag.

- Dijkstra, E. W. 1976. *A Discipline of Programming*. Englewood Cliffs, NJ: Prentice-Hall.
- Dijkstra, E. W. 1975. "Guarded Commands, Nondeterminacy, and the Formal Derivation of Programs." *Comm. ACM* 18(8): 453–457.
- Dijkstra, E. W. 1968a. "Goto Statement Considered Harmful" (letter to the editor). *Comm. ACM* 11(3): 147–148.
- Dijkstra, E. W. 1968b. "Co-operating sequential processes." In F. Genuys (ed.), *Programming Languages: NATO Advanced Study Institute*. New York: Academic Press.
- Donahue, J. E. and A. J. Demers. 1985. "Data Types Are Values." *ACM Transactions on Programming Languages and Systems* 7(3): 436–445.
- Duncan, R. 1990. "A Survey of Parallel Computer Architectures." *IEEE Computer* 23(2): 5–16.
- Dybvig, K. 1996. *The Scheme Programming Language: ANSI Scheme* (2nd ed.). Englewood Cliffs, NJ: Prentice-Hall.
- Ellis, M. A. and B. Stroustrup. 1990. *The Annotated C++ Reference Manual*. Reading, MA: Addison-Wesley.
- Falkoff, A. D. and K. Iverson. 1981. "The Evolution of APL." In Wexelblat [1981], pp. 661–674.
- Feinberg, N., Ed. 1996. *Dylan Programming : An Object-Oriented and Dynamic Language*. Reading, MA: Addison-Wesley.
- Feldman, M. and E. Koffman. 1999. *ADA 95: Problem Solving and Program Design* (3rd ed.). Reading, MA: Addison-Wesley.
- Flanagan, D. 1999. *Java in a Nutshell* (3rd ed.). O'Reilly & Associates, Inc.
- Flanagan, D. and Matsumoto, Y. [2008] *The Ruby Programming Language*. Sebastopol: O'Reilly Media.
- Friedman, D. P., C. T. Haynes, and E. Kohlbecker. 1985. "Programming with Continuations." In P. Pepper (Ed.), *Program Transformations and Programming Environments*. New York: Springer-Verlag.
- Futatsugi, K., J. Goguen, J.-P. Jouannaud, and J. Meseguer. 1985. "Principles of OBJ2." In *Proceedings of the ACM Symposium on Principles of Programming Languages*, pp. 52–66.
- Gabriel, R. P. 1985. *Performance and Evaluation of Lisp Systems*. Cambridge, MA: MIT Press.
- Gabriel, R. 1996. *Patterns of Software: Tales from the Software Community*. Oxford, UK: Oxford University Press.
- Gabriel, R. P., J. L. White, and D. G. Bobrow. 1991. "CLOS: Integrating Object-oriented and Functional Programming." *Comm. ACM* 34(9): 29–38.
- Gehani, N. H. 1984. *Ada: Concurrent Programming*. Englewood Cliffs, NJ: Prentice-Hall.
- Gelernter, D. and S. Jagannathan. 1990. *Programming Linguistics*. Cambridge, MA: MIT Press.
- Geschke, C., J. Morris, and E. Satterthwaite. 1977. "Early Experience with Mesa." *Comm. ACM* 20(8): 540–553.
- Ghezzi, C. and M. Jazayeri. 1997. *Programming Language Concepts* (3rd ed.). New York: John Wiley & Sons.
- Goguen, J. and G. Malcolm. 1997. *Algebraic Semantics of Imperative Programs*. Cambridge, MA: MIT Press.
- Goguen, J. and G. Malcolm (Eds.). 2000. *Software Engineering with OBJ: Algebraic Specification in Action*. Amsterdam: Kluwer Academic Publishers.
- Goguen, J. A., J. W. Thatcher, and E. G. Wagner. 1978. "An Initial Algebra Approach to the Specification, Correctness, and Implementation of Abstract Data Types." In Yeh [1978], pp. 80–149.
- Goldman, R. and R. P. Gabriel. 1989. "Qlisp: Parallel Processing in Lisp." *IEEE Software*, July 1989, 51–59.
- Goodenough, J. B. 1975. "Exception Handling: Issues and a Proposed Notation." *Comm. ACM* 16(7): 683–696.
- Gosling J., B. Joy, G. Steele, and G. Bracha. 2000. *The Java Language Specification* (2nd ed.). Reading, MA: Addison-Wesley.
- Graham, P. 1996. *ANSI Common Lisp*. Englewood Cliffs, NJ: Prentice-Hall.
- Graham, P. 2004. *Hackers & Painters: Big Ideas from the Computer Age*. Cambridge, MA: O'Reilly.
- Gregory, S. 1987. *Parallel Logic Programming in PARLOG: The Language and Its Implementation*. Reading, MA: Addison-Wesley.
- Gries, D. 1981. *The Science of Programming*. New York: Springer-Verlag.
- Griswold, R. E. 1981. "A History of the SNOBOL Programming Languages." In Wexelblat [1981], pp. 601–645.
- Griswold, R. E. and M. Griswold. 1996. "History of the ICON Programming Language." In Bergin and Gibson [1996], pp. 599–621.
- Griswold, R. E. and M. Griswold. 1983. *The Icon Programming Language*. Englewood Cliffs, NJ: Prentice-Hall.
- Griswold, R. E. and M. Griswold. 1973. *A SNOBOL4 Primer*. Englewood Cliffs, NJ: Prentice-Hall.
- Griswold, R. E., J. F. Poage, and I. P. Polonsky. 1971. *The SNOBOL4 Programming Language* (2nd ed.). Englewood Cliffs, NJ: Prentice-Hall.
- Gunter, C. A. 1992. *Semantics of Programming Languages: Structures and Techniques (Foundations of Computing Series)*. Cambridge, MA: MIT Press.
- Gunter, C. A. and J. C. Mitchell (Eds.). 1994. *Theoretical Aspects of Object-Oriented Programming: Types, Semantics, and Language Design (Foundations of Computing)*. Cambridge, MA: MIT Press.
- Guttag, J. V. 1977. "Abstract Data Types and the Development of Data Structures." *Comm. ACM* 20(6): 396–404.
- Halstead, R. H., Jr. 1985. "Multilisp: A Language for Concurrent Symbolic Computation." *ACM Transactions on Programming Languages and Systems* 7(4): 501–538.
- Hankin, C. 1995. *Lambda Calculi : A Guide for Computer Scientists* (Graduate Texts in Computer Science, No 3). Oxford: Clarendon Press.
- Hanson, D. R. 1981. "Is Block Structure Necessary?" *Software Practice and Experience* 11(8): 853–866.

- Harper, R. and Mitchell, J. 1993. "On the Type Structure of Standard ML." *ACM Transactions on Programming Languages and Systems* 15(2), April 1993, 211–252.
- Harry, A. 1997. *Formal Methods Fact File: Vdm and Z (Wiley Series in Software Engineering Practice)*, New York: John Wiley & Sons.
- Henderson, P. 1980. *Functional Programming: Application and Implementation*. Englewood Cliffs, NJ: Prentice-Hall.
- Henglein, F. [1993] "Type Inference with Polymorphic Recursion." *ACM Transactions on Programming Languages and Systems* 15(2), April 1993, 253–289.
- Hindley, J. R. 1997. *Basic Simple Type Theory (Cambridge Tracts in Theoretical Computer Science, No 42)*. Cambridge, England: Cambridge University Press.
- Hindley, J. R. 1969. "The Principal Type-scheme of an Object in Combinatory Logic." *Trans. Amer. Math. Soc.* 146(12): 29–60.
- Hoare, C. A. R. 1981. "The Emperor's Old Clothes." *Comm. ACM* 24(2): 75–83.
- Hoare, C. A. R. 1978. "Communicating Sequential Processes." *Comm. ACM* 21(8): 666–677.
- Hoare, C. A. R. 1974. "Monitors: An Operating System Structuring Concept." *Comm. ACM* 17(10): 549–557.
- Hoare, C. A. R. 1973. "Hints on Programming Language Design." *ACM SIGACT/SIGPLAN Symposium on Principles of Programming Languages*. Reprinted in Horowitz [1987], pp. 31–40.
- Hoare, C. A. R. 1969. "An Axiomatic Basis for Computer Programming." *Comm. ACM* 12(10): 576–580, 583.
- Hoare, C. A. R. and N. Wirth. 1966. "A Contribution to the Development of ALGOL." *Comm. ACM* 9(6): 413–431.
- Hopcroft, J. E. and J. D. Ullman. 1979. *Introduction to Automata Theory, Languages, and Computation*. Reading, MA: Addison-Wesley.
- Horn, A. 1951. "On Sentences Which Are True of Direct Unions of Algebras." *J. Symbolic Logic* 16: 14–21.
- Horowitz, E. 1987. *Programming Languages: A Grand Tour* (3rd ed.). Rockville, MD.: Computer Science Press.
- Horowitz, E. 1984. *Fundamentals of Programming Languages* (2nd ed.). Rockville, Md.: Computer Science Press.
- Horowitz, E. and S. Sahni. 1984. *Fundamentals of Data Structures in Pascal*. Rockville, MD.: Computer Science Press.
- Horstmann, C. 2006. *Object-Oriented Design & Patterns*. Hoboken, NJ: John Wiley & Sons.
- Horstmann, C. and G. Cornell. 1999. *Core JAVA 1.2, Vols I and II*. Palo Alto, CA: Sun Microsystems Press/Prentice-Hall PTR.
- Hudak, P. 1989. "Conception, Evolution, and Application of Functional Programming Languages." *ACM Computing Surveys* 21(3): 359–411.
- Hudak, P. 2000. *The Haskell School of Expression: Learning Functional Programming through Multimedia*, New York: Cambridge University Press.
- Hui, R. K. W. et al. 1990. "APL?" in *APL90 Conf. Proc.*, 20(4): 192–200.
- Ichbiah, J. D., J. G. P. Barnes, J. C. Heliard, B. Krieg-Brueckner, O. Roubine, and B. A. Wichmann. 1979. "Rationale for the Design of the Ada Programming Language." *ACM SIGPLAN Notices* 14(6), Part B.
- IEEE. 1985. "ANSI/IEEE Std 754-1985: Standard for Binary Floating-Point Arithmetic." Reprinted in *ACM SIGPLAN Notices* 22(2): 9–25.
- IEEE P1178. 1991. *IEEE Standard for the Scheme Programming Language*.
- ISO 8652. 1995. ISO/IEC Standard—Information technology—Programming languages—Ada. International Organization for Standardization, Geneva, Switzerland.
- ISO 9899. 1999. ISO/IEC Standard—Information technology—Programming languages—C. International Organization for Standardization, Geneva, Switzerland.
- ISO 13211-1. 1995. ISO/IEC Standard—Information technology—Programming languages—Prolog-Part 1: General core. International Organization for Standardization, Geneva, Switzerland.
- ISO 14882-1. 1998. ISO/IEC Standard—Information technology—Programming languages—C++. International Organization for Standardization, Geneva, Switzerland.
- ISO 14977. 1996. ISO/IEC Standard—Information technology—Syntactic metalanguage—Extended BNF. International Organization for Standardization, Geneva, Switzerland.
- Iverson, K. 1962. *A Programming Language*. Hoboken, NJ: John Wiley & Sons.
- Jaffar, J., S. Michaylov, P. Stuckey, and R. Yap. 1992. "The CLP(R) Language and System," *ACM TOPLAS*, 14(3): 339–395.
- Jaffar, J. and Maher, M. 1994. "Constraint Logic Programming: A Survey," *Journal of Logic Programming*, 19/20, 503–581.
- Johnson, S. C. 1975. "Yacc—Yet Another Compiler Compiler," Computing Science Technical Report No. 32. Murray Hill, N.J.: AT&T Bell Laboratories.
- Jones, G. and M. Goldsmith. 1988. *Programming in Occam 2*. Englewood Cliffs, N.J.: Prentice-Hall.
- Josuttis, N. 1999. *The C++ Standard Library—A Tutorial and Reference*. Reading, MA: Addison-Wesley.
- Kamin, S. 1983. "Final Data Types and Their Specification." *ACM Trans. on Programming Languages and Systems* 5(1): 97–123.
- Karp, A. H. 1987. "Programming for parallelism." *IEEE Computer* 21(5): 43–57.
- Karp, A. H. and R. O. Babb II. 1988. "A Comparison of 12 Parallel Fortran Dialects." *IEEE Software*, September 1988, 52–67.
- Kay, A. C. 1996. "The Early History of Smalltalk." In Bergin and Gibson [1996], pp. 511–579.

- Kergommeaux, J. C. and P. Codognet. 1994. "Parallel LP Systems," *ACM Computing Surveys* 26(3), 295–336.
- Kernighan, B. W. and D. M. Ritchie. 1988. *The C Programming Language* (ANSI Standard C) (2nd ed.). Englewood Cliffs, N.J.: Prentice-Hall.
- King, K. N. 1988. *Modula-2: A Complete Guide*. Lexington, MA.: D. C. Heath.
- Knuth, D. E. 1974. "Structured Programming with GOTO Statements." *ACM Computing Surveys* 6(4), 261–301.
- Knuth, D. E. 1972. "Ancient Babylonian Algorithms." *Comm. ACM* 15(7), 671–677.
- Knuth, D. E. 1967. "The Remaining Trouble Spots in Algol60." *Comm. ACM* 10(10), 611–617. Also reprinted in Horowitz [1987], pp. 61–68.
- Knuth, D. E. and L. Trabb Pardo. 1977. "Early Development of Programming Languages." In *Encyclopedia of Computer Science and Technology*, Vol. 7, pp. 419–493. New York: Marcel Dekker.
- Koenig, A. and B. Moo. 1996. *Ruminations on C++: A Decade of Programming Insight and Experience*. Reading, MA: Addison-Wesley.
- Koenig, A. and B. Stroustrup. 1990. "Exception Handling for C++," *Proceedings of the USENIX C++ Conference* (pp. 149–176), San Francisco, April 1990; reprinted in *JOOP* Vol. 3, No. 2, July/Aug 1990, pp. 16–33.
- Kowalski, R. A. 1988. "The Early Years of Logic Programming." *Comm. ACM* 31(1): 38–43.
- Kowalski, R. A. 1979a. "Algorithm 5 Logic 1 Control." *Comm. ACM* 22(7): 424–436.
- Kowalski, R. A. 1979b. *Logic for Problem Solving*. New York: Elsevier/North-Holland.
- Kurtz, T. E. 1981. "BASIC." In Wexelblat [1981], pp. 515–537.
- Lajoie, J. 1994a. "Exception Handling: Supporting the Runtime Mechanism." *C++ Report* (March/April 1994).
- Lajoie, J. 1994b. "Exception Handling: Behind the Scenes." *C++ Report* (June 1994).
- Lalonde, W. 1994. *Discovering Smalltalk* (Addison-Wesley Object Technology Series). Reading, MA: Addison-Wesley.
- Lambert, K. and M. Osborne. 1997. *Smalltalk in Brief: Introduction to Object-Oriented Software Development*. Boston: PWS.
- Lambert, K. and D. Nance. 2001. *Fundamentals of C++*. Boston: Course Technology.
- Lambert, K. 2010. *Fundamentals of Python: From First Programs through Data Structures*. Boston: Course Technology.
- Lambert, K. and M. Osborne. 2010. *Fundamentals of Java*. Boston: Course Technology.
- Lampson, B. W. 1983. "A Description of the Cedar Language," Technical Report CSL-83-15. Palo Alto, CA: Xerox Palo Alto Research Center.
- Lampson, B. W., J. J. Horning, R. L. London, J. G. Mitchell, and G. J. Popek. 1981. "Report on the Programming Language Euclid," Technical Report CSL-81-12. Palo Alto, CA: Xerox Palo Alto Research Center.
- Lampson, B. W. and D. Redell. 1980. "Experience with Processes and Monitors in Mesa." *Comm. ACM* 23(2): 105–117.
- Landin, P. J. 1966. "The Next 700 Programming Languages." *Comm. ACM* 9(3), 157–165.
- Lapalme, G. and F. Rabhi. 1999. *Algorithms: A Functional Programming Approach*. Reading, MA: Addison-Wesley.
- Lea, D. 2000. *Concurrent Programming in Java: Design Principles and Patterns*. (2nd ed.). Reading, MA: Addison-Wesley.
- Lenkov, D., D. Cameron, P. Faust, and M. Mehta. 1992. "A Portable Implementation of C++ Exception Handling," *Proceedings of the Usenix C++ Conference*, Portland, OR.
- Lesk, M. E. 1975. "Lex—A Lexical Analyzer Generator," Computing Science Technical Report No. 39. Murray Hill, NJ: AT&T Bell Laboratories.
- Lewis, J., M. Shields, E. Meijer, and J. Launchbury. 2000. "Implicit Parameters: Dynamic Scoping with Static Types." In *Proceedings of the 27th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pp. 108–118. New York: ACM Press.
- Lewis, S. 1995. *The Art and Science of Smalltalk*. Englewood Cliffs, NJ: Prentice-Hall.
- Lindsey, C. H. 1996. "A History of Algol 68." In Bergin and Gibson [1996], pp. 27–84.
- Lippman, S. and J. Lajoie 1998. *C++ Primer* (3rd ed.). Reading, MA: Addison-Wesley.
- Liskov, B. 1996. "A History of CLU." In Bergin and Gibson [1996], pp. 471–497.
- Liskov, B., R. Atkinson, T. Bloom, E. Moss, J. C. Schaffert, R. Scheifler, and A. Snyder. 1984. *CLU Reference Manual*. New York: Springer-Verlag.
- Liskov, B. and A. Snyder. 1979. "Exception Handling in CLU." *IEEE Transactions on Software Engineering* SE-5(6): 546–558. Also reprinted in Horowitz [1987], pp. 254–266.
- Liskov, B., A. Snyder, R. Atkinson, and C. Schaffert. 1977. "Abstraction Mechanisms in CLU." *Comm. ACM* 20(8): 564–576. Also reprinted in Horowitz [1987], pp. 267–279.
- Liu, C. 2000. *Smalltalk, Objects, and Design*. Lincoln, NE: iUniverse.com.
- Lloyd, J. W. 1984. *Foundations of Logic Programming*. New York: Springer-Verlag.
- Louden, K. 1987. "Recursion versus Non-recursion in Pascal: Recursion Can Be Faster." *ACM SIGPLAN Notices* 22(2): 62–67.
- Louden, K. 1997. *Compiler Construction: Principles and Practice*. Boston: PWS.
- Louden, K. 1997b. "Compilers and Interpreters." In Tucker [1997], pp. 2120–2147.
- Louden, K. 2004 "Compilers and Interpreters," in *The Computer Science and Engineering Handbook*, 2nd Edition (2004), Allen B. Tucker (Editor), CRC Press.
- Luckam, D. C. and W. Polak. 1980. "Ada Exception Handling: An Axiomatic Approach." *ACM Transactions on Programming Languages and Systems* 2(2), 225–233.
- MacLennan, B. *Functional Programming: Practice and Theory*. 1990. Reading, MA: Addison-Wesley.

- MacQueen, D. B. 1988. "The Implementation of Standard ML Modules." *ACM Conference on Lisp and Functional Programming*, 212–223. New York: ACM Press.
- Malpas, J. *Prolog: A Relational Language and Its Applications*. 1987. Prentice Hall.
- Mandrioli, D. and C. Ghezzi. 1987. *Theoretical Foundations of Computer Science*. Hoboken, NJ: John Wiley & Sons.
- Marriott, K. and P. J. Stuckey (Eds.). 1997. *Programming with Constraints*. Cambridge, MA: MIT Press.
- Martin-Löf, P. 1979. "Constructive Mathematics and Computer Programming." In L. J. Cohen et al. (Eds.) *Logic, Methodology and the Philosophy of Science*, Vol. VI. New York: North-Holland. 1982.
- McCarthy, J. 1981. "History of LISP." In Wexelblat [1981], pp. 173–185.
- Metcalfe, M. and J. Reid. 1999. *Fortran 90/95 Explained* (2nd ed.). Oxford, UK: Oxford University Press.
- Meyer, B. 1997. *Object-Oriented Software Construction* (2nd ed.). Englewood Cliffs, NJ: Prentice-Hall.
- Milner, R. 1978. "A Theory of Type Polymorphism in Programming." *J. Computer and System Sciences* 17(3): 348–375.
- Milner, R. 1994. *Communication and Concurrency*. Englewood Cliffs, NJ: Prentice-Hall.
- Milner, R. and M. Tofte. 1991. *Commentary on Standard ML*. Cambridge, MA: MIT Press.
- Milner, R., M. Tofte, R. Harper, and D. MacQueen. 1997. *The Definition of Standard ML (Revised)*. Cambridge, MA: MIT Press.
- Minker J. (Ed.). 2000. *Logic-Based Artificial Intelligence* (The Kluwer International Series in Engineering and Computer Science Volume 597). Amsterdam: Kluwer Academic Publishers.
- Mitchell, J. C. 1996. *Foundations for Programming Languages (Foundations of Computing Series)*. Cambridge, MA: MIT Press.
- Mitchell, J. C. and R. Harper. 1988. "The Essence of ML." *Fifteenth ACM Symposium on Principles of Programming Languages*, New York: ACM Press, pp. 28–46.
- Mitchell, J. G., W. Maybury, and R. Sweet. 1979. "Mesa Language Manual, Version 5.0," Technical Report CSL-79-3. Palo Alto, CA: Xerox Palo Alto Research Center.
- Moon, D. A. 1986. "Object-oriented Programming with Flavors." *OOPSLA 1986, ACM SIGPLAN Notices* 21(11): 1–8.
- Moon, D. A. 1984. "Garbage Collection in Large Lisp Systems." *Proceedings of the 1984 ACM Symposium on Lisp and Functional Programming*, New York: ACM Press, pp. 235–246.
- Moore, D. L. 1977. *Ada, Countess of Lovelace: Byron's Legitimate Daughter*. London: Murray.
- Morrison, P. and E. Morrison (Eds.). 1961. *Charles Babbage and His Calculating Engines*. New York: Dover.
- Muhlbacher, J. (Ed.). 1997. *Oberon-2 Programming with Windows*. New York: Springer-Verlag, 1997.
- Naur, P. 1981. "The European Side of the Last Phase of the Development of Algol 60." In Wexelblat [1981], pp. 92–139.
- Naur, P. (Ed.). 1963a. "Revised Report on the Algorithmic Language Algol 60." *Comm. ACM* 6(1): 1–17. Also reprinted in Horowitz [1987], pp. 44–60.
- Naur, P. 1963b. "GOTO Statements and Good Algol Style." *BIT* 3(3): 204–208.
- Nelson, G. (Ed.). 1991. *Systems Programming with Modula-3*. Englewood Cliffs, NJ: Prentice-Hall.
- Nelson, G. 1989. "A Generalization of Dijkstra's Calculus." *ACM Transactions on Programming Languages and Systems* 11(4): 517–561.
- Nygaard, K. and O.-J. Dahl. 1981. "The Development of the SIMULA Languages." In Wexelblat [1981], pp. 439–480.
- Okasaki, C. 1999. *Purely Functional Data Structures*. Cambridge, UK: Cambridge University Press.
- O'Donnell, M. J. 1985. *Equational Logic as a Programming Language*. Cambridge, MA: MIT Press.
- OOPSLA. 1986ff. ACM Conference on Object-Oriented Programming Systems and Languages. Also published as various issues of *ACM SIGPLAN Notices*.
- Ousterhout, J. K. 1998. "Scripting: Higher-Level Programming for the 21st Century," *IEEE Computer* 31(3): pp. 23–30.
- Pacheco, P. 1996. *Parallel Programming with MPI*. San Francisco: Morgan Kaufmann.
- Pamas, D. L. 1985. "Software Aspects of Strategic Defense Systems." *Comm. ACM* 28(12): 1326–1335. Reprinted from the *American Scientist* 73(5): 432–440.
- Paulson, L. 1996. *ML for the Working Programmer* (2nd ed.). Cambridge, UK: Cambridge University Press.
- Perlis, A. J. 1981. "The American Side of the Development of Algol." In Wexelblat [1981], pp. 75–91.
- Peyton Jones, S. L. 1987. *The Implementation of Functional Programming Languages*. Englewood Cliffs, NJ: Prentice-Hall.
- Popek, G. J., J. J. Homing, B. W. Lampson, J. G. Mitchell, and R. L. London. 1977. "Notes on the Design of Euclid." *ACM SIGPLAN Notices* 12(3): 11–19.
- Pountain, D. 1995. "Constraint Logic Programming: A Child of Prolog Finds New Life Solving Programming's Nastier Problems," *BYTE*, February 1995.
- Radin, G. 1981. "The Early History and Characteristics of PL/I." In Wexelblat [1981], pp. 551–575.
- Randell, B. and L. J. Russell. 1964. *Algol 60 Implementation*. New York: Academic Press.
- Reade, C. 1989. *Elements of Functional Programming*. Reading, MA: Addison-Wesley.

- Reynolds, J. C. 1998. *Theories of Programming Languages*. Cambridge, UK: Cambridge University Press.
- Richards, M. and C. Whitby-Strevens. 1979. *BCPL—The Language and Its Compiler*. Cambridge, UK: Cambridge University Press.
- Ringwood, G. A. 1988. "Parlog86 and the Dining Logicians." *Comm. ACM* 31(1): 10–25.
- Ripley, G. D. and F. C. Druseikis. 1978. "A Statistical Analysis of Syntax Errors." *Computer Languages* 3(4): 227–240.
- Ritchie, D. M. 1996. "The Development of the C Programming Language." In Bergin and Gibson [1996], pp. 671–687.
- Roberts, E. [1995] "Loop Exits and Structured Programming: Reopening the Debate," *ACM SIGCSE Bulletin*, (27)1: 268–272.
- Robinson, J. A. 1965. "A Machine-Oriented Logic Based on the Resolution Principle." *Journal of the ACM* 12(1): 23–41.
- Rosen, S. 1972. "Programming Systems and Languages 1965–1975." *Comm. ACM* 15(7): 591–600.
- Rosen, S. (Ed.). 1967. *Programming Systems and Languages*. New York: McGraw-Hill.
- Rosser, J. B. 1982. "Highlights of the History of the Lambda Calculus." *Proceedings of the ACM Symposium on Lisp and Functional Programming*, New York: ACM Press, pp. 216–225.
- Rubin, F. 1987. "'GOTO Statement Considered Harmful' Considered Harmful" (letter to the editor). *Comm. ACM* 30(3), pp. 195–196. Replies in the June, July, August, November, and December 1987 issues.
- Sammet, J. E. 1981. "The Early History of COBOL." In Wexelblat [1981], pp. 199–243.
- Sammet, J. E. 1976. "Roster of Programming Languages for 1974–75." *Comm. ACM* 19(12): 655–699.
- Sammet, J. E. 1972. "Programming Languages: History and Future." *Comm. ACM* 15(7): 601–610.
- Sammet, J. E. 1969. *Programming Languages: History and Fundamentals*. Englewood Cliffs, NJ: Prentice-Hall.
- Saraswat, V. and P. Van Hentenryck (Eds.). 1995. *Principles and Practice of Constraint Programming: The Newport Papers*. Cambridge, MA: MIT Press.
- Schmidt, D. A. 1994. *The Structure of Typed Programming Languages (Foundations of Computing Series)*. Cambridge, MA: MIT Press.
- Schmidt, D. A. 1986. *Denotational Semantics: A Methodology for Language Development*. Dubuque, IA: Wm. C. Brown.
- Schneider, S. 2000. *Concurrent and Real-time Systems: The CSP Approach*. Hoboken, NJ: John Wiley & Sons.
- Schneiderman, B. 1985. "The Relationship between COBOL and Computer Science." *Annals of the History of Computing* 7(4): 348–352. Also reprinted in Horowitz [1987], pp. 417–421.
- Schönfinkel, M. 1924. "Über die Bausteine der mathematischen Logik." *Mathematische Annalen* 92(3/4): 305–316.
- Scott, M. L. 2000. *Programming Language Pragmatics*. San Francisco: Morgan Kaufmann.
- Sebesta, R. W. 1996. *Concepts of Programming Languages* (3rd ed.). Reading, MA: Addison-Wesley.
- Seibel, P. 2005. *Practical Common Lisp*. Berkeley: Apress.
- Sethi, R. 1996. *Programming Languages Concepts and Constructs* (2nd ed.). Reading, MA: Addison-Wesley.
- Shapiro, E. 1989. "The Family of Concurrent Logic Programming Languages." *ACM Computing Surveys* 21(3), 412–510.
- Shapiro, E. and D. H. D. Warren (Eds.). 1993. "Special Section: The Fifth Generation Project: Personal Perspectives, Launching the New Era, and Epilogue," *Communications of the ACM*, 36(3): 46–103.
- Sharp, A. 1997. *Smalltalk by Example: The Developer's Guide*. New York: McGraw-Hill.
- Shaw, C. J. 1963. "A Specification of JOVIAL." *Comm. ACM* 6(12): 721–736.
- Shaw, Mary (Ed.). 1981. *Alphard Form and Content*. New York: Springer-Verlag.
- Sipser, M. 1997. *Introduction to the Theory of Computation*. Boston: PWS.
- Snyder, A. 1986. "Encapsulation and Inheritance in Object-oriented Languages." *ACM SIGPLAN Notices* 21(11): 38–45.
- Sperber, M., Dybvig, K., Flatt, M., and van Straaten, A. [2010] *Revised [6] Report of the Algorithmic Language Scheme*. Cambridge, UK: Cambridge University Press.
- Springer, G. and D. P. Friedman. 1989. *Scheme and the Art of Programming*. Cambridge, MA: MIT Press.
- Stallings, W. 2000. *Operating Systems: Internals and Design Principles*. Englewood Cliffs, NJ: Prentice-Hall.
- Steele, G. L., Jr., and R. P. Gabriel. 1996. "The Evolution of Lisp." In Bergin and Gibson [1996], pp. 233–309.
- Steele, G. 1984. *Common Lisp: The Language*. Burlington, MA: Digital Press.
- Steele, G. 1982. "An Overview of Common Lisp." *Proceedings of the ACM Symposium on Lisp and Functional Programming*, pp. 98–107. New York: ACM Press.
- Steele, G. 1977. "Debunking the 'Expensive Procedure Call' Myth." *Proceedings of the National Conference of the ACM*, pp. 153–162. New York: ACM Press.
- Stein, D. 1985. *Ada: A Life and Legacy*. Cambridge, MA: MIT Press.
- Sterling, L. and E. Shapiro. 1986. *The Art of Prolog*. Cambridge, MA: MIT Press.
- Stoy, J. E. 1977. *Denotational Semantics: The Scott-Strachey Approach to Programming Language Semantics*. Cambridge, MA: MIT Press.
- Stroustrup, B. 1997. *The C++ Programming Language* (3rd ed.). Reading, MA: Addison-Wesley.
- Stroustrup, B. 1996. "A History of C++: 1979–1991." In Bergin and Gibson [1996], pp. 699–755.
- Stroustrup, B. 1994. *The Design and Evolution of C++*. Reading, MA: Addison-Wesley.

- Sussman, G. J. and G. L. Steele, Jr. 1975. "Scheme: An Interpreter for Extended Lambda Calculus." AI Memo 349. Cambridge, MA: MIT Artificial Intelligence Laboratory.
- Swinehart, D. C., P. T. Zellweger, R. J. Beach, and R. B. Hagmann. 1986. "A Structural View of the Cedar Programming Environment." *ACM Transactions on Programming Languages and Systems* 8(4): 419–490.
- Tanenbaum, A. S. 1976. "A Tutorial on Algol68." *ACM Computing Surveys* 8(2): 155–190. Also reprinted in Horowitz [1987], pp. 69–104.
- Teitelman, W. 1984. "A Tour through Cedar." *IEEE Software*, April 1984, 44–73.
- Tesler, L. 1985. "Object Pascal Report." *Structured Language World* 9(3): 10–14.
- Thompson, S. 1999. *Haskell: The Craft of Functional Programming* (2nd ed.). Reading, MA: Addison-Wesley.
- Tick, E. 1991. *Parallel Logic Programming*. Cambridge, MA: MIT Press.
- Tucker, A. 1997. *The Computer Science and Engineering Handbook*. Boca Raton, FL: CRC Press.
- Turner, D. A. 1986. "An Overview of Miranda." *ACM SIGPLAN Notices* 21(12): 158–166.
- Turner, D. A. 1982. "Recursion Equations as a Programming Language." In Darlington J. et al. (Eds.), *Functional Programming and Its Applications*. Cambridge, UK: Cambridge University Press.
- Ueda, K. 1987. "Guarded Horn Clauses." In E. Y. Shapiro (Ed.), *Concurrent Prolog: Collected Papers*, Cambridge, MA: MIT Press, pp. 140–156.
- Ueda, K. 1999. "Concurrent Logic/Constraint Programming: The Next 10 Years." In Apt et al. [1999].
- Ullman, J. 1998. *Elements of ML Programming, ML97 Edition*. Englewood Cliffs, NJ: Prentice-Hall.
- Ungar, D. 1984. "Generation Scavenging: A Non-Disruptive High-Performance Storage Reclamation Algorithm." Proceedings of the ACM SIGSOFT/SIGPLAN Symposium on Practical Software Development Environments, *ACM SIGPLAN Notices* 19(5): 157–167.
- Ungar, D. and R. Smith. 1987. "SELF: The Power of Simplicity." *OOPSLA* 1987.
- Wall, L., T. Christiansen, and J. Orwant. 2000. *Programming Perl* (3rd ed.). Sebastopol, CA, O'Reilly & Associates.
- Warren, D. H. D. 1980. "Logic Programming and Compiler Writing." *Software Practice and Experience* 10(2): 97–125.
- Wegner, P. 1976. "Programming Languages—The First 25 Years." *IEEE Transactions on Computers* C-25(12): 1207–1225. Also reprinted in Horowitz [1987], pp. 4–22.
- Wegner, P. 1972. "The Vienna Definition Language." *ACM Computing Surveys* 4(1): 5–63.
- Wegner, P. and S. A. Smolka. 1983. "Processes, Tasks, and Monitors: A Comparative Study of Concurrent Programming Primitives." *IEEE Transactions on Software Engineering* SE, 9(4): 446–462. Also reprinted in Horowitz [1987], pp. 360–376.
- Welch, B. B. 2000. *Practical Programming in Tcl and Tk*. Englewood Cliffs, NJ: Prentice-Hall.
- Wexelblat, R. L. 1984. "Nth Generation Languages." *Datamation*, September 1, pp. 111–117.
- Wexelblat, R. L. (Ed.). 1981. *History of Programming Languages*. New York: Academic Press.
- Whitaker, W. A. 1996. "Ada—The Project: The DoD High Order Language Working Group." In Bergin and Gibson [1996], pp. 173–228.
- Whitehead, A. N. 1911. *An Introduction to Mathematics*. Oxford, UK: Oxford University Press.
- Wilkinson, B. and Allen, C. M. [1997]. *Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers*. Englewood Cliffs, NJ: Prentice-Hall.
- Wilson, P. R. 1992. "Uniprocessor Garbage Collection Techniques." In Bekkers et al. (Eds.), *International Workshop on Memory Management*, Springer Lecture Notes in Computer Science #637, pp. 1–42. New York: Springer-Verlag.
- Winograd, T. 1979. "Beyond Programming Languages." *Comm. ACM* 22(7): 391–401.
- Winskel, G. 1993. *The Formal Semantics of Programming Languages: An Introduction (Foundations of Computing Series)*. Cambridge, MA: MIT Press.
- Wirth, N. 1996. "Recollections about the Development of Pascal." In Bergin and Gibson [1996], pp. 97–111.
- Wirth, N. 1988a. *Programming in Modula-2* (4th ed.). Berlin: Springer-Verlag.
- Wirth, N. 1988b. "From Modula to Oberon." *Software Practice and Experience* 18(7): 661–670.
- Wirth, N. 1988c. "The Programming Language Oberon." *Software Practice and Experience* 18(7): 671–690.
- Wirth, N. 1976. *Algorithms 1 Data Structures 5 Programs*. Englewood Cliffs, NJ: Prentice-Hall.
- Wirth, N. 1974. "On the Design of Programming Languages." *Proc. IFIP Congress* 74, 386–393. Amsterdam: North-Holland. Reprinted in Horowitz [1987], pp. 23–30.
- Wirth, N. and H. Weber. 1966a. "Euler: A Generalization of Algol, and Its Formal Definition," Part 1. *Comm. ACM* 9(1): 13–23.
- Wirth, N. and H. Weber. 1966b. "Euler: A Generalization of Algol, and Its Formal Definition," Part II. *Comm. ACM* 9(2): 89–99.
- Wulf, W. A., R. L. London, and M. Shaw. 1976. "An Introduction to the Construction and Verification of Alphard Programs." *IEEE Transactions on Software Engineering* 2(4): 253–265.
- Wulf, W. A., D. B. Russell, and A. N. Habermann. 1971. "BLISS: A Language for Systems Programming." *Comm. ACM* 14(12): 780–790.
- Yeh, R. T. (Ed.). 1978. *Current Trends in Programming Methodology, Vol. IV, Data Structuring*. Englewood Cliffs, NJ: Prentice-Hall.
- Zuse, K. 1972. "Der Plankalkül." *Berichte der Gesellschaft für Mathematik und Datenverarbeitung*, No. 63, Part 3, Bonn.

A

- absolute values, 14
- abstract classes, 151, 158, 170
- abstract data type. *See* ADT (abstract data type)
- abstractions, 6, 8–14, 144
- abstract machine, 547
- abstract syntax, 216, 546
- abstract syntax trees, 213–216
- access chaining, 468
- access links, 467–468
- accessor methods, 165
- Access-type Lifetime Rule, 470, 472
- access types, 299, 351
- activated, 448
- activation, 12, 292, 404
 - procedures, 459–472
- activation records, 292, 445, 448–449, 460, 462, 464, 471
- actual parameters, 13, 403, 450
- Ada, 7
 - Access-type Lifetime Rule, 472
 - access types, 299, 351
 - aliases and subtypes rule, 357
 - arrays, 342–343
 - assignment operator (`:=`), 147
 - automatic garbage collection, 309
 - blocks, 262, 402
 - BNFs (Backus-Naur forms), 204
 - bounded buffer as task, 619–620
 - bounded buffer problem, 613–615
 - Cartesian products, 335
 - case statement, 415–416
 - casts, 365–355, 366
 - composite types, 351
 - constant declaration, 302
 - constrained parameterization, 529
 - dangling references, 307, 472
 - definitions, 257
 - dynamic allocation, 289
 - entry/accept mechanism, 617
 - enumerated types, 332–333
 - erroneous programs, 458
 - exception handlers, 428, 429
 - exceptions, 426, 427, 470
 - explicit conversions, 365
 - explicitly parameterized functions, 380
 - explicitly parameterized polymorphic functions, 381–382
 - floating-point operations, 327
 - for-loop, 419
 - function overloading, 284–285
 - functions, 446
 - generic packages, 194, 512–514
 - generic unit, 380
 - heap allocation, 295–296
 - identifiers, 205, 236
 - implicit conversions, 364, 365
 - index set, 340
 - instance variables, 195–196
 - integer types, 351
 - lexical addresses, 262
 - library I/O functions, 515

- limited private type, 528
- multidimensional arrays, 343–344
- named scopes, 279
- name equivalence, 356–357
- nested procedures, 465–466
- numeric types, 351
- operator overloading, 285–287, 364
- operators, 31
- overloading, 194, 282
- packages, 10, 193, 263, 380, 509–515, 529
- package specification, 380–381, 510, 511
- parallel matrix multiplication, 621–622
- parameter communication, 458
- parse trees, 313
- pointers, 346, 511
- procedures, 12–13, 446
- real types, 351
- reserved words, 236
- scalar types, 351
- scope, 265–267
- short-circuit Boolean operators, 407
- static allocation, 289
- statically typed, 39
- subranges, 333–334, 362–363
- subtypes, 339
- symbolic constant, 432
- symbol table, 280
- syntax analysis, 313
- task rendezvous, 616–621
- tasks, 14, 594, 611–612
- termination points, 418
- try-catch block, 428
- type checking, 360
- type system, 331
- variable declarations, 268
- variant record, 337
- while-loop, 417

Ada95, 608

- Access-type Lifetime Rule, 470
- “address-of” operator, 299
- concurrency and monitors, 611–615
- data types, 299
- function parameters and types, 345
- object-oriented mechanisms, 193
- procedures creating another procedure, 471–472
- protected objects, 612

Ada83 and rendezvous, 616

ad hoc polymorphism, 194

ADT (abstract data type), 10, 492

- algebraic specification, 494–498
- complex data type, 494
- implementations, 497
- mathematics, 532–536
- mechanisms, 143, 498–500, 524–532
- model for specification, 532
- modules, 500–502
- object-oriented programming, 493
- semaphores, 605
- signatures, 494, 533
- sorts, 532–533
- syntactic specification, 494

- algebraic notation, 6
- algebraic specification, 493
 - constants, 496
 - semantics, 535
 - sort-signature terminology, 533
- algebras, 558
- Algol60, 7, 142, 181
 - blocks, 262, 308, 402
 - goals, 26
 - pass by name, 410, 455
 - stack-based environment, 308
 - structural equivalence, 343
 - structured control, 402
 - syntax, 204
- Algol68
 - blocks, 402
 - bracketing keywords, 413
 - orthogonality, 32–33
 - structural equivalence, 343
- ALGOL (ALGOrithmic Language), 6–7, 26, 262
- Algol-like language, 309, 332
- ALGOrithmic Language. *See* ALGOL (ALGOrithmic Language)
- algorithmic logic and Horn clauses, 129–130
- algorithms and logic programming systems, 126
- aliases, 454
 - for argument, 452
 - assignment by sharing, 306
 - data types, 355
 - pointer variables, 304–306
 - semantics, 303–306
 - side effects, 305
- allocation, 197, 289–297
- alpha-conversion, 93
- ambiguity, 216–220
- ambiguous call, 283
- ambiguous grammars, 217–218
- American National Standards Institute. *See* ANSI (American National Standards Institute)
- Amoeba, 38
- and-parallelism, 622, 624–625
- and special form, 77
- Annotated Reference Manual*. *See* ARM (*Annotated Reference Manual*) (Ellis and Stroustrup)
- anonymous functions, 74, 83
- anonymous pointer type, 366
- anonymous types, 330, 354
- anonymous union, 339
- ANSI (American National Standards Institute), 16
- API (application programming interface), 10
- APL and dynamic scoping, 277
- appending lists, 120
- append operation, 59, 60, 119
- application frameworks, 143–144
- application of expressions, 90–91
- applicative order evaluation, 53, 404, 409
- approximation, 573
- arguments, 13, 403
 - procedures, 445
 - thunks, 456
- arithmetic and Prolog, 117–118

arithmetic operators in ML (MetaLanguage), 66
ARM (Annotated Reference Manual) (Ellis and Stroustrup), 37
 Armstrong, Joe, 626
 array-based queue class, 160
 array constructor, 330
 array data structure, 7
 arrays, 9, 330, 340–345
 component type, 340
 functional languages, 344
 index type, 340
 length, 341
 multidimensional, 343–344
 printing contents, 149–150
 unconstrained, 342
 array types, 340–345, 384
 assemblers, 5, 18
 assembly language, 4–6
 assignment
 denotational semantics, 561–563
 Java, 300
 operational semantics, 551–553
 programming languages, 299
 return value, 32
 assignment by cloning, 299
 assignment by sharing, 299–300, 306
 assignment compatibility, 361
 assignment statements, 298
 axiomatic semantics, 569–570
 associativity, 216–220
 asynchronous exceptions, 424
 atomically, 604
 atoms, 50, 51
 AT&T Bell Labs, 181
 attribute functions, 367
 attributes, 257–260
 automatic allocation, 294, 296
 automatic conversions, 284
 automatic deduction systems, 104
 automatic garbage collection, 473
 automatic memory management, 473–475
 automatic theorem provers, 104
 axiomatic semantics, 17, 104, 543, 565–571
 assignment statements, 569–570
 axiomatic specification, 567
 distributivity of conjunction or disjunction, 568
 if statements, 570
 law of monotonicity, 568
 law of the excluded miracle, 568
 predicate transformer, 567
 proofs of program correctness, 571–574
 sample language, 569–571
 statement-lists, 569
 weakest assertion or precondition, 567
 while statements, 571
 wp predicate transformer, 568–569
 axiomatic specification, 567
 axioms, 105, 107, 494–495
 equality, 495, 534
 error axioms, 496
 quotient algebra, 533
 axiom systems, 534–535

B

backing store, 172–173
 backtracking in Curry, 132–133
 backtracks, 120–123

Backus, John, 6, 204
 Backus-Naur forms. *See* BNFs (Backus-Naur forms)
 base classes, 181
 BASIC and goto statements, 420
 basic types, 350
 BCPL and abstractions, 35
 begin special form, 57
 behaviors, 145
 beta-abstraction, 92
 beta-conversion, 92
 beta-reductions, 92, 94
 binary arithmetic operators, 66
 binary messages, 147
 binary operator, 403
 binary search trees, 60, 73, 347
 binding list, 54
 bindings, 257–260
 scope, 263–265
 binding times, 258–259
 Bison, 224, 233
 blocked, 583
 blocks, 148–149, 260–269
 activated, 448
 allocating and deallocating space for, 291–292
 block-structured languages, 447–448
 functions, 444
 linked lists, 473–474
 procedures, 444
 reference counting, 475–476
 scope rules, 265–266
 stack frame, 448
 symbol table, 269
 block structure, 262
 block-structured languages
 activations of procedure blocks, 462
 blocks, 447–448
 garbage collection, 308
 global variables, 289
 lexical scope, 292
 local variables, 289
 scope, 62
 stack-based runtime environment, 469
 block variables, 148
 BNF grammar function-definition, 221
 BNF-like form grammars, 233
 BNF notation, 209, 221
 BNF rules
 abstract syntax, 216
 recursive set equations, 348
 semantics, 543
 BNFs (Backus-Naur forms), 204, 208–213, 221, 257
 body, 109, 445
 Boole, George, 104
 Boolean expressions, 77, 406–407
 bottom-up algorithm, 233
 bottom-up parsers, 224, 233
 bounded buffer class, 605–606
 bounded buffer problem, 588, 613–615
 bound identifier, 570
 bound occurrence, 91–92
 bound variables, 63
 box-and-circle diagram, 297, 304–305
 box and pointer notation, 58–59
 bracketing keywords, 413
 branch instructions, 11
 Brinch-Hansen, Per, 608
 Brooks, Fred, 27

built-in arithmetic functions, 69
 built-in collections in imperative languages, 157
 built-in operators, overloading, 287
 Burstall, Rod, 65
 busy-wait, 608
 byte code, 18

C

C, 263
 address of operator (&), 298, 307
 ADT (abstract data types), 502–506
 anonymous pointer type, 366
 arrays, 340–341, 347, 452
 attributes, 257
 basic types, 350
 blocks, 402
 BNFs (Backus-Naur forms), 204, 221
 Boolean operators, 407
 Cartesian products, 335
 casts, 365–366
 compilers and static type checking, 359
 compound statements, 261
 constants, 302
 dangling references, 306–307
 data allocation strategy, 348
 data types, 277, 328
 declaration before use rule, 264
 declarations, 261
 definitions, 257, 260
 derived types, 350
 disambiguating rule, 412
 do-while statement, 417
 dynamic allocation, 289
 end of line token, 206
 enumerated types, 332–333
 explicit conversions, 365
 expressions, 52, 53, 444
 flexibility, 36
 floating-point division, 330
 floating types, 350
 for-loop, 418
 functions, 31, 163, 293, 344–345, 446
 function variable, 303
 garbage, 307–308
 global procedures, 472
 global variables, 268
 if statements, 16–17
 implicit conversions, 366
 implicitly or explicitly binding, 260
 implicit types, 361
 incomplete type, 261
 index set, 340
 instance variables, 195–196
 integer calculator based on grammar, 227–230
 integer division, 330
 integral types, 350
 load-time constants, 302
 member declarations, 262–263
 memory leaks, 34
 multidimensional arrays, 343–344
 naming types, 330
 numeric types, 350
 overlapping types, 362
 parametric polymorphism, 506
 parsing, 236
 pass by reference, 453
 pass by value, 452

- pointers, 33, 346, 348
- portability, 36
- procedures, 31, 444
- prototypes, 445, 502–503
- recorded structures, 195
- recursion, 348
- recursive declarations, 268
- recursive definition, 278–288
- reserved words, 236
- scanners, 206, 207–208
- scope, 264
- short-circuit Boolean operators, 407
- shorthand notation, 10
- stack-based runtime environment, 33
- statements, 444
- static allocation, 289
- storage, 473
- storage class, 296
- structural equivalence, 343
- switch statement, 414–415
- symbol table, 278
- syntax diagrams, 223
- termination points, 418
- type compatibility, 361
- type conversion, 364–365
- type equivalence, 357–358
- type nomenclature, 350
- undiscriminated unions, 336
- user-defined functions, 404–405
- variables, 32, 163, 299
- while-loop, 417
- C++, 181–191
 - ambiguous call, 283
 - anonymous union, 339
 - arrays, 340, 347
 - automatic conversions, 284
 - base classes, 181
 - basic elements, 181–184
 - Boolean operators, 407
 - casts, 365–366
 - Cfront compiler, 36–37
 - classes, 181–184, 192–193, 336
 - class mechanism, 193, 506
 - commercial implementation, 37
 - compatibility, 33, 36
 - compilation, 502–506
 - complex addition, 194
 - constant declaration, 302
 - constructors, 182, 378, 528
 - Cpre preprocessor, 37
 - data members, 181–184
 - data types, 283–284
 - declarations, 445
 - definitions, 257, 260, 445
 - dependencies, 529
 - derived classes, 181
 - destructors, 182
 - dynamic allocation, 289, 293
 - dynamic binding, 186–191
 - exception handlers, 427–429
 - exceptions, 426
 - explicit conversions, 365
 - explicit parametric polymorphism, 378–380
 - expressions, 444
 - extensibility, 37
 - floating-point operations, 327
 - for-loop, 418
 - functions, 31, 446
 - generic collection, 184–186
 - global variables, 268
 - goals, 27, 36
 - implicit conversions, 365–366
 - incomplete type, 261
 - index set, 340
 - inheritance, 191, 194
 - inline functions, 191
 - instance variables, 181–184
 - integer type, 334
 - keywords, 332
 - member functions, 181–184, 191
 - memory leaks, 34
 - methods, 181–184
 - Microsoft Foundation Classes, 144
 - multidimensional arrays, 343–344
 - multiple inheritance, 190–191
 - name clashes, 506
 - namespaces, 193, 263, 279, 506–509
 - object-oriented constructs, 287
 - object-oriented extension of C, 35–38
 - object-oriented features, 191
 - objects, 181, 182, 197
 - operator overloading, 167, 183, 285–287, 364
 - operators, 31
 - overloaded functions, 283
 - overloading, 194, 282
 - parametric polymorphism, 195
 - pass by reference, 452–453
 - pass by value, 452
 - pointers, 346
 - postfix & operator, 346
 - private members, 181
 - protected members, 181
 - public members, 181
 - pure virtual declaration, 187–188
 - recursive declarations, 268
 - reference arguments, 454
 - references, 346
 - reserved words, 236
 - retrospective, 37–38
 - runtime environment, 197
 - scope resolution operator (::), 182, 266, 267
 - semantics, 37
 - shared inheritance, 191
 - short-circuit Boolean operators, 407
 - standard library modules, 427
 - statements, 444
 - static allocation, 289
 - statically typed, 39
 - static binding, 186–191
 - STL (Standard Template Library), 37
 - structs, 279
 - structured if and for statements, 11
 - template classes, 184–186
 - templates, 194, 378–379, 506
 - type checking, 36, 359
 - type constructors, 192
 - undiscriminated unions, 336
 - variables, 32, 183
 - virtual functions, 186–191
 - while-loop, 417
 - zero-overhead rule, 36
- called, 13
- callee, 446
- caller, 446
- calling context, 283
- calling environment, 449
- call unwinding, 431
- Cartesian products, 335–336
- case analysis and Curry, 131
- case expressions, 68, 407–408
- case-statements, 414–416
- casts, 365–366
- Centrum voor Wiskunde en Informatica. *See* CWI (Centrum voor Wiskunde en Informatica)
- Cfront compiler, 36–37
- character arithmetic, 334
- children processes, 591
- Chomsky, Noam, 204
- Church, Alonzo, 8, 90
- Church-Rosser theorem, 94
- circlearea function, 55, 63
- classes, 10
 - abstract, 170
 - base classes, 181
 - collection hierarchy, 158
 - concrete, 170
 - declarations, 263, 269
 - instance methods, 155
 - instance variables, 155
 - interfaces, 169–172
 - members, 181
 - methods, 152–154
 - versus* modules, 192–193
 - namespaces as, 192–193
 - naming, 155
 - object-oriented languages, 269, 336
 - packages, 193
 - type checking, 192
 - type constructors, 192
 - versus* types, 192
- class loader, 509
- class messages, 146
- class method, 153–154, 164
- class variables, 152
- clauses, 120
- client, 628–629
- cloning objects, 306
- CLOS (Common Lisp Object System), 167, 194
- closed form, 450
- closed-word assumption, 127
- closure, 451, 459
- CLU, 423, 520–521
- coalescing, 474
- COBOL, 26–28, 343
- code
 - efficient, 28–30
 - implementation of class, 192
 - writability, 327
- code bloat, 376
- code inlining, 404
- code reuse, 150, 165
- coercions, 365
- collection classes and Smalltalk, 150
- collections, 157
 - built-in types, 159
 - functional languages, 157
 - hierarchy, 157–162
 - reference types, 169
 - type parameters, 168–169
- column-major form, 344
- COM (Common Object Model), 616

- commits, 625
- Common Lisp, 50
 - lexical scoping, 277
 - macros, 34
 - statically scoped, 62–63
- Common Lisp Object System. *See* CLOS (Common Lisp Object System)
- Common Object Model. *See* COM (Common Object Model)
- communicates, 449
- Communicating Sequential Processes language
 - framework. *See* CSP (Communicating Sequential Processes) language framework
- communication problem, 585–586
- comparison operators
 - abstract classes, 151
 - ordinal types, 334
- compatible types, 361
- compiled languages, 289
- compiler-compilers, 224, 232
- compilers, 7, 18–19, 547
 - basis for, 36
 - dynamic type checking, 359
 - options, 589
- compile-time constants, 301–302
- complete, 534, 543
- complex data types, 330, 494–495
- complex numbers, 494
- components, 10
- component selector operation, 335
- component types, 340
- composite types, 351
- composition, 49
- compound statements, 261
- computational paradigms, 15
- computers
 - logic, 104
 - multicore architectures, 7–8
- concatenation and regular expressions, 206
- concatenation (++) operator, 82
- concrete classes, 150–151, 158, 170
- concrete syntax, 216
- concurrency, 611–615, 626
- concurrent programming, 583
- cond form, 54
- conditional expressions, 54
- conditional statements, 411–416
- conditions, 132–133, 618
- condition variables, 609
- connectives, 105, 110
- consistent, 534, 543
- cons operator, 60, 82
- constant procedure value, 447
- constants, 105, 107, 205, 300–303
 - algebraic specification, 496
 - compile-time, 301–302
 - dynamic, 301–302
 - load-time, 301
 - manifest, 301
 - names, 301
 - static, 301–302
 - symbolic, 301
 - values are functions, 302–303
 - value semantics, 301
- constrained parameterization, 529
- constrained parametric polymorphism, 372
- constructed lists, 120

- constructor chaining, 165
- constructors, 497
 - arrays, 330
 - C++, 182, 378, 528
 - Java, 165, 182, 197, 378
 - types, 346–349
 - variable initialization, 183
- consumer threads, 601
- context, 211
- context-free grammar, 204, 208–213, 235–236, 543
- continue call, 609
- continue operations, 609–610
- continue statement, 417
- control, 425
 - basic abstractions, 10
 - exception handlers, 430–432
 - operational semantics, 553–554
 - reduction rules, 553–554
 - structured abstractions, 11–14
 - unit abstractions, 14
- control abstraction, 8
- control facilities, 616
- control information, 129–131
- control link, 462
- control mechanisms, 423
- control problem, 108
- control statements
 - denotational semantics, 563–564
 - Smalltalk, 148–149
- control structures, 122–126, 326
- conversion function, 69
- copy-in, 454
- copy-out, 454
- copy-restore, 454
- correctness rules and semantics, 360
- Cpre preprocessor, 36–37
- C programs and symbol table structure, 269–270
- critical region, 604
- CSP (Communicating Sequential Processes)
 - language framework, 616
- curried, 75–76
- Curry, 131–134
- Curry, Haskell B., 75, 80, 131
- cut operator, 124–126
- CWI (Centrum voor Wiskunde en Informatica), 38
- C with Classes, 36

D

- daemons, 596
- Dahl, Ole-Johan, 142
- dangling-else problem, 412–413
- dangling references, 303–307, 470
- data, 8–10, 326
 - extension, 143
 - finitude, 326
- data abstraction, 8
- data constructors, 73, 338
- data encapsulation, 165
- data entities, 326
- data keyword, 83
- data members, 181–184
- data structures, 9, 492
 - data, 326
 - naming, 9
 - Scheme, 57–60
- data type queue, parameterized, 496

- data types, 9, 326, 328–331
 - algebraic specification, 493
 - Algol-like language, 332
 - aliases, 355
 - anonymous, 330
 - characteristics, 492
 - versus* classes, 192
 - comparing, 70, 331
 - compatibility, 361
 - complex, 330
 - constructors, 330, 335–349
 - conversions, 283–284, 364–367
 - defining, 10
 - dynamically allocating space, 349
 - dynamic scoping, 277
 - encapsulation, 493
 - environment, 349
 - explicit, 327
 - implicit, 361–362
 - information hiding, 493
 - lists, 67
 - mathematical concept, 493
 - mechanism for constructing, 493
 - modifiability, 493
 - modules, 524, 529–531
 - names, 330
 - overlapping, 362–363
 - recursive, 73
 - reusability, 493
 - security, 493
 - as set, 329
 - shared operations, 363–364
 - simple types, 332–334
 - static binding, 277
 - synonyms, 73
 - union, 336–339
 - user-defined, 72, 73, 330–331
 - values, 55, 65
- deadlock, 585–586
- declaration equivalence, 358
- declarations, 9, 445
 - accessibility, 266–268
 - Ada packages, 263
 - attributes, 257–258
 - blocks, 261–262
 - classes, 263
 - constructing environment, 289
 - Java packages, 263
 - keyword modifiers, 268
 - language constructs, 261
 - local, 261
 - member declarations, 262–263
 - modules, 263
 - namespaces, 263
 - nested blocks, 266
 - nonlocal, 261
 - order within class, 269
 - recursive, 268
 - scope, 263–270
 - self-referential, 268
 - tasks, 263
 - type expressions, 354
 - type information, 386–388
- declarative programming, 126
- declarative style, 131
- dedicated processors, 7
- deductive databases, 108

- definite clause grammars, 110
 - definition, 445
 - definitional interpreters, 547
 - definitional translator, 19
 - delayed evaluation, 54, 77–81, 93, 408, 455–458
 - Delay operation, 604, 609
 - delay special form, 78
 - De Morgan, Augustus, 104
 - denotational semantics, 17, 542, 556–565
 - assignment, 561–563
 - control statements, 563–564
 - environments, 561–563
 - implementing, 564–565
 - integer arithmetic expressions, 560–561
 - semantic domains, 556, 558
 - semantic functions, 559
 - syntactic domains, 556, 558
 - dereferenced, 346
 - dereferenced variables, 293, 361
 - derivations, 209–210, 215–218
 - derived classes, 181
 - derived types, 350
 - design criteria, 27–33
 - design tool, 327
 - destructors, 182
 - Digitalk, 145
 - Dijkstra, E. W., 411, 421, 604, 616
 - disambiguating rule, 218, 412
 - discrete types, 351
 - discriminated unions, 336
 - discriminator, 336
 - distributed applications, 631
 - distributed-memory system, 585–586
 - distributed system, 582
 - distributivity of conjunction or disjunction, 568
 - documentation, 16
 - domains, 47
 - domain theory, 558
 - dot notation, 167
 - double-dispatch problem, 194
 - double-ended queue, 143
 - do-while statement, 417
 - Dynabook Project, 142, 144
 - dynamic allocation, 293–294, 296
 - dynamically computed procedures, 469–472
 - dynamically typed languages, 331
 - dynamic binding, 155, 176, 258
 - member functions, 186–191
 - methods, 196–197
 - dynamic constants, 301–302
 - dynamic environment, 289, 449
 - dynamic link, 462
 - dynamic memory management, 473–476
 - dynamic scoping, 273, 467
 - free variables and formal parameters, 63
 - symbol table, 274–277
 - dynamic type checking, 55–56, 359
 - dynamic typing, 145, 626
- E**
- EBNF rules
 - if-statement, 411
 - recursive-descent parser, 227
 - syntax diagrams, 243–244
 - top-level rule, 244
 - EBNFs (extended Backus-Naur forms), 204, 220–224
 - grammar rules, 232
 - left recursion removal, 226
 - Scheme, 51
 - TinyAda, 237–239
 - Edinburgh LCF (Logic for Computable Functions) system, 65
 - Edinburgh Prolog, 115
 - efficiency of execution, 27
 - Eiffel, 532
 - else clause, 54
 - empty environments, 545
 - empty list, 67, 70
 - encapsulation, 144, 493, 501
 - “The End of History and the Last Programming Language” (Gabriel), 20
 - entries
 - entry barrier, 612–613
 - tasks, 616–617
 - entry/accept mechanism, 617
 - entry barrier, 612–613
 - enumerated types, 332
 - enumerations, 332, 338
 - enumeration type, 384, 387
 - environments, 260, 289–297, 447
 - data types, 349
 - denotational semantics, 561–563
 - exception handling, 477–478
 - fully dynamic, 469–472
 - fully static, 459–461
 - heap, 294
 - objects, 292
 - operational semantics, 551–553
 - procedures, 459–472
 - reduction rules, 551–553
 - sample language, 544–545
 - stack, 294
 - stack-based runtime, 462–469
 - ep (environment pointer), 462–464
 - equality, 118–119, 495
 - equality types, 70
 - equations, 132, 494–497
 - equivalence relation, 534
 - Ericsson, 626
 - Erlang parallelism with message passing, 626–631
 - erroneous programs, 458
 - error axioms, 496
 - eta-conversion, 93
 - eta-reduction, 93
 - Euclid
 - limiting aliasing, 305
 - modules, 519–520
 - simplicity and abstraction, 28
 - Euclidian algorithm, 109, 118, 130
 - evaluation rules, 51–53, 404
 - exception handlers, 423, 425, 427–429
 - control, 430–432
 - pointers, 477
 - scope, 429
 - try-catch blocks, 427–428
 - exception handling, 423–432
 - control, 425
 - environments, 477–478
 - exception handlers, 423, 425, 427–429
 - exceptions, 423, 425–427
 - functional mechanisms, 423
 - object-oriented mechanisms, 423
 - resumption model, 431
 - runtime overhead, 431–432
 - termination model, 431
 - exceptions, 423–427
 - propagating, 430
 - raising, 430, 477
 - executable specification, 555
 - executing, 583
 - execution efficiency, 327
 - execution model, 459
 - existential quantifier, 106
 - existential type, 532
 - explicit, 423
 - conversions, 365
 - exception testing, 425
 - pointer references, 295
 - polymorphism, 376–382
 - type information, 367
 - explicitly binding, 260
 - explicitly constrained parametric polymorphism, 382
 - explicitly parameterized polymorphic data types, 377–378
 - explicitly parameterized polymorphic functions, 381–382
 - explicit parametric polymorphism, 376–377
 - explicit parametric polymorphism, 372, 378–380
 - expression languages, 402
 - expressions, 402
 - control statements, 407–408
 - forcing left-to-right execution, 57
 - fully parenthesized, 404
 - functional languages, 402
 - identifiers, 403
 - lazy evaluation, 80–81
 - literals, 403
 - nonfunctional languages, 402
 - normal order of evaluation, 409–410
 - operands, 388–389, 403
 - operators, 403
 - polymorphic, 371
 - programming languages, 402
 - rules for evaluating, 404
 - semantics, 402
 - sequence operator, 406
 - side effects, 402, 405–406
 - substitution rule, 409
 - expression sequence, 71
 - expression statement, 444
 - expressiveness, 29
 - extended BNF. *See* EBNF (extended BNF)
 - extensibility, 38–39
 - extensible languages, 34–35
- F**
- fact function, 65–66, 68
 - factorial function, 56–57
 - facts, 109
 - fair scheduling, 608
 - falls through, 415
 - FGHC (flat guarded Horn clauses), 625
 - fields, 263
 - Fifth Generation Project, 104
 - filters, 80–81
 - final algebra, 535
 - final semantics, 535
 - finger typing, 39
 - finitude, 326
 - first-order predicate calculus, 105–109

- fixed format language, 205
 - fixed point, 94–95
 - flat guarded Horn clauses. *See* FGHC (flat guarded Horn clauses)
 - flatten, 80–81
 - flexibility, 36
 - floating-point division, 330
 - floating types, 350
 - Follow sets, 232
 - force special form, 78
 - for-each loop, 420
 - fork-join model, 592
 - fork-join parallelism, 597
 - for-loop, 173, 418–419
 - formal definition, 257
 - formal parameters, 445, 450
 - formal semantics, 543
 - axiomatic semantics, 565–571
 - denotational semantics, 556–565
 - methods, 546
 - operational semantics, 547–556
 - sample small language, 543–547
 - FORmula TRANslation language. *See* FORTRAN (FORmula TRANslation language)
 - FORTH, 404
 - Fortran77, 420, 444
 - FORTRAN (FORmula TRANslation language), 142
 - activation, 462
 - algebraic notation, 6
 - calling procedures, 446
 - fully static environment, 459–461
 - functions, 446
 - multidimensional arrays, 343–344
 - overloading, 282
 - pass by reference, 452, 454
 - procedures, 444, 446
 - static environment, 289
 - static storage allocation only, 29
 - structural equivalence, 343
 - subroutines, 446
 - free algebra of terms, 533–534
 - free-format language, 205
 - free identifier, 570
 - free lists, 474
 - free occurrence, 91–92
 - free variables, 63, 106
 - fully curried, 76
 - fully dynamic environment, 472
 - fully parenthesized, 404
 - fully static environments, 459–461
 - functional languages, 46
 - arrays, 344
 - automatic garbage collection, 473
 - collections, 157
 - constants, 301
 - delayed evaluation, 77–81, 455
 - deterministic computation, 132
 - dynamic binding, 258
 - exceptions, 425
 - expressions, 402
 - functions, 444–445, 447
 - garbage collection, 308
 - higher-order functions, 62
 - if-else constructs, 125
 - imperative constructs, 344
 - implicit pointers, 346
 - integers, 326
 - lambda form, 177
 - lists, 344
 - map, 159
 - programs, 131
 - safe union mechanism, 338
 - unification, 370
 - functional lazy languages, 81
 - functional paradigm, 15
 - functional programming, 46
 - assignment, 48–49
 - delayed evaluation, 54
 - functional perspective, 131
 - generator-filter programming, 80
 - lambda calculus, 90–95
 - local state, 49
 - loops, 48
 - mathematics, 90–95
 - programs as functions, 47–50
 - recursion, 48–49
 - static typing, 65–76
 - variables, 48
 - functional programs, 49
 - function application, 47–48, 92
 - function calls, 287
 - function declaration, 65, 447
 - function definitions, 47–48, 68
 - function literal, 303
 - function overloading, 284–285
 - functions, 13–15, 105, 340–345, 403, 444, 612
 - activations, 404
 - anonymous, 74
 - applicative order evaluation, 77
 - arguments, 75
 - behavior, 85
 - body, 65
 - bound variables, 63
 - calling, 66
 - closed form, 450
 - closure, 451
 - composition, 49
 - curried, 75–76
 - declarations, 356
 - defining, 47, 87
 - dependent variable, 47
 - domain, 47
 - free variables, 63
 - functional languages, 444–445, 447
 - function parameters, 61
 - higher-order, 13–14, 49, 61–62, 74–76
 - implementing methods as, 196
 - independent variable, 47
 - lists, 58
 - naming, 65
 - nonstrict, 77
 - overloading names, 282
 - parameters, 61, 65, 450
 - pass by name evaluation, 77–79
 - polymorphic, 372–375
 - prefix form, 403
 - primitive, 55
 - programs as, 47–50
 - range, 47
 - recursive, 55, 74, 347
 - referential transparency, 49
 - return types, 31
 - return values, 74–75, 446
 - signatures, 86
 - tail recursive, 56
 - type recognition, 56
 - type specifier, 84
 - uncurried version, 76
 - von Neumann concept, 13
 - function types, 340–345
 - functors, 515–518
 - future, 624
- ## G
- Gabriel, Richard, 20
 - garbage, 303–309
 - garbage collection, 62, 145, 308–309, 472–473, 476
 - gcd function, 13, 49, 82, 303
 - generality, 30–31
 - general-purpose programming languages, 26
 - generational garbage collection, 476
 - generator-filter programming, 80
 - generators, 80–81
 - generic collections, 169, 184–186
 - generic interfaces and classes, 194
 - generic packages, 194, 512–514
 - generic type variables, 375
 - generic unit, 380
 - GI/GI principle (garbage in, garbage out), 77
 - global declaration, 266
 - global environment, 449
 - global variables, 268
 - goal-oriented activity, 567
 - goals, 26, 36, 108
 - Gödel, Kurt, 104
 - Goldberg, Adele, 144
 - Gosling, James, 162
 - goto controversy, 420–423
 - Graham, Paul, 20
 - grammar rules, 210, 220
 - alternatives, 230–231
 - for declaration in EBNF, 232
 - left factoring, 227
 - recursive nature, 212
 - set equation, 213
 - special notation for, 221
 - syntax diagrams, 222–224
 - grammars, 16, 208–213
 - abstract syntax trees, 213–216
 - ambiguity, 216–220
 - associativity, 216–220
 - BNF-like form, 233
 - context-free, 208–213
 - context-sensitive, 211
 - converting into parser, 230
 - disambiguating rule, 218
 - EBNFs, 220–224
 - integer calculator based on, 227–230
 - left-associative, 218–219
 - left-recursive rule, 219, 226
 - metasymbols, 209
 - parse trees, 213–216
 - parsing techniques and tools, 224–235
 - precedence, 216–220
 - right associative, 218–219
 - right-recursive rules, 219, 221
 - start symbol, 209
 - syntax diagrams, 220–224
 - tokens, 224, 235–236
 - granularity, 592

green threads, 595
 guarded do command, 417, 616
 guarded Horn clauses, 624–625
 guarded if statement, 411, 616
 guards, 410–416, 625

H

Hacker and Painters (Graham), 20
 handler stack maintenance, 478
 handling exceptions, 477
 hardware interrupt, 583
 Haskell, 28
 binding, 258
 class definition, 87
 constants, 301
 default function definitions, 87
 elements, 82–83
 fully curried lazy language with overloading,
 81–89
 functions, 31
 heap allocation and deallocation, 295
 Hindley-Milner type checking, 65
 infinite lists, 84–85
 instance definition, 86–87
 lambda calculus, 90
 lambda forms, 83
 lazy evaluation, 80, 84–85, 410
 lists, 82, 84
 modules, 10, 263
 monads, 82
 overloaded functions, 86–90
 overloading, 194, 282
 parameters, 133
 pattern matching, 82
 patterns, 84
 pointer manipulation, 295
 predefined functions, 82
 predefined types, 87
 prefix functions, 83
 reverse function, 82
 security, 33
 signatures, 86
 static type checking, 360
 strongly typed, 331
 syntax, 82
 tuples, 83
 type classes, 86–90
 type parameters, 168
 type synonyms, 83
 type variables, 83
 unification, 120, 370
 user-defined data type, 89
 user-defined polymorphic types, 83
 variables, 301
 Haskell98, 81
 hd (head) operator, 67–68
 head, 109
 header file, 502–503
 heap, 294–296, 473
 heavyweight processes, 583–584
 higher-order functions, 13–14, 49, 61–62, 83,
 176–181
 high-level programming languages, 7
 code, 27
 identifiers, 312
 Hindley-Milner type checking,
 367–370

explicitly parameterized polymorphic data
 types, 377–378
 let-bound polymorphism, 375
 occur-check, 375
 simplifying type information, 371
 type expression, 375
 Hoare, C. A. R., 27, 414, 608
 HOPE language, 65
 Horn, Alfred, 109
 Horn clauses, 109–111
 algorithmic logic, 129–130
 Euclid's algorithm, 118
 goal or lists of goals as, 112–113
 not expressing all of logic, 128–129

I

identifier role analysis, 312–315
 identifiers, 205, 257, 382–383, 403
 attributes, 314
 bound, 570
 free, 570
 high-level languages, 312
 roles, 312–313
 type-compatible, 383
 IEEE (Institute of Electrical and Electronics
 Engineers), 326
 IEEE 754 standard, 334
 if-expressions, 54, 407–408
 if form, 54, 77
 if-statements, 410–414
 axiomatic semantics, 570
 dangling-else problem, 412–413
 disambiguating rule, 412
 imperative constructs, 344
 imperative languages, 15, 46
 atoms as identifiers, 50
 built-in collections, 157
 if-else constructs, 125
 loops, 56
 pointers, 348
 variables, 48
 imperative programs and loops, 48–49
 implicit, 361, 423
 implicit conversions, 365, 366
 implicit dereferencing, 300
 implicitly binding, 260
 implicitly constrained parametric polymorphism, 380
 implicit parametric polymorphism, 372
 implicit types, 361–362
 incomplete type, 261, 505
 independence, 143–144, 534–535
 independent, 543
 independent variables, 47–48
 indexed component references, 389–390
 index type, 340
 inference rules, 107–108
 infinite lists, 84–85
 infinite loops, 122, 124, 127, 129, 226
 infix functions, 500
 infix operators, 66, 403–404
 information hiding, 9–10, 144, 493, 501
 Ingalls, Daniel, 144
 inheritance, 150, 153
 object-oriented languages, 196–197,
 339–340, 362
 repeated, 191
 shared, 191

subset types, 339
 vs. polymorphism, 194–195
 initial algebra, 534
 initial semantics, 534
 inline functions, 191
 inline member functions, 182
 inlining, 404
 inputLine function, 70–71
 inspectors, 497
 instance definition, 86–87
 instance messages, 146, 151
 instance methods, 154–155
 instance variables, 153
 Ada, 195–196
 C, 195–196
 C++, 181–184
 classes, 155
 Java, 165
 Smalltalk, 152, 155
 instantiated variables, 114
 Institute of Electrical and Electronics Engineers.
 See IEEE (Institute of Electrical and
 Electronics Engineers)
 integer arithmetic, 334
 integer arithmetic expressions
 denotational semantics, 560–561
 reduction rules, 548–551
 integer division, 330, 334
 integers, 326, 329
 integral types, 350
 interfaces, 146, 193, 445
 class implementing, 169–172
 Java, 167–168, 174–175, 177, 351
 standards, 10
 static typing, 327
 International Organization for Standardization.
 See ISO (International Organization for
 Standardization)
 interoperability, 10
 interpreted languages
 dynamic scoping, 277
 exceptions, 423
 interpreters, 18–19
 attribute binding, 289
 dynamic scoping, 277
 dynamic type checking, 359
 symbol table and environment, 289
 intlist procedure, 80
 invocation, 12
 ISO (International Organization for
 Standardization), 16
 iteration, 11–12
 iterators, 12
 backing store, 172–173
 for-each loop, 420
 loops, 420
 polymorphic, 158
 strings, 158

J

Java, 162–181, 608
 accessor methods, 165
 arrays, 341–342, 351
 assignment, 300, 361
 assignment by sharing, 306
 automatic conversions, 284
 automatic garbage collection, 34, 309

Java (*cont.*)

- backing store, 172–173
- basic elements, 163–167
- BNFs (Backus-Naur forms), 204
- Boolean operators, 407
- bounded buffer example, 601–604
- byte code, 18
- casts, 365
- classes, 163–167, 192, 263, 279, 351
- class loader, 509
- class method, 164
- cloning objects, 306
- code dependencies, 509
- collection classes, 380
- Collection interface, 168
- collections, 32
- complex addition, 194
- constants, 302
- constructors, 165, 182, 197, 378
- conversion functions, 367
- daemons, 596
- dangling references, 307
- data encapsulation, 165
- deallocation routines, 197
- default constructor, 165
- dot notation, 167
- do-while statement, 417
- dynamic allocation, 289
- explicit checks, 334
- explicit conversions, 365
- explicit dereferencing, 295
- extensibility, 34
- fields, 263
- filter method, 176–181
- floating-point operations, 327, 330
- for-loop, 418
- generic collections, 169, 184
- generic interfaces and classes, 194
- green threads, 595
- heap allocation, 295
- higher-order functions, 176–181
- identifiers, 332
- implicit conversions, 366
- implicit pointers, 346
- index set, 340
- inheritance, 167–175, 194
- instance variables, 165
- integers, 329–330, 334
- interfaces, 167–175, 177, 193, 351
- interpreter, 162
- iterators, 12, 172–174, 178–179
- Java collection framework, 167–175
- Java Virtual Machine (JVM), 162, 164
- libraries, 38, 163, 167
- linked lists, 170
- links, 172
- Lock and Condition interfaces, 610
- for loop, 173
- loops, 12
- map method, 176–181
- methods, 163–167, 263
- method synchronization, 598
- monitors, 609–610
- multidimensional arrays, 343
- name overloading, 288
- namespaces as classes, 193
- native threads, 595

- numbers package, 165
- object instantiation, 163
- objects, 163–167, 197, 452
- operators, 31
- output collection, 176
- overlapping types, 362
- overloaded operators, 167, 363–364
- overloading, 194, 282
- packages, 10, 163, 167, 193, 279, 506–509
- pass by value, 452
- pointers, 33
- polymorphism, 167–175
- portability, 162–163
- primitive types, 32–33, 163, 350–351
- private inner class, 171
- processes, 14
- references, 163, 346
- reference semantics, 166
- reference types, 32–33, 166, 169, 350–351
- reserved words, 236
- runtime garbage collection, 163
- scalar data types, 163
- shared data, 597–599
- short-circuit Boolean operators, 407
- shorthand notation, 10
- stacks, 170
- static allocation, 289
- static binding, 175–176
- static constants, 302
- static type checking, 181
- static typing, 175
- structural equivalence, 343
- Swing Toolkit, 10, 144
- switch statement, 415
- temporary object, 167
- threads, 14, 595–601
- try/catch exception-handling, 178
- type compatibility, 361
- type constructors, 351
- type equivalence, 358
- type nomenclature, 350–351
- type parameters, 168
- value semantics, 166
- variables, 163, 166
- while-loop, 173, 417

- Java collection framework, 167–175
- java.concurrent.locks package, 610–611
- java.lang package, 14, 163
- Java packages, 263
- java.util package, 163, 167
- javax.swing package, 10, 163
- Jensen, J., 458
- Jensen's device, 458
- jobs, 583
- Johnson, Steve, 233
- JVM (Java Virtual Machine), 162, 164

K

- Kay, Alan, 144
- keyword messages, 146–148
- keywords, 52, 205, 332
- Kowalski's principle, 108

L

- lambda abstraction, 90, 93
- lambda calculus, 8, 15, 90–95
- lambda expressions, 92–94
- lambda forms, 54–55, 64, 83, 148, 177

- lambda function, 78
- language of the grammar, 211
- language reference manual, 257
- latent type checking, 55
- law of monotonicity, 568
- law of the excluded miracle, 568
- layout rule, 82
- lazy evaluation, 80–81, 84–85, 131, 410, 455
- LC-3 assembly language, 5, 11
- least fixed point, 348
- least-fixed-point semantics, 95
- least-fixed-point solution, 564
- left associative, 226
- left-associative, 218–219, 221, 228
- left factoring, 227
- leftmost derivation, 218
- left recursion, 226
- left-recursive BNF grammar, 233
- left-recursive rules, 219, 225–226
- legal programs, 331
- length, 341
- let-bound polymorphism, 375
- let form, 54–55, 64
- levels, 8, 262
- LEX/Flex, 235
- lexical address, 262
- lexical environment, 467
- lexical scope, 62–63, 264, 277, 467
- lexical scoping rule, 449
- lexical structures, 204–208
- lexics vs. syntax vs. semantics, 235–236
- lifetimes, 289–297
- lifted domains, 561
- lightweight processes, 584
- limited private type, 528
- linked, 18
- linked lists, 170–172, 473–474
- linked nodes queues, 160–162
- LISP, 8, 28
 - and-parallelism, 622, 624
 - automatic garbage collection, 34
 - delayed evaluation, 54
 - do loop, 34–35
 - dynamic scoping, 50, 62, 277
 - dynamic typing, 145
 - extensibility, 34–35
 - fully parenthesized, 404
 - functions, 444–445
 - garbage collection, 145, 308
 - lambda calculus, 90
 - list data structure, 50
 - macros, 34
 - metacircular interpreter, 50
 - or-parallelism, 623
 - packages, 10
 - parallelism, 624
 - reference semantics, 300
 - static type-checking, 33
 - uniform treatment of functions, 50
 - untyped languages, 331
 - variables, 33, 261
 - while loop, 34–35
- list comprehensions, 84
- lists, 72
 - appending, 59, 68, 120
 - binary search tree, 57
 - box and pointer notation, 58–59

- constructed, 120
 - data types, 67
 - empty, 67
 - flatten, 80–81
 - functional languages, 344
 - functions, 58
 - head, 58–59, 67–68, 119–120
 - infinite, 80, 84–85
 - lazy evaluation, 80
 - linked lists, 170–172
 - recursive types, 347
 - reversing elements, 59
 - same-fringe problem, 80–81
 - with structure, 60
 - tail, 58–59, 67–68, 119–120
 - tuple, 67
 - literal array, 148
 - literals, 205, 301, 362, 403
 - loaded, 18
 - loader, 5
 - load-time constants, 301
 - local declarations, 261
 - local scope, 296
 - local symbol table, 278–279
 - local variable, 464
 - locations, 257
 - locks, 604
 - logic, 104, 105–108
 - logical expressions, 406–407
 - logical inference rules, 548
 - logical statements, 104–106, 109
 - logical variables, 133–134, 626
 - logic languages, 20, 108
 - programs, 131
 - unification, 114, 371
 - logic paradigm, 15
 - logic program, 108
 - logic programming, 46
 - control information, 129–131
 - first-order predicate calculus, 105
 - functional perspective, 131
 - Horn clauses, 109–111, 128–129
 - logical variables, 133
 - order of clauses, 122
 - problems, 126–131
 - resolution, 111–115
 - specification activity, 126
 - unification, 111–115, 133
 - logic programming systems, 108
 - algorithmic control information, 130
 - algorithms, 126
 - breadth-first search, 122
 - closed-word assumption, 127
 - control problem, 108
 - depth-first search, 122
 - Horn clauses, 111, 112–113
 - order clauses, 115
 - logic programs, 105–108
 - nonmonotonic reasoning, 128
 - loop invariant, 573
 - loops, 417–420
 - behavior, 419–420
 - control and increment, 419
 - ensuring execution, 417
 - exiting, 420–423
 - functional programming, 48
 - guarded do, 417
 - imperative languages, 48–49, 56
 - iterators, 420
 - loop invariant, 573
 - restrictions, 419
 - sample language, 546
 - sentinel-based, 422–423
 - structured exits, 422–423
 - terminating, 417–418
 - unstructured exits, 421–422
 - I-value, 298, 361
- M**
- machine dependencies, 326
 - machine independence, 7
 - machine language, 3–5
 - macros, 34
 - Magnitude hierarchy, 150–157
 - mailboxes, 616, 630
 - maintenance of free space, 473–475
 - manifest constants, 301
 - map function, 14, 61, 75, 83
 - maps, 13–14
 - Erlang, 627
 - functional languages, 159
 - mark and sweep, 476
 - master, 619
 - mathematical concept, 493
 - mathematical function package, 500–501
 - mathematical logic, 105
 - mathematics
 - ADT (abstract data type), 532–536
 - functional programming, 90–95
 - independent variables, 48
 - lambda calculus, 90–95
 - parameters, 48
 - principle of extensionality, 535–536
 - variables, 48
 - McCarthy, John, 8, 50, 277
 - member declarations, 262–263
 - member functions, 181, 191, 336
 - dynamic binding, 186–191
 - inline, 182
 - members, 181
 - memory, 260
 - compacted, 474–475
 - deallocating, 308, 473
 - dynamic management, 473–476
 - free list, 474
 - free space maintenance, 473–475
 - heap, 473
 - manual management, 473
 - reclamation, 475–476
 - wasted, 308
 - message passing, 146, 588, 615–616
 - parallelism, 626–631
 - Message Passing Interface. *See* MPI (Message Passing Interface)
 - messages
 - abstract classes, 151
 - mutators, 146
 - receiver, 149
 - selector, 146
 - metacircular interpreter, 50
 - MetaLanguage. *See* ML (MetaLanguage)
 - metalinguistic power, 63–64
 - metasymbols, 209, 233
 - methods, 153–154, 336
 - directly accessing data, 196
 - dynamic binding, 196–197
 - formal semantics, 546
 - header and comment, 152
 - object-oriented languages, 195–196
 - synchronized and unsynchronized, 609
 - Microsoft Foundation Classes, 144
 - Milner, Robin, 65
 - MIMD (multiple-instruction, multiple-data) systems, 584–585
 - Miranda, 81
 - ML97, 65
 - ML (MetaLanguage), 28, 65
 - ad hoc stream constructions, 80
 - applicative order evaluation rule, 77
 - arithmetic functions, 69
 - arithmetic operators, 66
 - array module, 344
 - assignment compatibility, 361
 - binary arithmetic operators, 66
 - binary search trees, 73–74
 - binding, 258
 - Boolean operators, 407
 - calling functions, 66
 - case construct, 416
 - constant, 303
 - constraints, 69
 - conversion function, 69
 - currying, 75–76
 - data constructors, 73, 338
 - data structure, 72–74
 - data types, 72–74, 528
 - elements, 65–72
 - empty list, 67
 - enumerated types, 332
 - enumeration, 338
 - equality types, 70
 - evaluation rule, 66
 - exception handlers, 428
 - exception handling, 429
 - exceptions, 426, 427
 - explicit parametric polymorphism, 376–377
 - expression sequence, 71
 - fn reserved word, 74
 - function composition, 75
 - function declaration, 65, 447
 - function literal, 303
 - functions, 71, 345
 - functors, 515–518, 530–531
 - fun reserved word, 65
 - heap allocation and deallocation, 295
 - higher-order functions, 74–76
 - Hindley-Milner type checking, 65, 378
 - infix functions, 500
 - infix operators, 66
 - lambda calculus, 90
 - lists, 66–67, 72
 - modules, 10, 263, 515–519
 - operators, 66
 - overloading, 69
 - parametric polymorphism, 65, 499
 - pass by reference, 453
 - pointer manipulation, 295
 - polymorphic function, 69
 - safe union mechanism, 338
 - security, 33, 65
 - short-circuit Boolean operators, 407

- ML (MetaLanguage) (*cont.*)
 - signatures, 515–518, 518
 - special ADT mechanism, 498–500
 - special forms, 77
 - static type checking, 360
 - strings, 71
 - structural equivalence, 343
 - structures, 70, 515–518
 - syntactic sugar, 74
 - tl (tail) operator, 67–68
 - tuples, 72, 76, 336
 - type check expressions, 69
 - type equivalence, 358
 - types, 70
 - type variables, 194
 - uncurried version of function, 76
 - unification, 120, 370
 - user-defined data types, 72, 73
 - value constructors, 73
 - values, 66
 - variables, 298
 - wildcard pattern, 416
 - wildcards, 68
- mnemonic symbols, 5
- mode declaration, 625
- models of computation, 8
- modification by extension, 143
- Modula-2
 - exported declarations, 522
 - expressions not including function calls, 31
 - modules, 521–524
 - nested procedures, 465–466
 - subranges, 363
- Modula-3, 343
- modules, 10, 493
 - ADT (abstract data type), 500–502
 - versus* classes, 192–193
 - controlled scopes, 501
 - data types, 524, 527–528
 - documenting dependencies, 502
 - earlier languages, 519–524
 - implementations, 501
 - imported types, 529–531
 - instantiates, 520
 - libraries, 501
 - name proliferation, 501
 - program decomposition, 501
 - semantics, 531–532
 - static entities, 524–527
- monads, 80, 82
- monitors, 588, 607–615
 - concurrency and, 611–615
 - condition variables, 609
 - java.concurrent.locks package, 610–611
 - mutual exclusion, 608–609
 - synchronized objects, 609–610
- monomorphic, 372
- Moore's Law, 7–8
- most closely nested rule, 412
- most general assertion, 567
- MPI (Message Passing Interface), 616
- MPMD programming, 592
- multidimensional arrays, 343–344
- multi-dispatch problem, 194
- Multilisp, 624
- multimethods, 167, 194
- multiple inheritance, 190–191
- multiply-typed values, 362–363
- multiprocessor system, 582
- multiprogramming, 582
- mutators, 146
- mutual exclusion, 585, 608–609
- N**
 - name capture problem, 92
 - name equivalence, 355–357, 383, 385
 - name proliferation, 501
 - name resolution, 282–288
 - names, 257
 - associating to values, 54
 - attributes, 257–258
 - bound to attributes, 258–259
 - bound to locations, 289
 - level number, 262
 - lexical address, 262
 - offset, 262
 - overloading, 287–288
 - namespaces, 279, 506–509
 - as classes, 192–193
 - narrowing conversions, 365
 - native threads, 595
 - natural numbers, 105
 - natural semantics, 548
 - Naur, Peter, 204
 - nested blocks, 266
 - nested procedures, 465–466, 468
 - nested scopes, 310
 - nesting depth, 468
 - nodes, 631
 - nondeterminism, 132–133
 - nonfunctional languages expressions, 402
 - non-imperative languages and parallelism, 622–631
 - nonlocal declarations, 261
 - nonlocal references, 447–448
 - allocated statically, 465
 - closure, 459
 - scope rules, 447
 - nonmonotonic reasoning, 128
 - nonprocedure block, 449
 - nonstrict evaluation, 408
 - nonstrict functions, 77, 94
 - non-tail-recursive procedure, 60–61
 - nonterminals, 210–211, 233
 - derivation, 215
 - Follow sets, 232
 - nonterminal symbol, 244
 - normal order evaluation side effects, 410
 - Norwegian Computing Center, 142
 - notation, 116
 - numbers as tokens, 235–236
 - numeric integer types, 334
 - numeric types, 350–351
 - Nygaard, Kristen, 142
- O**
 - object identity, 167
 - Objective-C, 192
 - object-oriented languages, 15, 46
 - allocation, 197
 - automatic garbage collection, 167, 473
 - classes, 10, 192–193, 263, 269, 336
 - conversion requirements, 366
 - design issues, 191–195
 - double-dispatch problem, 194
 - dynamic binding, 155, 196–197
 - dynamic capabilities, 191
 - exceptions, 425
 - garbage collection, 308
 - history, 142
 - implementation issues, 195–197
 - inheritance, 194–197, 339–340
 - initialization, 197
 - iterators, 12
 - methods, 195–196
 - modules, 192–193
 - multi-dispatch problem, 194
 - multimethods, 167, 194
 - objects, 195–196
 - polymorphism, 194–195, 372
 - runtime binding, 155
 - runtime environment, 191
 - runtime penalty, 191
 - translator, 191
 - types, 192
 - object-oriented paradigm, 15
 - object-oriented programming, 46, 142
 - ADT (abstract data types), 143, 493
 - C++, 181–191
 - code reuse and modifiability, 142
 - encapsulation mechanisms, 144
 - independence, 143–144
 - information-hiding mechanisms, 144
 - inheritance, 153, 362
 - Java, 162–181
 - polymorphism, 153
 - reference semantics, 148
 - Smalltalk, 144–162
 - software reuse, 143–144
 - objects, 142, 195–196
 - allocating, 197
 - behaviors, 145
 - cloning, 306
 - environment, 292
 - extent, 292
 - implementation of, 195–196
 - instantiation, 163
 - interface, 146
 - lifetime, 292
 - message passing, 8
 - pointers, 292–293
 - properties, 145
 - wait-list, 599
 - occam, 616
 - occur-check, 127, 375
 - offset, 262, 464
 - one-dimensional arrays, 57
 - opcode, 4
 - open conditions, 618
 - operands, 388–389, 403
 - operating systems
 - integrating operation of processors, 586–587
 - running programs in parallel, 591
 - operational semantics, 17, 542, 547–556
 - abstract machine, 547
 - assignment, 551–553
 - compilers, 547
 - control, 553–554
 - definitional interpreters, 547
 - environments, 551–553
 - executable specification, 555
 - implementing, 555–556
 - logical inference rules, 548

- reduction machine, 547
- reduction rules, 548–551
- operations
 - atomically, 604
 - extensions, 143
 - grouping, 206
 - redefinition, 143
- operator overloading
 - Ada, 286–287
 - C++, 167, 285–286
 - Java, 167
- operators, 403
 - associativity, 403
 - infix, 287, 403
 - inline code, 403–404
 - left-associative, 221, 228
 - mix-fix form, 408
 - operands, 408
 - overloaded, 282, 363
 - postfix, 403
 - precedence, 403
 - prefix, 403
 - right-associative, 221
 - ternary, 408
- ordered, 332
- ordinal types, 334
- or-parallelism, 623–624
- or special form, 77
- orthogonal, 30–31
- orthogonality, 31–32
- output function, 70–71
- overlapping types, 362–363
- overloaded functions, 86–90, 283, 371
 - boxing and tagging, 376
 - expansion, 376
 - symbol table, 282–283
- overloaded operators, 282, 363
- overloading, 194, 282–288
 - function names, 282
 - names, 287–288
 - operators, 282, 287
 - polymorphism, 371–372
 - scope, 281
- overload resolution, 282, 284
- owners, 616

P

- packages, 10
 - Ada, 193, 380, 509–515
 - automatically dereferencing name, 512
 - automatic namespaces, 512
 - classes, 193
 - Java, 163, 167, 279, 506–509
 - package body, 381, 509
 - package specification, 509
- package specification, 380–381, 509–510
- paradigms, 15
- parallelism, 108, 582–583
 - and-parallelism, 622
 - message passing, 626–631
 - non-imperative languages, 622–631
 - or-parallelism, 623
 - procedure-level, 593
 - program-level, 594–595
 - statement-level, 593
- parallel matrix multiplication, 588
- parallel processing, 582, 587–595

- distributed-memory system, 585–586
- library of functions, 590–591
- MIMD (multiple-instruction, multiple-data)
 - systems, 584–585
- procedure-level parallelism, 593–594
- process creation and destruction, 591–592
- processes, 583–584
- shared-memory system, 585
- SIMD (single-instruction, multiple-data)
 - systems, 584–585
- statement-level parallelism, 593
- parallel programming
 - bounded buffer problem, 588
 - message passing, 588, 615–621
 - monitors, 588, 608–615
 - parallel matrix multiplication, 588
 - producer-consumer problem, 588
 - program-level parallelism, 594–595
 - semaphores, 588, 604–608
 - threads, 588, 595–604
- parameter declarations, 479–480
- parameter list, 450
- parameter modes, 390, 479–481
- parameter-passing mechanisms
 - delayed evaluation, 455–458
 - versus* parameter specification, 458
- pass by name, 455–458
- pass by reference, 452–454
- pass by result, 454
- pass by value, 451–452
- parameters, 13, 48, 450
 - alias for argument, 452
 - checking references to, 480
 - functions, 65, 450
 - pass by need, 78
 - procedures, 450
 - type checking, 459
 - user-defined functions, 77
- parameter specification *versus* parameter-passing
 - mechanisms, 458
- parametric polymorphism, 194, 372
 - generic collections, 169
- parenthesized expressions, 50–52
- parent processes, 591
- Parlog, 625
- parser generators, 224, 232–233
- parsers
 - bottom-up, 224, 233
 - parsing shell, 240
 - predictive, 230–231
 - recognizer, 224
 - shift-reduce, 224
 - top-down, 230
 - type descriptors, 384
- parse trees, 213–216
 - BNF notation, 221
 - derivations, 216–218
 - terminals and nonterminals, 215
- parsing, 110, 224–235
 - bottom-up, 224
 - methods, 243–246
 - procedure, 225
 - recursive-descent, 225
 - single-symbol lookahead, 230
 - top-down, 224
- parsing methods, 243
- parsing shell, 240

- partial correctness, 574
- partial function, 47
- Pascal, 7
 - assignment operator (*:=*), 147
 - associating type with name, 328
 - constants, 31
 - declaration, 9
 - declaration equivalence, 358
 - definitions, 257
 - disambiguating rule, 412
 - functions, 31
 - gotos, 421
 - implicit types, 362
 - nested procedures, 465–466
 - pass by reference, 452–453
 - pass by value, 452
 - pointers restricted, 33
 - predefined identifiers, 258–259
 - procedures, 31
 - procedure variables, 472
 - reference arguments, 454
 - strongly typed, 331
 - subranges, 363
 - symbolic constant, 432
 - symbol table, 278
 - type equivalence, 358
- pass by name, 77–79, 410, 455–458
- pass by need, 78
- pass by reference, 452–454
- pass by result, 454
- pass by value, 451–452
- pass by value-result, 454
- pattern-directed invocation, 120
- pattern matching, 68, 338
- patterns and data constructors, 73
- Perl
 - dynamic scoping, 277
 - shell languages, 39
 - untyped languages, 331
- pipes, 591
- PL/I exception handling, 423
- pointers, 292–293, 300, 346–349
 - dangling references, 306–307
 - dereferenced, 346
 - exception handlers, 477
 - imperative languages, 348
 - implicit, 346
 - manual dynamic allocation, 348
 - recursive types, 347
- pointer semantics, 49, 299
- pointer variables, 304–306
- polymorphic expression, 371
- polymorphic functions, 69–70
- polymorphic type checking, 367–376
- polymorphic types, 376
- polymorphic typing, 131
- polymorphism, 144, 150, 153, 331
 - ad hoc, 194
 - explicit, 376–382
 - versus* inheritance, 194–195
 - Java, 167–175
 - overloading, 371–372
 - parametric, 194, 372
 - Smalltalk, 154–155
 - subtype, 194
- pool dictionaries, 152
- portability, 36, 39

- positional property, 210
- postconditions, 565–566
- postfix operators, 346, 403
- PostScript, 404
- precedence, 216–220
- preconditions, 565–566
- predecessor, 332
- predefined identifiers, 205, 236, 258–259, 332
- predefined operators, 405
- predefined types, 87, 332
- predicate calculus, 106–107, 110
- predicates, 105, 107, 116, 360, 497
- predicate transformers, 543, 567
- predictive parsers, 230–232
- prefix functions, 83
- prefix operators, 403–404
- prime numbers, 81
- primitive functions, 55
- primitive types, 32, 163, 166, 350–351
- principle of extensionality, 535–536
- principle of longest substring, 205
- principle type, 374
- private inner class, 171
- private members, 181
- procedural interpretation, 110
- procedure calls, 13, 462
- procedure declaration, 12, 447
- procedure-level parallelism, 593–594
- procedures, 12–13, 444, 612
 - activation, 12, 445–447
 - activation records, 449, 462
 - activations, 459–472
 - actual parameters, 13
 - advanced inlining process, 455
 - allocation, 459–472
 - arguments, 13, 445
 - body, 445, 450
 - called, 13, 448
 - callee, 446
 - caller, 446
 - calling, 445–446
 - closed form, 450
 - communication, 450
 - creating another procedure, 471–472
 - defining environment, 468
 - definition, 445–447
 - dynamically computed, 469–472
 - environments, 459–472
 - executing, 612
 - filters, 80–81
 - formal parameters, 445
 - generators, 80–81
 - interface, 445
 - invocation, 12
 - lexical scope, 449, 467
 - nested, 465–466, 468
 - nonlocal references, 447
 - parameter-passing mechanisms, 451–459
 - parameters, 13, 450
 - return-statement, 446
 - semantics, 447–451
 - specification, 445
 - symbolic name, 447
- processes, 14, 583–584
 - blocked, 583
 - busy-wait, 608
 - children, 591
 - continue call, 609
 - creation and destruction, 591–592
 - deadlock, 585–586
 - executing, 583
 - granularity, 592
 - heavyweight and lightweight, 583–584
 - parent, 591
 - spawn, 627
 - spin, 608
 - starvation, 608
 - suspending, 609
 - synchronizing, 604
 - waiting, 583
- processors
 - assigning, 589
 - communication problem, 585–586
 - integrating operation, 586–587
 - mutual exclusion, 585
 - organization, 584
 - race condition, 585
- producer-consumer problem, 588
- producer threads, 601
- productions, 210–211
- program-level parallelism, 594–595
- programmer efficiency, 29–30
- programming, 46
 - declarative style, 131
- programming languages
 - abstractions, 7, 8–14
 - affected by parallelism, 582–583
 - algebraic notation, 6
 - ALGOL family, 6–7
 - API (application programming interface), 10
 - assembly language, 4–6
 - assignment, 299
 - automatic memory management, 346
 - axiomatic semantics, 565–571
 - basic data abstractions, 8–9
 - BNFs (Backus-Naur forms), 204
 - compilers, 18
 - computation without von Neumann architecture, 7–8
 - data, 326
 - denotational definition, 557
 - denotational semantics, 556–565
 - derivation, 209–210
 - documentation, 16
 - dynamic memory allocation at runtime, 29
 - evolution of, 20
 - explicit types, 327
 - expressions, 402
 - expressiveness, 29
 - extensibility, 34–35
 - formal definitions, 16–17
 - fully curried, 76
 - functional paradigm, 15
 - function application, 47–48
 - function definition, 47–48
 - functions, 446
 - future of, 19–21
 - generality, 30–31
 - goals, 26
 - grammar, 16
 - identifiers, 257
 - imperative, 15
 - interpreters, 18
 - lexics (lexical structure), 17, 204–208, 236
 - logic, 20, 104
 - logic paradigm, 15
 - machine dependent, 326–327
 - machine language, 3–4
 - metasymbols, 209
 - models of computation, 8
 - monomorphic, 372
 - object-oriented paradigm, 15
 - operational semantics, 547–556
 - origins, 3–8
 - orthogonal, 30–32
 - paradigms, 15
 - parallel processing, 587–595
 - parameters, 78
 - precise semantic descriptions, 542
 - predefined types, 332
 - procedures, 446
 - regularity, 30–33
 - reliability, 30
 - security, 33–34
 - semantics, 16–17, 204, 257–315
 - special-purpose, 26
 - static typing, 328
 - strict, 77
 - strongly typed, 331
 - syntax, 16–17, 29–30, 204
 - tokens, 17, 204–208
 - translators, 18–19
 - uniformity, 31–32
 - von Neumann bottleneck, 15
 - von Neumann model, 15
 - weakly typed, 331
- programs
 - byte code, 18
 - compilation, 18
 - dangling references, 308
 - erroneous, 458
 - functional languages, 131
 - as functions, 46, 47–50
 - garbage, 308
 - identifiers, 205
 - legal, 331
 - logic language, 131
 - mathematical description of behavior, 542
 - predefined identifiers, 205
 - proofs of correctness, 542, 571–574
 - reliability, 327
 - robustness, 424
 - security, 327
 - syntactic structure, 204
 - unsafe, 331
- projection functions, 335
- Prolog, 104, 115–126
 - and-parallelism, 622, 624
 - appending lists, 120
 - arithmetic terms, 117–118
 - backtracks, 120–123
 - clauses, 120
 - compilers, 116
 - constants, 116, 626
 - control structures, 122–126
 - cut operator, 124–126, 129
 - data structures, 116
 - depth-first search, 122
 - equality, 118–119
 - Euclid's algorithm, 118
 - evaluating arithmetic, 117–118

- execution, 116–117
- functions, 116
- Horn clauses, 115, 129
- infinite loops, 122, 124, 127, 129
- interpreters, 116–117
- lists, 116, 119
- logical variables, 134, 626
- loops, 122–126
- notation, 116
- or-parallelism, 623–624
- parallelism, 624–625
- pattern-directed invocation, 120
- pattern matching, 116
- predicates, 116
- repetitive searches, 122–123
- resolution, 115, 121
- search strategy, 121–122
- shorten clauses, 120
- sorting lists, 132
- sorting program, 130
- standard predicates, 116
- subgoals, 121
- unification algorithm, 118–121, 127
- uninstantiated variables, 119
- variables, 116
- Prolog programs
 - append list operation, 119
 - control information, 129–131
 - definite clause grammars, 110
 - execution, 116–117
 - reverse list operation, 119
- promise to evaluate arguments, 78
- proofs of program correctness, 571–574
- propagating the exception, 430
- properties, 145
- protected objects, 612
- prototypes, 260, 445, 502–503
- proved, 105
- pseudoparallelism, 583
- public members, 181
- pure functional program, 48
- pure lambda calculus, 91
- pure polymorphism, 372
- pure virtual declaration, 187–188
- pure virtual functions, 188–189
- PVM (Python virtual machine), 39–40
- Python
 - application-specific libraries, 39
 - automatic garbage collection, 34
 - byte code, 18
 - data types, 38
 - dictionary, 38
 - doc, 10
 - dynamic typing vs. finger typing, 39–40
 - extensibility, 34, 38–39
 - functions, 31
 - general-purpose scripting language, 38–40
 - importance, 26
 - indentation, 205
 - interactivity, 39
 - machine-independent byte code, 39
 - modules, 10, 263
 - objects, 32
 - platforms, 39
 - portability, 39
 - predefined identifiers, 205
 - predefined operators, 31
 - primitive operations and data types, 38
 - reference semantics, 300
 - reference types, 32
 - regularity, 38–39
 - scripts, 39
 - semantically safe, 33–34
 - shorthand notation, 10
 - simplicity, 38–39
 - static type-checking, 33
 - syntax, 29–30, 38
 - type checking, 40
 - typeless variables, 29
 - untyped languages, 331
 - variable declarations, 33
 - variable names, 261
 - virtual machine, 38
- Python virtual machine. *See* PVM (Python virtual machine)
- Q**
 - Qlisp and-parallelism in let-bindings, 624
 - quantifiers, 106
 - queries, 108
 - Queue data type, 505
 - queues
 - linked lists, 170–172
 - linked nodes, 160–162
 - QUICKSORT algorithm, 27
 - quote keyword, 53
 - quotient algebra, 533
- R**
 - race condition, 585
 - raised, 423
 - range, 47
 - raw collections, 169
 - readability, 327
 - real numbers, 326–327
 - real types, 351
 - receive process, 627
 - receiver object, 162
 - receivers, 145, 149
 - reclamation of storage, 475–476
 - recognizer, 224, 232
 - records, 335
 - record structure, 335–336
 - recursion, 56–57, 131
 - functional programming, 48–49
 - recursive data types, 73
 - recursive declarations, 268
 - recursive-descent parser, 227
 - recursive-descent parsing, 110, 225–227, 241
 - recursive functions, 8, 55, 74, 347
 - recursive routines, 29
 - recursive subroutines, 29
 - recursive types, 346–349
 - reduced, 224
 - reduction machine, 547
 - reduction rules, 548
 - control, 553–554
 - environments, 551–553
 - integer arithmetic expressions, 548–551
 - lambda calculus, 90
 - reductions and transitivity rule, 550
 - reference counting, 472, 475–476
 - references, 163, 346, 361
 - reference semantics, 32, 147–148, 166, 299
 - reference types, 32, 166, 169, 350–351
 - collections, 169
 - equality operator (==), 167
 - reference semantics, 32
 - referential transparency, 49, 409
 - regular expressions, 206
 - regularity, 30–33, 38–39
 - reinterprets, 366
 - reliability, 30, 33, 327
 - Remote Method Invocation. *See* RMI (Remote Method Invocation)
 - Remote Procedure Call. *See* RPC (Remote Procedure Call)
 - remove ambiguities, 327
 - rendezvous, 616–621
 - repeated inheritance, 191
 - reserved words, 205, 236
 - resolution, 115, 121
 - Horn clauses, 111–112
 - resumption model, 431
 - returned values, 446
 - return-statement, 446
 - reusability, 10
 - reusable code, 15
 - reverse list operation, 119
 - reverse operation, 59
 - rev function, 70
 - right associative, 218–219
 - right-associative operators, 221
 - right-recursive rule, 219, 227
 - RMI (Remote Method Invocation), 616
 - robustness, 424
 - row-major form, 344
 - RPC (Remote Procedure Call), 616
 - Ruby, 20, 331
 - runtime binding, 155
 - runtime environments, 13, 39
 - dynamic scoping, 277
 - language design, 308
 - r-value, 298, 361
- S**
 - safe union mechanism, 338
 - same-fringe problem, 80–81
 - sample language, 543–547
 - abstract syntax, 546
 - axiomatic semantics, 569–571
 - empty environments, 545
 - environments, 544–545
 - loops, 546
 - proofs of program correctness, 571–574
 - semantics of arithmetic expressions, 544
 - statements, 545
 - type nomenclature, 349–315
 - save queue, 630
 - scalar types, 163, 351
 - scanner generator, 235
 - scanners, 210
 - getToken procedure, 225
 - token categories, 235–236
 - Scheme, 28, 50
 - ad hoc stream constructions, 80
 - applicative order evaluation, 53, 77
 - atoms, 50, 51
 - binding list, 54
 - box and pointer notation, 58–59
 - data structures, 57–60

Scheme (*cont.*)

- define special form, 55
- delayed evaluation, 54
- delay special form, 78
- dynamic type checking, 55–56
- elements of, 50–55
- environment in, 52
- evaluation rules, 51–53, 66
- expressions, 50–54
- extended Backus-Naur form, 51
- force special form, 78
- functions, 31, 54–55
- heap allocation and deallocation, 295
- higher-order functions, 61–62
- if form, 54
- interpreter, 63
- lambda forms, 54–55, 148
- let form, 54
- letrec form, 55
- lexical scoping, 277
- lists, 53, 57–60
- loops, 54
- memorization process, 78
- metalinguistic power, 63–64
- non-tail recursion, 56–57
- pass by value, 54
- pointer manipulation, 295
- predicates, 360
- primitives, 55, 59
- quote special form, 53
- recursion, 56–57, 60
- semantics, 54
- special forms, 52, 77
- statically scoped, 62–63
- strings, 57
- symbolic information processing, 63–64
- symbols, 51, 53, 63
- syntax, 51
- tail recursion, 56–57
- type checking, 359–360
- vector type, 344
- scope, 260–269
 - bindings, 263–265
 - block-structured languages, 62
 - declarations, 263–265, 269–270
 - exception handlers, 429
 - overloading, 281
 - scope holes, 266
 - symbol table, 280
 - variables, 62–63
- scope analysis, 269, 309–312
- scope hole, 266
- scope of a binding, 263–264
- scope resolution operator (::), 182, 266, 267
- scope rules
 - dynamic scoping, 273
 - exception declarations, 426
 - nonlocal references, 447
 - recursive declarations, 268
 - static scoping, 273
- scoping structure, 279
- search strategy, 121–122
- section, 82
- security, 33–34
 - data types, 493
 - ML (MetaLanguage), 65
 - programs, 327

- selectors, 146, 497
- semantically safe, 33–34
- semantic domains, 556–558
- semantic equations, 559
- semantic functions, 257–260, 559
- semantics, 16–17, 204, 257–315
 - algebraic specification, 535
 - aliases, 303–306
 - allocation, 289–297
 - attributes, 257–260
 - axiomatic semantics, 543
 - binding, 257–260
 - blocks, 260–269
 - BNF rules, 543
 - constants, 300–303
 - correctness rules, 360
 - dangling references, 303–307
 - declarations, 260–269
 - denotational semantics, 542
 - environment, 260, 289–297
 - expressions, 402
 - formal and informal, 17
 - formal definition, 257
 - garbage, 303–309
 - identifiers, 257
 - lifetimes, 289–297
 - locations, 257
 - memory, 260
 - name resolution, 282–288
 - names, 257
 - operational semantics, 542
 - overloading, 282–288
 - procedures, 447–451
 - scope, 260–269
 - semantic functions, 257–260
 - specifying, 257
 - state, 260
 - static expressions, 432–433
 - store, 260
 - symbol table, 259, 269–281
 - type inference, 360
 - values, 257
 - variables, 297–300
 - vs. lexics vs. syntax, 235–236
- semaphores, 588, 604–609
- sender, 145
- send process, 627
- sentinel-based loops, 422–423
- separate compilation, 502–506
- sequence operator, 406
- sequence types, 340
- server, 628–629
- set equation, 213
- set! special form, 57
- shadow, 266
- shared inheritance, 191
- shared-memory model, 585, 604
- shared operations, 363–364
- shell languages, 39
- shift-reduce parsers, 224
- short-circuit Boolean operations, 408
- short-circuit evaluation, 77, 406–407
- side effects, 305
 - expressions, 402, 405–406
 - normal order evaluation, 410
 - order of evaluation, 405–406
 - pass by name, 457

- sieve of Eratosthenes algorithm, 81, 85
- Signal operation, 604, 609
- signatures, 86, 494, 515–518
 - free algebra of terms, 533–534
- SIMD (single-instruction, multiple-data) systems, 584–585
- simple types, 332–334
- Simula, 142, 181
- Simula67, 35, 142
- single-entry, single-exit constructs, 402
- single-instruction, multiple-data systems. *See* SIMD (single-instruction, multiple-data) systems
- single-processor models of computation, 8
- single program multiple data. *See* SPMD programming (single program multiple data)
- single-symbol lookahead, 230
- Smalltalk, 144–162
 - abstract classes, 151
 - based-object approach, 195
 - basic elements, 145–150
 - binary messages, 147
 - blocks, 148–149
 - classes, 145, 150–157, 155, 192, 263
 - class hierarchy, 150–157
 - class messages, 146
 - class names, 146
 - class variables, 152
 - collection classes, 150
 - collection methods, 176
 - collections, 157–162
 - comments, 146
 - concrete classes, 150–151
 - control statements, 148–149
 - deallocation routines, 197
 - Dynabook Project, 144
 - dynamic binding, 155, 176
 - dynamic typing, 145
 - function variables, 345
 - garbage collection, 145
 - instance messages, 146
 - instance methods, 155
 - instance variables, 152, 155
 - iterators, 150, 158–159
 - keyword messages, 146–148
 - lambda form, 177
 - literal array, 148
 - Magnitude hierarchy, 150–157
 - message passing, 146
 - messages, 145, 150
 - methods, 145, 152, 470
 - mutator, 146
 - objects, 145, 167, 197
 - parameters, 345
 - polymorphic methods, 176
 - polymorphism, 154–155
 - pool dictionaries, 152
 - queues, 160
 - receivers, 145
 - reference semantics, 147–148, 300
 - runtime binding, 155
 - sender, 145
 - unary messages, 146
 - user interface, 145
 - variables, 147, 261
 - WIMP (windows, icons, mouse, pull-down menus) user interface, 144
- Smalltalk-72, 145

Smalltalk-76, 145
 Smalltalk-80, 142, 145
 Smalltalk/V, 145, 146
 SML97, 65
 SML (Standard ML), 65
 SNOBOL, 26, 277
 software reuse, 143–144
 sorting, 532–533
 free algebra of terms, 533–534
 lists, 132
 procedure, 129–130
 source program, 18
 spaghetti code, 421
 special forms, 52
 specializes, 374
 special symbols, 205
 specifications, 110–111, 445, 543
 spin, 608
 spin-locks, 608
 SPMD programming (single program multiple data), 591
 stack-based allocation, 294
 stack-based environment, 462–471
 stack-based languages, 404
 stack frame, 448
 stack unwinding, 431
 Standard ML. *See* SML (Standard ML)
 Standard Template Library. *See* STL (Standard Template Library)
 start symbol, 209–211
 starvation, 608
 state, 260
 statement-level parallelism, 593
 statements, 402, 444
 conditional, 410–416
 sample language, 545
 single-entry, single-exit, 402
 statically typed languages, 360, 367
 static binding, 175–176, 186–191
 binding times, 258
 data types, 277
 static constants, 301–302
 static environment, 289, 449
 static expressions, 390, 432–433, 435–436
 static link, 467
 static scope, 62–63, 273
 static semantic analysis, 259, 390
 static type checking, 33, 181, 327, 359
 static types, 327
 static typing, 39, 65–76
 STL (Standard Template Library), 37
 stop and copy, 476
 storage class, 296
 storage semantics, 49, 300
 store, 260
 streams, 80, 85
 strict evaluation, 404, 626
 strict functions, 94
 strings, 57
 iterators, 158
 predefined identifiers, 236
 reading and writing, 71
 reserved words, 236
 strongly typed language, 327, 331
 type checking parameters, 459
 unsafe program, 331
 Stroustrup, Bjarne, 27, 36, 181

Stroustrup, Mjarne, 35–38
 structs, 278–279
 structural equivalence, 352–355
 structural operational semantics, 548
 structure construction, 335
 structured abstractions, 8–9
 structured control, 402
 abstractions, 11–14
 constructs, 402
 statements, 7
 structured language exceptions, 425
 structure member operation, 335
 structures, 343, 515–518
 subexpressions, 405
 subgoals, 112, 121
 subprograms, 12
 subrange types, 332, 334, 384, 432
 subroutines, 12, 446, 459–460
 subset types, 339
 substitution, 92
 substitution rule, 409
 subtype polymorphism, 194, 372
 subtypes, 339
 successor, 332
 Sun Microsystems, 162
 suspend operations, 609–610
 Swing Toolkit, 10, 144
 switch-statements, 414–416
 symbolic, 301
 symbolic constants, 432–435
 symbolic information processing, 53, 63–64
 symbolic logic, 15
 symbols, 53, 63, 269
 symbol table, 259, 269–281, 309–311
 block declarations, 269
 compiler, 273
 declarations, 269, 273
 dynamically managed, 273–274
 dynamic scope, 274–277
 overloaded functions, 282–283
 scope, 279–281
 static declarations, 273
 structure, 269–273
 type names, 313
 synchronized methods, 609
 synchronized objects, 609–610
 synchronizing processes, 604
 synchronous exceptions, 424
 syntactic domains, 556–558
 syntactic specification, 494
 syntactic sugar, 10, 417
 syntax, 16–17, 204
 abstract syntax, 216
 analysis, 259, 313
 concrete syntax, 216
 parameter modes, 479
 static expressions, 432–433
 structure, 213
 syntax-directed semantics, 214
 vs. lexics vs semantics, 235–236
 syntax analyzer, 237–246
 syntax diagrams, 204, 220–224
 EBNF rules, 243–244
 recursive-descent parser, 227
 syntax-directed semantics, 214
 syntax trees, 216–218
 type checker, 368–370

T

tags, 336
 tail operator. *See* tl (tail) operator
 tail recursive, 56, 60
 target language, 18
 target program, 18
 task entries, 616
 tasks, 14, 263, 594, 611–612
 entries, 616–617
 master, 619
 parallel matrix multiplication, 619
 rendezvous, 616–621
 semaphore type, 619
 template classes, 184–186
 templates, 194, 378–379, 506
 terminals, 210, 215–216
 termination model, 431
 terms, 106–107
 ternary operator, 408
 TestAndSet machine instruction, 607–608
 theorems, 107–108
 threads, 14, 584, 588, 595–604
 bounded buffer problem, 601–604
 consumer, 601
 daemons, 596
 defining, 596
 destroying, 597
 fork-join parallelism, 597
 interrupting, 597
 producer, 601
 running, 596
 sharing memory or resources, 597–598
 waiting for conditions and resuming execution, 599–600
 thrown, 423
 thinks, 456
 TinyAda
 assignment statements, 383
 component references, 389–390
 data types, 313
 declarations, 239, 386–388
 EBNF grammar, 237–239
 expressions, 239, 388–389
 grammar rules, 243
 identifiers, 309, 310–312, 312–315, 382–385
 name equivalence, 383, 385
 nested scopes, 310
 parameter modes, 390, 479–481
 parser, 242–243, 383
 procedure calls, 389–390
 procedure declaration, 239, 311
 recursive rules, 239
 scope, 309–312
 semantic qualifiers, 237–239
 statements, 239
 static expressions, 390, 432–436
 static semantic analysis, 309–315, 390
 symbolic constants, 432, 433–435
 symbol table, 309–311
 syntax analyzer, 237–246
 TEST program, 239–240
 tokens, 240
 type checking, 382–390
 type descriptors, 383–386
 type equivalence, 383
 tl (tail) operator, 67–68
 token delimiters, 205

tokens, 17, 204–208, 210, 233, 240–241
 categories, 235–236
 constants, 205
 identifiers, 205
 keywords, 205
 literals, 205
 numbers as, 235–236
 principle of longest substring, 205
 regular expressions, 206
 reserved words, 205
 special symbols, 205
 token delimiters, 205
 white space, 205
 YACC/Bison, 235
 top-down parsers, 230
 top-down parsing, 224
 transitivity rule, 550
 translation, 259
 translation efficiency, 327
 translators, 18–19, 191
 bindings, 258–259
 code inlining, 404
 compiler options, 589
 parsing phase, 204
 processors, 589
 scanning phase, 204
 type checking, 331
 validated, 542
 true parallelism, 582
 try-catch blocks, 427–428
 tuples, 67, 72, 76, 83, 336
 tuple type, 66
 Turing machines, 90
 Turner, David A., 81
 twos complement notation, 4
 type checker, 367–370
 type checking, 330, 359–360, 382–390
 dynamic, 359
 Hindley-Milner type checking, 367–370
 implicit types, 361–362
 multiply-typed values, 362–363
 overlapping types, 362–363
 parameters, 459
 polymorphic, 367–376
 shared operations, 363–364
 static, 359–360
 translators, 331
 type inference, 360
 unification, 370
 type-checking algorithm, 358–359
 type classes, 86–90
 type class inheritance, 87–88
 type compatibility, 361, 383
 type constructors, 192, 330, 335–349, 351
 arrays, 340–345
 Cartesian products, 335–336
 classes, 192
 data types, 330
 functions, 340–345
 union, 336–339
 user-defined, 377
 type conversion, 364–367
 type correctness rules, 361
 type declaration, 330
 type definition, 330, 386
 type descriptors, 383–386, 387–389
 type equivalence, 352–359, 383

 name equivalence, 355–357
 structural equivalence, 352–355
 type equivalence algorithm and type-checking algorithm, 358–359
 type equivalence algorithms, 331, 360
 type form, 384
 type inference, 330, 360
 Hindley-Milner type checking, 367–370
 type information, 328–331
 declarations, 386–388
 recording, 477
 type names, 349–351, 354
 type parameters, 168–169, 372
 type recognition functions, 56
 type rules, 331
 type variables, 67, 83, 194, 369–370, 375

U

unary messages, 146
 unary operator, 403
 unconstrained arrays, 342
 indiscriminated unions, 336
 unification, 111–115, 370
 Curry, 133–134
 equality, 118–119
 functional languages, 370
 Haskell, 120
 logic programming, 133
 ML (MetaLanguage), 120
 occur-check problem, 127, 375
 Prolog, 118–121
 uniformity, 31–32
 uninstantiated variables, 119
 unions, 336–339
 unit abstractions, 8–10
 units, 14
 universally quantified variables, 110
 universal quantifier, 106
 University of Edinburgh, Scotland, 104, 115
 University of Glasgow, Scotland, 81
 UNIX operating system, 20, 26
 pipes, 591
 YACC (yet another compiler compiler), 233
 unordered collections, 157
 unsafe programs, 331
 unsynchronized methods, 609
 untyped languages, 331
 USENIX (UNIX users' association), 37
 user-defined data types, 72, 73, 89, 492
 user-defined exceptions, 424
 user-defined functions, 77, 404–405, 492
 user-defined operators, 500
 user-defined polymorphic types, 83
 user-defined type constructors, 377
 user-defined types, 330–331

V

validated, 542
 valuation functions, 557
 value constructors, 73
 values, 54–56, 257
 defining, 65–66
 implicitly pointers or references, 452
 literals, 301
 multiply-typed, 362–363
 operations applied to, 329
 value semantics, 32, 49, 147, 166, 300–301
 van Rossum, Guido, 35, 38–40

variable annotations, 625
 variable binding, 626
 variable declarations, 33, 163
 variable-length tokens, 205
 variables, 9, 48, 105, 297–300
 bound, 48, 63
 bound by quantifier, 106
 bound by the lambda, 91
 bound occurrence, 91–92
 box-and-circle diagram, 297
 concept of, 48
 data types, 9
 declarations, 32, 356
 dereferenced, 293, 361
 free, 63, 91–92, 106
 Horn clauses, 111
 instantiated, 114, 128
 lambda calculus, 91–92
 l-value, 298
 names, 9
 reference semantics, 147–148
 r-value, 298
 schematic representation, 297
 scope, 62–63
 storage class, 296
 unification, 114
 value, 298
 values, 298
 value semantics, 147
 visible declaration, 62–63

variant record, 337
 vector type, 344
 virtual functions, 186–191
 virtual machines, 18
 visibility, 266
 visibility by selection, 267
 von Neumann, John, 3
 von Neumann bottleneck, 15, 582
 von Neumann model, 7, 15

W

waiting, 583
 weakest assertion, 567
 weakest precondition, 567
 weakly typed language, 331
 while loop, 34–35, 173, 423
 while special form, 57
 while statements
 approximation, 573
 axiomatic semantics, 571
 Whitehead, A. N., 6
 white space, 205
 widening conversions, 365
 wildcard pattern, 416
 wildcards, 68
 WIMP (windows, icons, mouse, pull-down menus)
 user interface, 144
 Wirth, Niklaus, 7, 27, 204, 326
 writability, 27, 327

Y

YACC/Bison, 224, 233–235
 Yale University, 81
 yet another compiler compiler. *See* YACC (yet another compiler compiler)

Z

zero-overhead rule, 36